

# Phase-Based Planning Model for Software Work in a Human–Agent Cell

Aleksandr Rodionov  
Yandex, Saint Petersburg, Russia  
`rexarrior@yandex.ru`  
ORCID: 0009-0006-6710-923X

## Аннотация

Рассматривается фиксированный объём программной работы, который один разработчик выполняет с помощью нескольких ИИ-агентов. Предложена детерминированная фазовая модель, связывающая эффект ИИ на отдельных задачах, граф зависимостей, число управляемых агентных потоков, интеграционные издержки и последовательный ресурс человеческого внимания. Иерархия M0–M4 отделяет идеально делимую работу от неделимых задач, критического пути, координационных издержек и точного расписания чередующихся агентных и человеческих фаз с локальной блокировкой ожидающего агента.

Для M4 получена структурная нижняя граница срока  $B_4$ , объединяющая загрузку потоков, критический путь и человеческий ресурс, и введено ресурсно-дополненное фазовое представление точного makespan. Вычислительный пакет на основе CP-SAT воспроизводит иллюстративные сценарии и показывает, что  $B_4$  не всегда достижима: на замороженной точной сетке разрыв  $T^* > B_4$  возник в 30 из 90 сценариев. На исследованной синтетической сетке растущие издержки конфигурации приводят к конечному оптимуму полезного параллелизма, положительные издержки декомпозиции — к U-образному профилю точного срока, а interaction contrast параллелизма и гранулярности преимущественно положителен, хотя дробление не всегда даёт абсолютный выигрыш. Результаты являются вычислительной верификацией и анализом чувствительности модели, а не эмпирической оценкой производительности конкретного инструмента или команды.

**Language note.** В текущей рабочей версии заголовки и визуализации оформлены на английском языке, а основной текст сохранён на русском. Полный перевод статьи на английский язык планируется в одной из следующих редакций.

## 1 Introduction

Влияние ИИ-инструментов на производительность разработки программного обеспечения нельзя описать одним универсальным коэффициентом ускорения. В контролируемом эксперименте с GitHub Copilot участники, завершившие ограниченную задачу, в среднем затратили на неё на 55.8% меньше календарного времени, чем участники контрольной группы [26]. В рандомизированном исследовании опытных разработчиков, работавших над задачами в знакомых зрелых проектах с открытым исходным кодом,

доступ к инструментам начала 2025 года, напротив, увеличил скорректированное активное время реализации на 19% [4]. В продолжении этого исследования отмечено вероятное улучшение новых агентных инструментов, но одновременно выявлена более глубокая методическая проблема: разработчики выбирали задачи с учётом доступности ИИ и параллельно работали с несколькими агентами, поэтому длительность отдельной задачи уже не характеризовала производительность всего процесса [5]. Эти результаты относятся к разным задачам, инструментам, участникам и метрикам и потому не образуют прямого противоречия. Вместе они показывают, что вопрос об эффективности ИИ необходимо ставить для явно заданного способа работы, а не для модели или продукта вне организационного контекста.

Для планирования разработки этого уточнения недостаточно. Даже достоверная оценка длительности отдельной задачи с ИИ ещё не определяет срок набора связанных работ. Автономные интервалы нескольких агентов могут перекрываться, но задачи конкурируют за ограниченное число потоков, следуют зависимостям, требуют интеграции и периодически обращаются к одному разработчику за постановкой, уточнением, проверкой или приёмкой. В результате локальное ускорение может не изменить общий срок, если не сокращает активное ограничение, а дополнительный агент может не создать полезного параллелизма. И наоборот, даже режим, в котором отдельная задача выполняется дольше ручного baseline, может сократить срок всего набора работ благодаря перекрытию автономных фаз. Следовательно, между измеренным эффектом ИИ на задаче и календарным результатом проекта находится самостоятельная задача организации и расписания процесса.

Настоящая работа рассматривает одну *человеко-агентную ячейку*: один разработчик выполняет заранее заданный AI-поддержанный набор программных задач с помощью до  $P$  агентных потоков и доводит каждую задачу до фиксированного критерия приёмки. Исходным сценарием служит последовательное выполнение того же объёма одним разработчиком без ИИ. Различаются интерактивный, делегированный и полностью автономный режимы. В интерактивном режиме человек и инструмент совместно ведут задачу в одном потоке; в делегированном режиме постановка и приёмка требуют человека, а между ними возможны автономные агентные фазы; в полностью автономном режиме обязательная человеческая фаза отсутствует. Основной предмет статьи — интерактивные и делегированные задачи. Полностью автономный режим допускается как предельный случай с нулевой обязательной человеческой работой, но обнаружение новых задач и автоматическая приёмка результатов не моделируются.

Для делегированных задач принимается правило локального закрепления: начатая задача удерживает назначенный агентный поток до принятия результата. Если агент запрашивает внимание занятого разработчика, он ожидает и не получает другую задачу, тогда как остальные агенты продолжают работу независимо. Новая задача может начаться только в свободном потоке, а зависимая задача — только после приёмки всех предшественников. Такое правило описывает конкретную архитектуру процесса, а не любые системы агентов. Переиспользование заблокированного потока, preemption, несколько разработчиков, отдельные ограниченные ресурсы CI/API, динамическое раскрытие графа работ и межпроектное планирование находятся за границами постановки. Выбор полностью ручного или AI-поддержанного способа для отдельных задач также считается внешним решением; модель планирует уже выбранный AI-поддержанный набор.

Исследовательский вопрос формулируется следующим образом: *при каких условиях полностью заданная конфигурация работы одного разработчика с несколькими*

*ИИ-агентами сокращает время завершения фиксированного набора связанных программных задач по сравнению с последовательной работой без ИИ и какие ограничения определяют достижимое ускорение?* Конфигурация задаёт число агентных потоков и режим каждой задачи; содержательно полное сравнение должно также фиксировать либо явно изменять декомпозицию, граф зависимостей, фазовые профили, критерий приёма и функции издержек. Целевой показатель — makespan принятого результата. Качество и стоимость вычислений учитываются только через их влияние на длительность проверки, исправления и завершения задачи и не выступают самостоятельными целями оптимизации.

Основной тезис статьи состоит в том, что **достижимое ускорение является свойством полностью заданного человеко-агентного процесса, а не производением “скорости ИИ” на номинальное число агентов**. Срок может ограничиваться суммарной загрузкой потоков, неделимой задачей, технологическим критическим путём, последовательным человеческим ресурсом либо смешанной ресурсной цепочкой, возникающей из порядка фаз и очереди к разработчику. Поэтому сокращение задачи, не лежащей на текущем критическом пути, всё же может быть полезно, если оно освобождает дефицитный поток; увеличение доступной мощности может уменьшать срок до насыщения, но переход к конфигурации с большим настроенным  $P$  способен ухудшить его, если одновременно растут интеграционные издержки или стоимость переключения внимания. Декомпозиция действует столь же неоднозначно: она снимает искусственную сериализацию и открывает параллельную работу, но добавляет границы постановки, проверки и интеграции.

Для ответа на исследовательский вопрос предложена детерминированная фазовая модель M0–M4. Нормированная длительность  $k_i^{(r)}$  относится к тройке “задача — инструмент — режим” и сравнивает полный цикл получения принятого результата с базовой длительностью той же задачи. Это описательная нормировка, а не причинная оценка без отдельного идентификационного дизайна. На уровне M4 она раскладывается на автономную агентную и человеческую составляющие  $a_i^{(r)}$  и  $h_i^{(r)}$ , а их внутренний порядок задаётся последовательностью фаз. Иерархия начинается с идеально делимой агрегированной работы M0 и затем последовательно учитывает неделимость задач M1, граф зависимостей M2, координационные издержки M3 и точное расписание фаз с общим человеческим ресурсом и локальной блокировкой M4. Такая конструкция отделяет простые необходимые ограничения от комбинаторных потерь конкретного расписания.

Работа вносит четыре связанных результата.

Во-первых, задана операциональная единица сравнения: не ИИ-инструмент сам по себе, а полный сценарий “задачи — режимы — ресурсы — критерий приёма”. Это позволяет отделить наблюдаемую длительность задачи от прогноза, доступную мощность от настроенной конфигурации и автономную агентную фазу от полностью автономного режима.

Во-вторых, для уровней M0–M2 получены условия, связывающие нормированные длительности, параллелизм и структуру работ. На M0 ускорение эквивалентно  $P > \bar{k}_w$ . На M1–M2 это условие остаётся необходимым, а классическая граница списочного планирования даёт достаточный критерий через относительную длину самой длинной задачи или критического пути. Тем самым явно разделяются идеальная оценка ускорения, структурная нижняя граница и достижимый срок.

В-третьих, для M4 введена структурная нижняя граница

$$B_4 = \max\{W_4/P, L_4, \gamma(P)H\},$$

и доказан потолок ускорения, задаваемый последовательным человеческим ресурсом. Граница объединяет загрузку потоков, исходный критический путь и суммарную human-work, но в общем случае не равна оптимальному makespan  $T^*$ . Ресурсно-дополненный фазовый граф даёт точное представление  $T^*$  и показывает, как порядок обслуживания человека и порядок задач на агентных слотах создают смешанные пути, отсутствующие в исходном task-DAG. Это различает нижнюю границу, доказанный оптимум и срок конкретной политики  $T(\pi)$ .

В-четвёртых, воспроизводимый вычислительный пакет с точным решателем и фиксированной детерминированной политикой проверяет внутреннюю логику модели на сериях EXP-00–EXP-08. Он воспроизводит иллюстративные сценарии, предъявляет контрпримеры универсальной достижимости  $B_4$ , разделяет эффект доступной мощности и растущих конфигурационных издержек, исследует гранулярность и её взаимодействие с  $P$ , а также проверяет масштабную устойчивость и перенос направлений эффектов на синтетические DAG. Эти результаты являются вычислительной верификацией и анализом чувствительности формальной модели, а не эмпирической оценкой конкретной команды или ИИ-инструмента.

Математические элементы этой конструкции по отдельности известны: неделимые работы, критический путь, ограниченные ресурсы, общий обслуживающий ресурс и дуги ресурсного порядка принадлежат теории расписаний. Заявляемый вклад состоит не в повторном введении этих объектов, а в их последовательном отображении на полный цикл программной задачи с task/mode-specific коэффициентом  $k$ , отдельными agent/human-фазами, повторными обращениями к разработчику, удержанием агентного потока, издержками интеграции и критерием принятого результата. Эмпирическая калибровка такого отображения остаётся открытой задачей.

Статья построена в соответствии с этой логикой. После постановки и компактных обозначений сопоставляются эмпирические оценки ИИ-продуктивности, модели масштабирования, теория расписаний и исследования человеческого внимания. Затем вводится иерархия M0–M4 и точное ресурсное представление. Вычислительная часть сначала фиксирует метод и правила доказательности, после чего разбирает иллюстративный сценарий и серии EXP-00–EXP-08. Обсуждение связывает формальные и вычислительные результаты с практическим использованием модели; отдельные разделы фиксируют ограничения, угрозы валидности и направления развития. Полный словарь, подробные расчёты и дополнительные результаты вынесены в приложения.

## 2 Problem Setting and Compact Notation

Этот раздел даёт компактный содержательный обзор постановки и ключевых обозначений; формальные определения и детерминированные предпосылки вводятся в разделе *Formal Model*. Модель описывает фиксированный объём программной работы, доводимый до заранее заданного критерия приёма. Исходным сценарием служит последовательное выполнение того же объёма одним разработчиком без ИИ. Полный словарь терминов и развёрнутая система обозначений приведены в Приложении А.

### Задача и режим.

Задача  $i$  имеет сложность  $Z_i$  и выполняется в режиме  $r$ : интерактивном, делегированном либо полностью автономном. Режим задаёт порядок постановки, агентного исполнения, проверки и исправления результата. Основной анализ

М4 относится к интерактивному и делегированному режимам; полностью автономный режим без обязательной human-фазы служит предельным случаем.

#### Нормированная длительность с ИИ.

Величина  $k_i^{(r)} = T_i^{ai}(r)/(Z_i X)$  сравнивает длительность принятого результата с базовым временем той же задачи. Она относится к тройке «задача – инструмент – режим», а не к ИИ-инструменту вообще. Это описательная нормировка; причинная интерпретация требует отдельного дизайна сравнения.

#### Фазы.

Нормированная длительность раскладывается как  $k_i^{(r)} = a_i^{(r)} + h_i^{(r)}$ . Автономные agent-фазы требуют агентного слота, human-фазы — одновременно внимания разработчика и сохранения назначенного слота.

#### Управляемый параллелизм.

Параметр  $P$  обозначает число агентных потоков, настроенных в данной конфигурации и доступных расписанию; для этого значения оцениваются координационные издержки. Неиспользуемая дополнительная мощность при неизменной конфигурации сама по себе overhead не создаёт.

#### Конфигурация.

Запись  $\sigma = (P, r)$  или  $\sigma = (P, \rho)$  является сокращённой при фиксированных ИИ-инструменте и правилах приёмки. Если инструмент или критерий принятого результата меняются, их необходимо явно указывать при сравнении, хотя они не добавляются в размерность  $\sigma$ .

#### Расписание и срок.

Допустимое расписание соблюдает зависимости DAG, порядок фаз, мощность человеческого ресурса 1 и локальное удержание слота. Его срок обозначается  $T(\pi)$ , а минимальный срок среди допустимых расписаний —  $T^*$ .

| Обозначение            | Смысл                                                    |
|------------------------|----------------------------------------------------------|
| $X$                    | базовая трудоёмкость единицы работы без ИИ               |
| $Z_i$                  | сложность задачи $i$ в единицах $X$                      |
| $k_i = a_i + h_i$      | нормированная длительность и её agent/human-составляющие |
| $P$                    | управляемое число агентных потоков конфигурации          |
| $G = (V, E)$           | ориентированный ациклический граф задач                  |
| $W = X \sum_i Z_i k_i$ | суммарная работа при $C = \gamma = 1$                    |
| $A, H$                 | суммарная автономная агентная и человеческая работа      |
| $C(P), \gamma(P)$      | множители межагентных и человеческих издержек            |
| $T(\pi), T^*$          | срок конкретного расписания и оптимальный makespan       |

## 3 Related Work

### 3.1 Empirical Estimates of AI Productivity

Эмпирические исследования не позволяют задать единый коэффициент влияния ИИ на разработку. В эксперименте Peng и соавт. участники, завершившие ограниченную задачу на JavaScript с помощью GitHub Copilot, в среднем затратили на неё на 55.8% меньше календарного времени, чем завершившие задачу участники контрольной

группы [26]. Результат относится к одной задаче, ранней версии Copilot и выборке, включавшей только завершивших работу участников; параллельное применение нескольких агентов не изучалось. В полевом рандомизированном исследовании Becker и соавт. доступ к инструментам начала 2025 года, напротив, увеличил скорректированное активное время реализации на 19% у опытных разработчиков зрелых проектов с открытым исходным кодом [4]. Здесь измерялось активное время, а не календарный срок всего набора работ; кроме того, экспериментальное условие включало неоднородные инструменты и способы их применения. Поэтому оценки относятся к разным участникам, задачам, инструментам, правилам включения и показателям результата и не противоречат друг другу.

Различия в эффекте наблюдаются и вне программирования. При внедрении генеративного помощника в службе поддержки Brynjolfsson и соавт. оценили средний рост пропускной способности, измеренной числом решённых обращений в час, в 15% [9]. Эффект зависел от опыта сотрудника и типа обращения, а пропускную способность динамического потока нельзя напрямую преобразовать во время фиксированной программной задачи. В терминах модели оценки  $k_i^{(r)}$  сопоставимы только при одинаковых границах задачи, исходной конфигурации и критериях приёмки. Календарное время (elapsed time), активное время (active time), пропускная способность (throughput), успешность (success), качество (quality) и понимание результата (comprehension) остаются различными показателями.

Исследования взаимодействия объясняют часть неоднородности. Vaithilingam и соавт. не обнаружили статистически значимого выигрыша Copilot по времени или доле успешно выполненных трёх коротких задач, но выявили затраты на понимание, проверку, редактирование, отладку и переписывание сгенерированного кода [31]. Barke и соавт. показали, что программисты переключаются между использованием ИИ для ускорения работы и для исследования возможного решения [3]. Поэтому режим  $r$  описывает полный способ постановки, исполнения, проверки и исправления, а  $h_i^{(r)}$  включает отложенную проверку и повторную работу.

Переход к агентам программирования (coding agents) особенно ясно требует разделять время и качество. В исследовании Valepur и соавт. агент ускорял выполнение исходной веб-задачи и повышал её точность по сравнению с ограниченным чат-ботом, но ухудшал понимание кода; при последующем расширении без агента устойчивого преимущества по времени или точности не обнаружено [2]. Отсутствие условия без ИИ не позволяет оценить  $k$ . Телеметрия миллионов сессий агента GitHub Copilot также показывает длинные цепочки вызовов инструментов, слабый внутренний параллелизм и повторные попытки, но не связывает сессии с фиксированными принятыми задачами или действиями человека [24]. Наконец, METR отмечает, что выбор задач с учётом доступности ИИ и параллельная работа с несколькими агентами смещают оценку эффекта на уровне отдельной задачи [5]. Эти результаты обосновывают отдельное измерение автономных и человеческих интервалов и сравнение полных конфигураций рабочего процесса.

### 3.2 Effort Estimation as an External Model Layer

Оценка трудозатрат решает близкую, но самостоятельную задачу. Boehm, Abts и Chulani систематизируют параметрические, регрессионные, аналоговые, экспертные и комбинированные методы прогнозирования трудозатрат, стоимости и длительности [6]. Они оценивают ещё неизвестные входные данные, но сами по себе не назначают

неделимые задачи потокам, не проверяют ресурсную допустимость и не минимизируют makespan фиксированного набора работ. Сопоставление формальных моделей и экспертных оценок не выявляет универсально лучшего подхода [20]; систематическая карта 304 журнальных публикаций также фиксирует преобладание ретроспективной проверки и недостаточное внимание к неопределённости и перспективной проверке (prospective validation) [21]. Поэтому наблюдаемое после задачи  $k_i^{(r)}$  не является готовым прогнозом. До проекта требуется внешняя локально откалиброванная оценка  $\hat{k}_i^{(r)}$  с явно заданной неопределённостью; модель расписания преобразует эти оценки в календарные ограничения.

### 3.3 Scaling, Orchestration, and Coding-Agent Architectures

Для фиксированного объёма работ закон Амдала связывает предельное ускорение с последовательной долей [1]. Здесь используется только структурная аналогия: последовательным ресурсом служит измеренное участие человека, а не участок машинной программы. Универсальный закон масштабируемости Гантера допускает насыщение и отрицательную отдачу из-за конкуренции за ресурсы (contention) и согласования состояния (coherency) [18]; его знаменатель является гипотезой для  $C(P)$ , а не эмпирически установленным законом для агентов программирования. Рассуждение Брукса о коммуникации и концептуальной целостности аналогично обосновывает интеграционные издержки [7], но число связей в человеческой команде нельзя буквально переносить на топологию «один разработчик — несколько агентов».

В универсальных испытаниях многоагентных систем (multi-agent systems, MAS) Kim и соавт. сравнили 260 конфигураций и обнаружили зависимость успешности от делимости задачи, последовательных зависимостей, базовой модели и топологии [22]. Однако результатами служили точность и успешность, человек не участвовал, а число агентов менялось вместе с архитектурой. Это подтверждает зависимость координационных издержек от структуры, но не оценивает  $C(P)$  или makespan рассматриваемой человеко-агентной ячейки.

Системы, работающие с репозиториями, дают более близкие, но всё ещё неполные аналоги. MAGIS разделяет роли планировщика, хранителя репозитория, разработчика и контроля качества и допускает автоматизированные циклы проверки и доработки [30]. Его основной результат — доля решённых задач SWE-bench; четыре роли не означают четыре одновременно работающих потока, а автоматизированный контроль качества не является человеческим ресурсом. CAID, напротив, строит план с учётом зависимостей и действительно исполняет агентов-разработчиков конкурентно в изолированных рабочих деревьях Git с последующим слиянием, тестированием и итоговой проверкой [15]. В каждой из шести основных пар «семейство модели — набор задач» CAID повышал показатель качества, но одновременно увеличивал календарное время выполнения и стоимость. Сравнение конфигураций с 2, 4 и 8 агентами показывало немонотонную зависимость качества от их числа и рост издержек. Следовательно, CAID подтверждает различие между заданным числом агентов, фактической конкурентностью и полезным параллелизмом, но не свидетельствует об ускорении и не содержит человеческих фаз, условия без ИИ или разложения  $a_i/h_i$ .

Таким образом, следует различать внутреннее параллельное использование инструментов одним агентом, одновременное исполнение независимых задач, число именованных ролей и архитектуру координации. Фактическую конкурентность уславливают только наблюдаемые перекрывающиеся интервалы при неизменном усло-

вии сравнения; показатели успешности и качества не заменяют измерения makespan.

### 3.4 Scheduling Theory and Repeated Shared Service

Математическое ядро уровней M1–M2 стандартно. Для одинаковых параллельных машин используются makespan, суммарный объём работы, длительность самой длинной работы и критический путь [28]; списочное планирование при отношениях предшествования и метод LPT для независимых работ имеют классические гарантии [16, 17]. Дизъюнктивные дуги, или дуги ресурсного порядка (resource-ordering arcs), также являются стандартным способом представлять конкуренцию машин [28]. Поэтому статья не заявляет новизной makespan, ориентированный ациклический граф (DAG), критический путь, ресурсно-дополненный граф или общий ресурс.

Задача календарного планирования с ограниченными ресурсами (RCPSP) объединяет DAG отношений предшествования с возобновляемыми ресурсами конечной мощности, а многорежимные постановки допускают альтернативные профили длительности и ресурсов [8]. В таком ядре внимание можно представить ресурсом мощности 1, а фазы — отдельными операциями. Однако стандартная модель не задаёт коэффициент  $k$  для программной задачи и режима, смысл принятого результата, операционное разложение  $a_i/h_i$  или внутренние интеграционные издержки и издержки переключения.

Ещё ближе модели параллельных машин с общим обслуживающим устройством (common server). В постановках только с подготовкой обслуживающее устройство последовательно подготавливает работы перед параллельной обработкой [19, 23]. В постановке с загрузкой и выгрузкой (loading/unloading) каждая работа использует то же устройство до и после машинной обработки; требование немедленной выгрузки может удерживать завершившую обработку машину до освобождения устройства [14]. Это формальный аналог последовательности «постановка человеком — работа агента — проверка человеком» и частичный аналог удержания потока, но без DAG и повторного цикла «доработка агентом — проверка человеком».

Ещё более близкая возвратная постановка (reentrant scheduling) проводит работу через назначенную основную машину, удалённый сервер и затем возвращает её на ту же основную машину [10]. Она воспроизводит чередование «исполнитель — общее обслуживание — тот же исполнитель», но содержит лишь одно повторное обращение к серверу, независимые работы и заранее заданный маршрут. Существенно, что во время удалённого обслуживания основная машина освобождается; поэтому эта модель не воспроизводит локальную блокировку M4. Ни модели с общим сервером, ни возвратная постановка сами по себе не содержат коэффициент  $k$  для программной работы, человеческую приёмку и интеграционные издержки.

Сопоставление уточняет вклад M4: известные механизмы теории расписаний используются как ядро, а предлагаемое отображение на программный процесс связывает полный цикл «постановка человеком — работа агента — проверка человеком — доработка агентом — приёмка человеком» с режимом задачи, отдельными длительностями фаз, удержанием назначенного агентного потока и фактом принятия результата. Если человеческая фаза не удерживает поток,  $A$  и  $H$  следует планировать как разные ресурсы; это соответствует другому правилу использования ресурсов.



### 3.5 Human Attention in Supervising Autonomous Workers

Исследования взаимодействия человека и робота (human-robot interaction, HRI) дают содержательный аналог одного оператора и нескольких автономных исполнителей. Показатель числа роботов на одного оператора (fan-out) связывается с продолжительностью взаимодействия и автономной работы без внимания оператора [25]; модели многозадачного управления требуют укладывать обслуживание одного робота в автономные интервалы остальных [12]. Распределение внимания с учётом состояния показывает зависимость результата от правила выбора и состояния системы, а не только от средней доли участия [11]. Теория многопоточного познания (threaded cognition) дополнительно предупреждает, что не вся человеческая активность единообразно последовательна: интерференция возникает на отдельных когнитивных ресурсах [29].

Cummings и Mitchell проверили расширенную модель числа аппаратов на одного оператора в имитационном эксперименте с 12 участниками и несколькими беспилотными летательными аппаратами (unmanned aerial vehicles, UAV) [13]. Учёт ожидания взаимодействия, ожидания в очереди, ожидания восстановления ситуационной осведомлённости и когнитивной переориентации снизил расчётную оценку пропускной способности оператора на 36–67% в рассмотренных условиях; авторы прямо подчёркивают зависимость результата от задачи и контекста. Эксперимент поддерживает последовательную очередь обслуживания и конечность полезного числа автономных исполнителей только как структурную аналогию. Он не калибрует число агентов программирования,  $\gamma(P)$  или makespan программной работы: аппараты были однородными и независимыми, а результатом являлась расчётная оценка пропускной способности, а не срок выполнения принятой программной задачи.

### 3.6 Synthesis and Bounded Empirical Gap

Таблица 1 показывает, что отличие статьи не сводится к добавлению DAG или общего ресурса. Ближайшие частичные аналоги распределены между несколькими направлениями: RCPSP задаёт отношения предшествования и ограничения мощности ресурсов; модели с общим сервером и возвратным обслуживанием — повторное обслуживание и разные правила удержания машины; HRI — очередь и переключение одного оператора; универсальные MAS — влияние топологии координации; MAGIS — автоматизированный цикл проверки и доработки; CAID — фактически конкурентные ветви, слияние и тестирование. Каждый из этих элементов известен отдельно.

В основном корпусе, сформированном по поисковому протоколу с датой завершения 2026-08-09, не обнаружена работа, которая одновременно калибровала бы на программных задачах коэффициент  $k$  для задачи и режима, автономные и человеческие интервалы, DAG зависимостей, фактическую конкурентность, повторные обращения к единственному человеку, локальное удержание агентного потока, интеграционные издержки и издержки переключения, а также календарный makespan принятого результата. Это утверждение относится только к просмотренному корпусу и каналам поиска, указанным в протоколе; оно не является универсальным утверждением обо всей литературе.

Ближайшая программная система CAID наблюдает конкурентность, календарное время выполнения и интеграцию, но не включает человека и условие без ИИ. MAGIS содержит автоматизированную проверку и доработку, но не устанавливает фактическую конкурентность или человеческое обслуживание. Модели с общим сервером

и возвратным обслуживанием формализуют повторный сервис, но не отображают программный процесс; HRI измеряет ожидание человека, но не программную работу; универсальные MAS оценивают преимущественно успешность и качество, а не makespan человеко-агентной ячейки.

Формальный вклад настоящей работы состоит в совместном представлении перечисленных величин и проверке свойств модели на синтетических сценариях. Статья сама не выполняет эмпирическую калибровку реального рабочего процесса. Сбор наблюдений по принятым программным задачам и перспективная проверка (prospective validation) того, предсказывает ли совместно оценённая M4 календарный makespan лучше более простых уровней, остаются будущей работой.

Таблица 1: Ближайшие классы постановок и область настоящей модели. Значения «да», «частично» и «нет» показывают наличие свойства в классе постановок, а не эмпирическую эквивалентность или новизну.

| Класс постановок                                                                       | DAG      | Потоки исполнителей | Сервис мощности 1 | Повторный сервис | Удержание потока | $k$ задачи и режима | Разделение $a_i/h_i$ | Издержки координации и интеграции | Основной результат                                 | Калибровка реального программного процесса |
|----------------------------------------------------------------------------------------|----------|---------------------|-------------------|------------------|------------------|---------------------|----------------------|-----------------------------------|----------------------------------------------------|--------------------------------------------|
| RCPSP и многорежимная RCPSP [8] <sup>a</sup>                                           | да       | частично            | да                | частично         | нет              | нет                 | нет                  | частично                          | makespan, стоимость, допустимость                  | нет                                        |
| Параллельные машины с общим сервером [19, 23, 14, 10] <sup>b</sup>                     | нет      | да                  | да                | частично         | частично         | нет                 | частично             | нет                               | makespan, алгоритмические оценки                   | нет                                        |
| HRI: число роботов на оператора и распределение внимания [25, 12, 11, 13] <sup>c</sup> | нет      | да                  | да                | да               | частично         | нет                 | частично             | частично                          | пропускная способность, выполнение миссии          | нет                                        |
| Универсальные MAS и централизованная координация [22] <sup>d</sup>                     | нет      | частично            | нет               | частично         | нет              | нет                 | нет                  | частично                          | успешность, качество; иногда стоимость             | нет                                        |
| Агенты программирования: MAGIS и CAID [30, 15] <sup>e</sup>                            | частично | частично            | нет               | частично         | нет              | нет                 | нет                  | частично                          | успешность, качество; CAID также время и стоимость | нет                                        |
| Настоящая работа <sup>f</sup>                                                          | да       | да                  | да                | да               | да               | да                  | да                   | да                                | makespan принятого результата                      | нет                                        |

<sup>a</sup> Возобновляемый ресурс RCPSP не обязан представлять одинаковые агентные потоки; повторное обслуживание и издержки требуют явных операций, а стандартная постановка не вводит автоматическое удержание потока в ожидании другого ресурса.

<sup>b</sup> Повторный сервис присутствует в вариантах с загрузкой/выгрузкой и возвратом, но не в постановках только с подготовкой. Немедленная выгрузка может удерживать машину, тогда как возвратная модель с удалённым сервером освобождает её; машинная и сервисная длительности не откалиброваны как доли работы агента и человека.

<sup>c</sup> Автономная система ожидает оператора или теряет результативность, однако это не равнозначно удержанию программного потока. Интервалы взаимодействия и автономности не являются программными  $a_i/h_i$ ; очередь, потеря ситуационной осведомлённости и переориентация не включают интеграцию кода.

<sup>d</sup> Последовательная взаимозависимость измеряется, но явный DAG задач не строится; число агентов или ролей не доказывает перекрытие интервалов. Централизованный координатор не равнозначен наблюдаемому сервису мощности 1, повторные сообщения — человеческому циклу, а различия результатов — калибровке издержек M4.

<sup>e</sup> CAID строит план зависимостей и показывает перекрывающиеся ветви; для MAGIS и класса в целом эти свойства не установлены. Автоматизированные планировщик, контроль качества и доработка не являются человеческим сервисом; ожидание зависимостей или слияния не тождественно локальной блокировке. CAID наблюдает слияние, тестирование, время и стоимость, но не калибрует человеко-агентный процесс. Оценивание на программных наборах задач не является эмпирической калибровкой реального рабочего процесса с наблюдаемыми человеческими фазами.

<sup>f</sup> В статье используются синтетические сценарии; эмпирическая калибровка и перспективная проверка остаются будущей работой.

Статья не заявляет новизной DAG, makespan, общий ресурс, повторный сервис, возврат к исполнителю, удержание ресурса, дуги ресурсного порядка или число автономных исполнителей на одного оператора.

## 4 Formal Model

### 4.1 Formal Definitions and Deterministic Assumptions

**Definition 1.** Пусть заданы следующие параметры:

- $X > 0$  — базовая трудоёмкость единицы работы (время реализации элементарной функциональности);
- $Z_i \in \mathbb{R}^+$  — сложность  $i$ -й задачи, измеренная в единицах  $X$ . Дробные значения допустимы: при детализации проекта подзадача может занимать долю базовой единицы работы (например,  $Z_i = 0.5$ ). В прикладном расчёте значения обычно выбираются из конечной дискретной шкалы  $D \subset \mathbb{Q}^+$ ;
- $r \in R$  — **режим** организации работы с ИИ, то есть способ взаимодействия разработчика с инструментом (например, интерактивная работа в одном диалоге, делегированное выполнение с автономной агентной фазой и последующей проверкой либо полностью автономное выполнение без обязательной human-фазы). Основной анализ M4 относится к интерактивному и делегированному режимам; полностью автономный режим служит предельным случаем. Конечное множество режимов  $R$  задаётся при применении модели;
- $k_i^{(r)} \in \mathbb{R}^+$  — нормированная длительность  $i$ -й задачи при использовании ИИ в режиме  $r$ . Величина является свойством тройки «задача  $\times$  инструмент  $\times$  режим» и определяется операционально:

$$k_i^{(r)} = \frac{T_i^{ai}(r)}{Z_i X},$$

где  $T_i^{ai}(r)$  — фактическое время выполнения  $i$ -й задачи с применением ИИ в одном рабочем потоке режима  $r$  при полной доступности разработчика, то есть без конкуренции с другими потоками за его внимание. Оно включает формулирование запросов, ожидание генерации, проверку и исправление результата. Значение  $k_i^{(r)} < 1$  соответствует более короткой длительности относительно базовой оценки  $Z_i X$ ,  $k_i^{(r)} = 1$  — совпадению с ней, а  $k_i^{(r)} > 1$  — более длинной длительности. Это описательная нормировка, а не причинная оценка эффекта ИИ без отдельного идентификационного дизайна. Если режим зафиксирован или ясен из контекста, верхний индекс опускается:  $k_i := k_i^{(r)}$ ;

- $h_i^{(r)}, a_i^{(r)}$  — **человеческая** и **агентная** составляющие коэффициента  $k_i^{(r)}$ . Для определения  $k_i^{(r)}$  время  $T_i^{ai}(r)$  измеряется по двум каналам. К первому относятся интервалы, в течение которых разработчик занят задачами: формулирует запросы, читает и проверяет результаты, направляет исправления. Ко второму относятся интервалы автономной работы агента, когда разработчик свободен: генерация, самопроверка и выполнение тестов. Внутри задачи эти интервалы могут чередоваться; их порядок задаётся фазами уровня M4. После нормирования обеих частей на  $Z_i X$  по построению получаем:

$$k_i^{(r)} = a_i^{(r)} + h_i^{(r)}, \quad 0 \leq h_i^{(r)} \leq k_i^{(r)}.$$

Текущая модель относит каждый учитываемый интервал ровно к одному из этих двух каналов; отдельные ограниченные ресурсы CI, API и внешней инфраструктуры в неё не входят. При  $P = 1$  это разбиение не влияет на длительность

задачи: без очереди к человеку она равна  $k_i^{(r)} Z_i X$  при любом соотношении составляющих. Различие становится существенным при  $P > 1$ : агентные этапы разных задач могут выполняться параллельно, тогда как все человеческие этапы обслуживает один разработчик (уровень М4, Замечание 10);

- $P \geq 1$  — настроенное число одновременно доступных агентных потоков; для исходного сценария без ИИ принимается  $P_h = 1$ . На уровнях М1–М4, где задачи и фазы неделимы,  $P$  является целым. Только в агрегированных формулах М0 величину  $P$  допустимо интерпретировать как эффективный, усреднённый за период параллелизм. Важно различать дополнительную *доступную* мощность и реально выбранную конфигурацию: неиспользуемый поток сам по себе не замедляет расписание, тогда как переход к конфигурации с большим  $P$  может изменить  $C(P)$  и  $\gamma(P)$ ;
- $\sigma$  — **конфигурация процесса**: пара  $\sigma = (P, r)$  либо, в общем случае, пара  $(P, \rho)$ , где  $\rho: \{1, \dots, N\} \rightarrow R$  — назначение режимов задачам (разные потоки работы могут вестись в разных режимах). Эта запись является сокращённой при фиксированных ИИ-инструменте и правилах приёма: если инструмент или критерий принятого результата меняются, их необходимо явно указывать при сравнении, хотя они не добавляются в размерность  $\sigma$ . Производные величины модели — суммарная работа  $W$ , средневзвешенный коэффициент  $\bar{k}_w$ , время  $T_{ai}$  и ускорение  $S_E$  — являются функциями конфигурации. Сравнение «вариантов организации работы» в модели означает сравнение конфигураций целиком: параметры  $P$  и  $\{k_i^{(r)}\}$  не варьируются независимо друг от друга (см. Замечание 3);
- $N \in \mathbb{N}$ ,  $N \geq 1$  — общее число задач в проекте;
- $T_h, T_{ai}$  — общее время выполнения всех задач без ИИ и с ИИ соответственно.

**Remark 1** (Task Independence). Сначала предполагается, что все задачи  $i = \overline{1, N}$  **не пересекаются**, то есть не используют общие ресурсы, не блокируют друг друга и могут выполняться параллельно без дополнительных затрат на синхронизацию. Это допущение позволяет использовать аддитивную модель времени. Зависимости между задачами и критический путь вводятся далее на уровне М2.

## 4.2 Model Hierarchy Overview

**Remark 2** (Model Hierarchy М0–М4). Уровни М0–М4 образуют последовательность уточнений одной постановки. Каждый следующий уровень снимает часть релаксаций и вводит одну или несколько связанных сторон модели; это не означает, что переход между любыми двумя соседними уровнями добавляет ровно одно ограничение.

| Уровень | Основное уточнение                                   | Нижняя оценка                        |
|---------|------------------------------------------------------|--------------------------------------|
| M0      | идеально делимая агрегированная работа               | $B_0 = W/P$                          |
| M1      | неделимость задач                                    | $B_1 = \max\{W/P, M_N\}$             |
| M2      | граф зависимостей                                    | $B_2 = \max\{W/P, L\}$               |
| M3      | эффективные веса и интеграционные издержки           | $B_3 = \max\{W_3/P, L_3\}$           |
| M4      | фазы, локальное закрепление, очередь и общий человек | $B_4 = \max\{W_4/P, L_4, \gamma H\}$ |

Формальные уточнения вводятся соответственно в Предположении 1 и Замечаниях 5–10.

### 4.3 Level M0: Ideally Divisible Work

**Lemma 1** (Baseline Time without AI). При последовательном выполнении задач разработчиком без использования ИИ общее время составляет:

$$T_h = \sum_{i=1}^N Z_i X. \quad (1)$$

**Assumption 1** (Idealized AI-Assisted Time Model (Level M0)). В рамках основной аналитической модели работа считается идеально делимой, а параллелизм распределён равномерно. Обозначим суммарный объём работы с ИИ:

$$W := X \sum_{i=1}^N Z_i k_i.$$

При использовании ИИ с коэффициентом параллелизма  $P$  общее время выполнения задач принимается равным:

$$T_{ai}^{(0)} = \frac{W}{P} = \frac{1}{P} \sum_{i=1}^N Z_i k_i X. \quad (2)$$

Все дальнейшие теоретические выводы строятся на этом допущении, если явно не указан иной уровень иерархии моделей (Замечание 2).

**Observation 1** (Speedup Criterion and Factor at Level M0). Обозначим средневзвешенную нормированную длительность (веса — сложности задач  $Z_i$ ):

$$\bar{k}_w := \frac{\sum_{i=1}^N Z_i k_i}{\sum_{i=1}^N Z_i}.$$

Подстановка Леммы 1 и Предположения 1 непосредственно даёт цепочку эквивалентностей

$$T_{ai}^{(0)} < T_h \iff \sum_{i=1}^N Z_i k_i < P \sum_{i=1}^N Z_i \iff P > \bar{k}_w. \quad (3)$$

Это не самостоятельный теоретический результат, а алгебраическая перезапись определения времени на уровне M0.

Коэффициент ускорения равен

$$S_E = \frac{T_h}{T_{ai}^{(0)}} = P \cdot \frac{\sum_{i=1}^N Z_i}{\sum_{i=1}^N Z_i k_i} = \frac{P}{\bar{k}_w}. \quad (4)$$

В частности, если  $k_i \leq 1$  для всех  $i$  и  $P > 1$ , то  $S_E \geq P > 1$ , т.е. ускорение гарантировано.

Кроме того, из  $\min_j k_j \leq \bar{k}_w \leq \max_j k_j$  следует двусторонняя оценка

$$\frac{P}{\max_i k_i} \leq S_E \leq \frac{P}{\min_i k_i},$$

позволяющая быстро оценивать диапазон ускорения без взвешивания по  $Z_i$ .

**Remark 3** (Speedup Factorization and Baseline Choice). Формула Наблюдения 1 допускает факторизацию

$$S_E = \underbrace{P}_{\text{организационный множитель}} \cdot \underbrace{\frac{1}{\bar{k}_w}}_{\text{инструментальный множитель}}.$$

Первый множитель отражает эффект параллельной организации работы, а второй — влияние ИИ на трудоёмкость задач. В качестве исходного сценария  $T_h$  намеренно выбрана последовательная работа одного разработчика ( $P_h = 1$ ). Тем самым модель отвечает на вопрос о том, что получает один разработчик при переходе к ИИ-агентам, а не сравнивает ИИ с командой из  $P$  человек. При  $k_i \equiv 1$  получаем  $S_E = P$ : такое ускорение целиком обусловлено организацией процесса и в принципе достижимо также командой из  $P$  людей. Основное допущение состоит в том, что  $P$  параллельных потоков обеспечивают ИИ-агенты под контролем одного и того же разработчика, без привлечения дополнительных специалистов. Издержки контроля учитываются на уровнях МЗ–М4: функция  $C(P)$  описывает межагентную интеграцию (Замечание 9), а член  $\gamma(P)H$  — человеческий ресурс и связанный с ним потолок  $S_E^{\text{ach}} \leq 1/\bar{h}_w$  (Замечание 10, Предложение 4). Для сравнения с командой из  $q$  разработчиков исходное время следует заменить на  $T_h(q) = T_h/q$  в идеализированном приближении; тогда условие превосходства процесса с ИИ принимает вид  $P/\bar{k}_w > q$ .

Факторизация справедлива для каждой фиксированной конфигурации  $\sigma = (P, r)$ , но её множители нельзя варьировать независимо. На практике изменение  $P$  часто сопровождается сменой режима  $r$ , например переходом от интерактивной работы к автономным агентам. Вместе с режимом меняется весь вектор  $\{k_i^{(r)}\}$  и, следовательно,  $\bar{k}_w$ . Поэтому нельзя оценить  $\bar{k}_w$  в одном режиме и использовать параллелизм другого, например измерить  $k_i$  при интерактивной работе и подставить  $P = 4$  для автономных агентов. Сравнивать следует конфигурации целиком:  $S_E(\sigma_1)$  и  $S_E(\sigma_2)$ .

**Remark 4** (Critical Parallelism Threshold for  $k_i > 1$ ). Из тождества Наблюдения 1

$$\sum_{i=1}^N Z_i k_i < P \sum_{i=1}^N Z_i$$

следует, что ускорение  $S_E > 1$  достигается тогда и только тогда, когда

$$P > \bar{k}_w,$$

где  $\bar{k}_w$  — средневзвешенная нормированная длительность, введенная в Наблюдении 1.

Подчеркнём: условие  $P > \bar{k}_w$  необходимо и достаточно только на уровне M0. На уровнях M1–M2 оно остаётся необходимым, но перестаёт быть достаточным: достижимое время дополнительно ограничено снизу самой длинной задачей ( $M_N$ , Замечание 5) и критическим путём ( $L$ , Замечание 6); содержательный достаточный результат для этих уровней даёт Теорема 1. На уровне M3 необходимое условие по суммарной занятости потоков принимает вид  $C(P)\bar{a}_w + \bar{h}_w < P$  (Замечание 9). На уровне M4 необходимы одновременно все три неравенства, задаваемые  $B_4 < T_h$ :  $W_4/P < T_h$ ,  $L_4 < T_h$  и  $\gamma(P)H < T_h$ . Они не являются достаточными, поскольку существуют экземпляры с  $T^* > B_4$  из-за локальных блокировок и очереди к человеку.

- Если  $k_i \equiv k > 1$  для всех  $i$  (ИИ равномерно замедляет выполнение), то критический порог упрощается до:

$$P_{\text{crit}} = k.$$

То есть параллелизм должен **строго превышать** коэффициент замедления, иначе использование ИИ нецелесообразно.

- Если коэффициенты  $k_i$  неоднородны, то порог определяется взвешенным средним: задачи большей сложности вносят больший вклад в  $\bar{k}_w$ . Это означает, что даже при наличии отдельных задач с  $k_i \gg 1$  ускорение возможно, если их вклад в общую сложность мал, а параллелизм достаточен для компенсации.
- В практическом применении  $\bar{k}_w$  характеризует длительность выбранной AI-поддержанной конфигурации для данного проекта. Процедура оценки приведена в Замечании 14. Если  $\bar{k}_w$  не меньше доступного  $P$ , эта конфигурация не даёт временного выигрыша. В этом случае следует:
  1. пересмотреть состав автоматизируемых задач, начиная с задач с наибольшими  $k_i$ ;
  2. увеличить допустимый параллелизм процесса, например за счёт асинхронного выполнения;
  3. изменить способ применения ИИ — постановку запросов, состав контекста и последующую обработку — так, чтобы снизить  $k_i$ .

Итак, модель задаёт не бинарную оценку полезности ИИ, а **количественное условие принятия решения**, которое связывает длительности в выбранном режиме ( $k_i^{(r)}$ ), структуру проекта ( $Z_i$ ) и доступный команде параллелизм ( $P$ ). Критерий применяется к конфигурации  $\sigma = (P, r)$  целиком: значения  $\bar{k}_w$  и  $P$  должны относиться к одному режиму работы (Замечание 3). Выбор между полностью ручным и AI-поддержанным выполнением отдельных задач лежит вне постановки; после такого выбора модель планирует только включённый AI-поддержанный набор.

## 4.4 Level M1: Task Indivisibility

**Remark 5** (Workload Lower Bound  $B_1$ ). Формула уровня M0 предполагает идеальное равномерное распределение нагрузки. В реальности для неделимых задач время выполнения не может быть меньше длительности самой длинной задачи, даже при сколь угодно большом  $P$ . Обозначим

$$M_N := X \max_{i \in 1, N} (Z_i k_i).$$



Более строгая оценка записывается как неравенство:

$$T_{ai} \geq B_1 := \max \left\{ \frac{W}{P}, M_N \right\}.$$

Здесь  $B_1$  — нижняя оценка времени завершения всех работ; точное значение зависит от распределения задач между целым числом исполнителей и может строго превышать  $B_1$  даже при оптимальном расписании.

## 4.5 Level M2: Dependencies and the Critical Path

**Remark 6** (Critical-Path Lower Bound  $B_2$ ). При наличии логических или ресурсных зависимостей вместо аддитивной модели используется графовая. Пусть  $G = (V, E)$  — ориентированный ациклический граф, вершины  $V$  которого соответствуют задачам, а дуги  $E$  задают отношения предшествования. Тогда общее время выполнения с ИИ удовлетворяет неравенству:

$$T_{ai} \geq B_2 := \max \left\{ \frac{W}{P}, L \right\},$$

где  $L$  — длина критического пути в  $G$  с весами  $Z_i k_i X$ . Любая отдельная задача образует путь в  $G$ , поэтому  $L \geq M_N$  и уровень M2 обобщает M1: при пустом множестве дуг ( $E = \emptyset$ ) выполняется  $L = M_N$  и  $B_2 = B_1$ . Равенство  $T_{ai} = B_2$  в общем случае не гарантируется, так как задача планирования на ограниченном числе исполнителей является NP-трудной.

### 4.5.1 Attainability and Speedup Guarantees for M1–M2

**Remark 7** (Attainability Guarantee: Graham’s Bound). На уровнях M1–M2, при целом числе  $P$  одинаковых исполнителей и  $C(P) \equiv 1$ , нижние оценки  $B_1$  и  $B_2$  можно дополнить гарантией достижимости. Из классического результата Грэма для списочного планирования с фиксированным приоритетом, при котором исполнитель не простаивает, если имеется готовая задача, следует неравенство [16]

$$T_{ai} \leq \frac{W}{P} + \left(1 - \frac{1}{P}\right) L \leq \left(2 - \frac{1}{P}\right) B_2.$$

Для независимых задач (уровень M1) те же соотношения выполняются с заменой  $L$  на  $M_N$  и  $B_2$  на  $B_1$ . Таким образом, оптимальное время завершения  $T^*$  находится в интервале

$$B_j \leq T^* \leq \left(2 - \frac{1}{P}\right) B_j, \quad j \in \{1, 2\}.$$

Следовательно, достижимое ускорение отличается от верхней оценки  $\bar{S}^{(j)} = T_h/B_j$  не более чем в  $2 - 1/P$  раза:

$$\frac{\bar{S}^{(j)}}{2 - 1/P} \leq S_E^{\text{ach}} \leq \bar{S}^{(j)}.$$

Для независимых задач при упорядочивании по невозрастанию длительностей (правило LPT) известна более сильная гарантия  $T_{\text{LPT}} \leq (4/3 - 1/(3P)) T^*$  [17]. Однако

эта константа относится к оптимуму  $T^*$ , а не к нижней границе  $B_j$ , и потому не переносится на приведённый выше интервал. Отношение  $T^*/B_1$  может превышать  $4/3 - 1/(3P)$ : например, для трёх задач равной длительности при  $P = 2$  имеем  $T^*/B_1 = 4/3 > 4/3 - 1/6$ . На уровнях М3–М4 эти гарантии непосредственно не распространяются, поскольку постановка Грэма не включает координационные издержки и общий человеческий ресурс. Такие ресурсы рассматриваются в более общих моделях теории расписаний, в частности в RCPSP и задачах параллельных машин с одним общим обслуживающим устройством [8, 19, 23].

**Theorem 1** (Sufficient Speedup Criterion at Levels M1–M2). Пусть  $C(P) \equiv 1$ ,  $P$  — целое число одинаковых исполнителей. Обозначим относительную длину узкого места:

$$\mu := \frac{M_N}{T_h} \quad (\text{уровень M1}), \quad \mu := \frac{L}{T_h} \quad (\text{уровень M2}).$$

Если

$$\bar{k}_w < P - (P - 1)\mu, \quad (5)$$

то ускорение гарантировано: для любого жадного расписания выполняется  $T_{ai} < T_h$ . В более общем случае целевой уровень ускорения  $S_E^{\text{ach}} \geq S_{\text{target}} \geq 1$  гарантирован при

$$\bar{k}_w \leq \frac{P}{S_{\text{target}}} - (P - 1)\mu. \quad (6)$$

При  $S_{\text{target}} = 1$  нестрогая версия условия (6) гарантирует отсутствие замедления,  $S_E^{\text{ach}} \geq 1$ , тогда как её строгая версия совпадает с (5) и гарантирует собственно ускорение,  $S_E^{\text{ach}} > 1$ . При  $\mu \rightarrow 0$  (отсутствие доминирующего узкого места) строгий достаточный критерий сходится к необходимому  $\bar{k}_w < P$  из Замечания 4. Зазор между необходимым и достаточным условиями равен  $(P - 1)\mu$  — это «плата за неделимость» (M1) или «плата за зависимости» (M2).

*Доказательство.* По Замечанию 7 любое жадное расписание удовлетворяет  $T_{ai} \leq W/P + (1 - 1/P)L$  (на уровне M1 — с  $M_N$  вместо  $L$ ). Разделив обе части на  $T_h$  и учитывая  $W = \bar{k}_w T_h$ , получаем

$$\frac{T_{ai}}{T_h} \leq \frac{\bar{k}_w}{P} + \left(1 - \frac{1}{P}\right)\mu.$$

Правая часть не превосходит  $1/S_{\text{target}}$  в точности при условии (6) (умножение на  $P$  и перенос слагаемого), откуда  $S_E^{\text{ach}} = T_h/T_{ai} \geq S_{\text{target}}$ ; случай (5) получается при  $S_{\text{target}} = 1$  со строгим неравенством. Ч.Т.Д.  $\square$

**Remark 8** (Equivalent Form via Bottleneck Dominance). Введём относительную доминантность узкого места — отношение самой длинной задачи (на M2 — критического пути) к средней нагрузке на исполнителя:

$$b := \frac{M_N}{W/P} \quad (\text{M1}), \quad b := \frac{L}{W/P} \quad (\text{M2}).$$

Параметры связаны точным соотношением  $\mu = b \bar{k}_w / P$ , поэтому критерий (5) эквивалентно записывается в виде

$$\bar{k}_w \left(1 + \left(1 - \frac{1}{P}\right)b\right) < P.$$

Одно неравенство выполняется тогда и только тогда, когда выполняется другое, — это тот же критерий в координатах  $(\bar{k}_w, b)$  вместо  $(\bar{k}_w, \mu)$ .

Из формы через  $b$  следуют два упрощённых практических условия. Они *не эквивалентны* исходному критерию, но дают более консервативную оценку: каждое условие строго сильнее (5), поэтому его выполнение по-прежнему гарантирует ускорение.

1. **Упрощение коэффициента.** Если заменить множитель  $1 - 1/P < 1$  единицей, получим условие

$$\bar{k}_w (1 + b) < P,$$

поправка которого не зависит от  $P$  при фиксированном  $b$ . Из него следует (5), но не наоборот: критерий (5) может выполняться и при  $\bar{k}_w(1 + b) \geq P$ . При сравнении конфигураций  $P$  величину  $b = b(P)$  следует пересчитывать совместно с  $W/P$ .

2. **Фиксация  $b$  как правила декомпозиции.** Если разбиение работ гарантирует  $b \leq b_0$ , то есть ни одна задача и ни один путь зависимостей не превышают  $b_0$  средних нагрузок на исполнителя, то условие

$$\bar{k}_w (1 + b_0) < P$$

достаточно для ускорения независимо от деталей конкретного набора задач. Необходимым оно не является: фактическая доминантность узкого места может оказаться меньше  $b_0$ , и ускорение возможно даже при нарушении условия. Отсюда следует практическая рекомендация: декомпозировать работу до  $b \leq b_0$  и обеспечивать запас параллелизма  $P > \bar{k}_w(1 + b_0)$ .

## 4.6 Level M3: Coordination Overhead

**Remark 9** (Effective Weights and Lower Bound  $B_3$ ). Для учёта межагентной интеграции и повторной работы, возникающей при параллельном изменении общих артефактов, вводится функция  $C(P) \geq 1$ . Предполагается, что  $C(1) = 1$  и  $C(P)$  не убывает с ростом  $P$ ; для каждого конкретного процесса это предположение требует эмпирической проверки. Функция применяется только к автономной агентной работе, но не к человеческим фазам. В фиксированной конфигурации  $C(P)$  считается экзогенной константой: она не зависит от назначения задач, ресурсного порядка и реализовавшихся очередей. Обозначим

$$A := X \sum_{i=1}^N Z_i a_i, \quad H := X \sum_{i=1}^N Z_i h_i, \quad W = A + H.$$

При смешанном назначении режимов здесь и далее под  $a_i$  и  $h_i$  понимаются  $a_i^{(\rho(i))}$  и  $h_i^{(\rho(i))}$ . Эффективная длительность задачи и суммарная занятость агентных потоков на уровне M3 равны

$$w_i^{(3)}(P) := Z_i X (C(P) a_i + h_i), \quad W_3(P) := \sum_i w_i^{(3)}(P) = C(P) A + H.$$

Пусть  $L_3(P)$  — длина критического пути в  $G$  с весами  $w_i^{(3)}(P)$ . Тогда

$$T_{ai} \geq B_3 := \max \left\{ \frac{W_3(P)}{P}, L_3(P) \right\}.$$

Такое определение устраняет асимметрию прежней агрегированной записи: дополнительные интеграционные затраты увеличивают не только суммарную работу, но и те пути, на которых они фактически возникают.

Условие ускорения по линейной части в модели с координационными издержками принимает вид:

$$\frac{C(P)A + H}{P} < T_h \iff C(P)\bar{a}_w + \bar{h}_w < P,$$

где  $\bar{a}_w := \sum_i Z_i a_i / \sum_i Z_i$  и  $\bar{h}_w := \sum_i Z_i h_i / \sum_i Z_i$ .

Чтобы разграничить  $k_i^{(r)}$  и издержки параллелизма, примем следующее правило учёта. В  $k_i^{(r)}$  входит всё, что можно измерить при  $P = 1$  в режиме  $r$ : формулирование запросов, ожидание генерации, проверка и исправление результата в одном потоке. К издержкам параллелизма относятся только эффекты, возникающие при  $P > 1$ . Без этого правила одни и те же затраты могут быть учтены дважды — и в  $k$ , и в множителях при  $P > 1$  — либо, напротив, остаться неучтёнными, если координационные издержки неявно включены в измеренное  $k$ , а дополнительные множители приняты равными единице.

С учётом разложения  $k_i^{(r)} = a_i^{(r)} + h_i^{(r)}$  (Определение 1) и звездообразной топологии «один разработчик —  $P$  агентов» издержки разделяются на три группы:

1. координационные издержки *на уровне отдельной задачи* — подготовка спецификации, передача контекста и проверка результата — не зависят от  $P$  и уже входят в  $k_i^{(r)}$ ; поэтому они увеличивают веса задач, в том числе на критическом пути;
2. издержки *разработчика*, растущие с  $P$ , — переключение между потоками и восстановление контекста при обслуживании запросов — относятся к человеческому ресурсу и учитываются множителем  $\gamma(P)$  при  $H$  на уровне М4 (Замечание 10);
3. функция  $C(P)$  описывает *межагентные* интеграционные издержки, возрастающие с числом параллельных потоков: конфликты слияния, рассогласование интерфейсов и повторную работу из-за одновременных изменений.

В качестве рабочей параметрической формы можно использовать знаменатель универсального закона масштабируемости Гантера:  $C(P) = 1 + \alpha(P - 1) + \beta P(P - 1)$ , где  $\alpha, \beta \geq 0$ ,  $\alpha$  характеризует конкуренцию за ресурсы, а  $\beta$  — затраты на согласование [18]. Применение этой зависимости к агентам программирования является гипотезой настоящей модели, а не эмпирическим результатом указанной работы. Формула Брукса с  $\sim P(P - 1)/2$  попарными каналами связи относится к человеческой команде и не описывает звездообразную топологию буквально. Однако косвенная связанность через общий код, интерфейсы и интеграцию сохраняется даже без непосредственного общения агентов [7].

## 4.7 Level M4: Phase Semantics and the Exact Schedule

**Remark 10** (Local Blocking and Lower Bound  $B_4$ ). Разложение  $k_i^{(r)} = a_i^{(r)} + h_i^{(r)}$  (Определение 1) на уровне М4 дополняется порядком фаз. Задача  $i$  представляется конечной последовательностью  $\mathcal{F}_i = (f_{i1}, \dots, f_{im_i})$ . Агентные фазы включают автономную реализацию, самопроверку и повторную работу; человеческие — постановку, уточнение требований, проверку и приёмку. Суммы их длительностей при  $P = 1$

равны соответственно  $Z_i X a_i$  и  $Z_i X h_i$ . Фазы одной задачи следуют в заданном порядке, поэтому модель допускает несколько запросов к человеку и повторные циклы «проверка — исправление — повторная проверка».

После начала первой фазы задача закрепляется за одним агентом до принятия результата. Человеческая фаза требует одновременно внимания разработчика и сохранения выделенного агентного потока. Если разработчик занят, запрос встаёт в очередь, а соответствующий агент прекращает работу и остаётся заблокированным. Остальные агенты продолжают свои задачи независимо, пока сами не запросят человеческое внимание. Таким образом, блокировка является *локальной*, а не глобальной.

В каждый момент незавершённые назначенные задачи занимают не более  $P$  агентных потоков, а разработчик обслуживает не более одной человеческой фазы. Переиспользование заблокированного потока для другой задачи и запуск дополнительного, приоритетного потока сверх  $P$  в модели не допускаются. Новая задача может быть назначена только свободному агенту. Если  $(i, j) \in E$ , первая фаза задачи  $j$  может начаться только после приёмки задачи  $i$ . Агентные и человеческие фазы считаются непрерываемыми; порядок обслуживания одновременно ожидающих запросов является частью расписания  $\pi$ .

При  $P > 1$  эффективная длительность человеческих фаз принимается равной  $\gamma(P)H$ , где  $\gamma(P) \geq 1$ ,  $\gamma(1) = 1$ . Множитель описывает дополнительное время обслуживания из-за переключения и восстановления контекста, но не включает ожидание в очереди. Как и  $C(P)$ , для фиксированной конфигурации он считается экзогенным и не зависит от конкретного расписания. В качестве редуцированного приближения используется  $\gamma(P) = 1 + \delta(P - 1)$ ,  $\delta \geq 0$ ; при эмпирической калибровке его следует связывать с фактическим числом одновременно сопровождаемых потоков и частотой вмешательств.

Определим эффективные веса и агрегаты уровня M4:

$$w_i^{(4)}(P) := Z_i X (C(P)a_i + \gamma(P)h_i),$$

$$W_4(P) := \sum_i w_i^{(4)}(P) = C(P)A + \gamma(P)H,$$

а через  $L_4(P)$  обозначим длину критического пути с весами  $w_i^{(4)}(P)$ . Тогда фиксированная нижняя оценка времени имеет вид

$$T^* \geq B_4 := \max \left\{ \frac{W_4(P)}{P}, L_4(P), \gamma(P)H \right\},$$

где  $T^* := \min_{\pi} T(\pi)$  — оптимальное время среди допустимых расписаний, а  $T(\pi)$  — время завершения конкретного расписания  $\pi$ .

Пусть  $Q(\pi)$  — суммарное время, в течение которого уже назначенные агенты заблокированы в очереди к разработчику. Это время возникает из самого расписания и не входит ни в  $k_i$ , ни в  $H$ . Для любого допустимого расписания дополнительно выполняется

$$T(\pi) \geq \frac{W_4(P) + Q(\pi)}{P}.$$

Поэтому равенство  $T^* = B_4$  не гарантируется: одновременные запросы к человеку могут увеличить срок даже при небольшом  $H$ .

Например, если из двух параллельно работающих агентов агент А запросил помощь, он останавливается и удерживает свой поток до обслуживания. Агент Б при

этом продолжает работу. Если затем помощь потребуется и ему, оба агента окажутся в очереди к одному разработчику; простой всей системы возникнет как следствие совпадения запросов, а не как правило глобальной блокировки.

Агрегаты  $A$ ,  $H$ ,  $W_4$ ,  $L_4$  и  $B_4$  описывают объём работы и три обязательных ограничения, но не кодируют взаимное расположение человеческих фаз. Поэтому два экземпляра с одинаковыми значениями этих агрегатов могут иметь разные  $T^*$ : нижняя граница одинакова, а величина очередей и достижимость границы определяются полным фазовым профилем. Вычислительный пример такого расхождения приведён в разделе 6.

**Definition 2** (Resource-Augmented Phase Graph). Зафиксируем назначение каждой задачи одному из  $P$  агентных потоков, порядок задач внутри каждого потока и порядок обслуживания всех человеческих фаз. Если эти порядки совместимы с исходными зависимостями, построим ориентированный граф  $G_R(\omega)$ , вершинами которого служат фазы, а весом вершины — её эффективная длительность с множителем  $C(P)$  для агентной или  $\gamma(P)$  для человеческой фазы. В граф добавляются дуги четырёх типов:

1. между последовательными фазами одной задачи;
2. от последней фазы предшественника к первой фазе зависимой задачи;
3. между последовательными человеческими фазами в выбранном порядке обслуживания;
4. от последней фазы одной задачи к первой фазе следующей задачи в том же агентном потоке.

Набор решений о назначении и ресурсных порядках обозначим  $\omega$ . Допустимыми считаются только такие  $\omega$ , для которых  $G_R(\omega)$  ацикличесен. Через  $\Lambda(\omega)$  обозначим длину максимального взвешенного пути в этом графе.

**Proposition 1** (Exact Representation of the M4 Optimum). Для фиксированного допустимого ресурсного порядка  $\omega$  расписание ранних начал имеет длительность  $\Lambda(\omega)$  и является допустимым. Обратно, любое допустимое расписание  $\pi$  индуцирует некоторый порядок  $\omega(\pi)$ , причём  $\Lambda(\omega(\pi)) \leq T(\pi)$ . Следовательно,

$$T^* = \min_{\omega \in \Omega} \Lambda(\omega),$$

где  $\Omega$  — конечное множество допустимых назначений и ресурсных порядков.

*Доказательство.* При фиксированном ациклическом графе время раннего начала каждой фазы равно максимальной длине пути до неё. Дуги первых двух типов обеспечивают технологический порядок, третьего типа — непересечение человеческих фаз, четвёртого — непересечение задач, закреплённых за одним потоком. Поэтому построенное расписание допустимо и завершается за  $\Lambda(\omega)$ . В обратную сторону непересекающиеся операции любого расписания можно упорядочить по каждому ресурсу; все добавленные дуги согласованы с фактическим временем и потому не образуют цикла, а длина любого пути не превосходит  $T(\pi)$ . Минимизация двух представлений даёт равенство.  $\square$

Это представление играет роль *resource-augmented critical path*: исходный критический путь дополняется дугами конкуренции за человека и агентные потоки. Оно не вводит новый уровень М5, поскольку не добавляет ограничение к М4, а раскрывает точную комбинаторную структуру уже заданного расписания. Такая конструкция близка к дизъюнктивным графам и ресурсно-ограниченным моделям расписаний [8].

**Corollary 1** (Attainability Certificate for the Lower Bound). Для любого экземпляра М4 выполняется  $B_4 \leq T^*$ . Равенство  $T^* = B_4$  имеет место тогда и только тогда, когда существует допустимое расписание  $\pi$  с  $T(\pi) = B_4$ ; эквивалентно, существует  $\omega \in \Omega$  с  $\Lambda(\omega) = B_4$ . Явное расписание такой длины служит сертификатом достижимости. Если решатель доказывает отсутствие расписания с горизонтом  $B_4$ , граница недостижима и  $T^* > B_4$ .

**Corollary 2** (Simple Tightness Cases). Если  $P = 1$  либо task-DAG является полной последовательной цепочкой всех задач, то  $T^* = B_4 = W_4$ .

*Доказательство.* При  $P = 1$  имеем  $C(1) = \gamma(1) = 1$ , а все задачи последовательно занимают единственный поток. Их допустимо выполнить в любом топологическом порядке без простоя, поэтому срок равен сумме эффективных фаз  $W_4$ ; остальные ветви  $B_4$  её не превышают. В полной цепочке никакие две задачи не могут пересекаться при любом  $P$ , её длина равна сумме весов  $L_4 = W_4$ , и последовательное расписание достигает этой границы.  $\square$

Достижимость горизонта  $B_4$  можно проверять как задачу существования допустимого расписания, а  $T^*$  — прямой минимизацией makespan. Конкретная вычислительная формулировка и протокол проверки приведены в разделе 5.

**Proposition 2** (Monotonicity of Available Capacity). Если длительности фаз и параметры  $C, \gamma$  зафиксированы и не меняются при добавлении потока, то

$$T^*(P + 1) \leq T^*(P).$$

*Доказательство.* Любое расписание для  $P$  потоков допустимо при  $P + 1$ : дополнительный поток можно не использовать. Поэтому множество допустимых расписаний только расширяется.  $\square$

**Proposition 3** (Existence of a Finite Optimal Configuration). Пусть сравниваются конфигурации с

$$C(P) = 1 + \alpha(P - 1) + \beta P(P - 1), \quad \gamma(P) = 1 + \delta(P - 1),$$

где  $P \in \mathbb{N}$ . Если  $A > 0$  и  $\beta > 0$  либо  $H > 0$  и  $\delta > 0$ , то  $T^*(P) \rightarrow \infty$  при  $P \rightarrow \infty$ , а значит минимум  $T^*(P)$  достигается при конечном  $P$ .

*Доказательство.* В первом случае  $W_4(P)/P \geq C(P)A/P \sim \beta AP$ ; во втором  $B_4 \geq \gamma(P)H \sim \delta HP$ . В обоих случаях  $T^*(P) \geq B_4(P) \rightarrow \infty$ . Последовательность на натуральных  $P$ , уходящая в бесконечность, достигает глобального минимума на конечном начальном отрезке.  $\square$

Предложения 2 и 3 относятся к разным вопросам. Первое запрещает ухудшение от неиспользуемой мощности при неизменных длительностях. Второе сравнивает разные организационные конфигурации, в которых рост настроенного параллелизма сам изменяет издержки  $C(P)$  и  $\gamma(P)$ . Поэтому внутренний минимум по  $P$  не следует из одной лишь дискретности расписания или очередей, но гарантируется указанными растущими штрафами.

**Proposition 4** (Human-Resource Speedup Ceiling). Для фиксированного набора задач и назначения режимов  $\rho$ , если  $H > 0$ , при любом параллелизме  $P \geq 1$  и любых  $C(P) \geq 1$ ,  $\gamma(P) \geq 1$ :

$$S_E^{\text{ach}} := \frac{T_h}{T_{ai}} \leq \frac{T_h}{\gamma(P)H} \leq \frac{T_h}{H} = \frac{1}{\bar{h}_w}, \quad \bar{h}_w := \frac{\sum_{i=1}^N Z_i h_i^{(\rho(i))}}{\sum_{i=1}^N Z_i}.$$

*Доказательство.* Человеческие интервалы всех задач обслуживаются одним разработчиком, поэтому попарно не пересекаются во времени, а их суммарная длительность составляет не менее  $H$  (и не менее  $\gamma(P)H$  с учётом переключений). Следовательно,  $T_{ai} \geq \gamma(P)H \geq H$ , откуда  $S_E^{\text{ach}} = T_h/T_{ai} \leq T_h/H = 1/\bar{h}_w$ . Ч.Т.Д.  $\square$

При  $H = 0$  обязательные человеческие фазы отсутствуют, поэтому ограничение является тривиальным и потолок по человеческому ресурсу условно принимается равным бесконечности. Остальные ветви  $B_4$  и ограничения расписания сохраняются.

**Remark 11** (Analogy with Amdahl's Law). Предложение 4 представляет собой структурный аналог закона Амдала: роль последовательной составляющей играет средневзвешенная нормированная человеческая работа  $\bar{h}_w$  [1]. Потолок  $1/\bar{h}_w$  не зависит от  $P$ . Если целевое ускорение превышает эту величину, достичь его добавлением агентов невозможно; необходимо уменьшать человеческую составляющую  $h$  за счёт более автономных режимов, автоматизации проверки или формализованных критериев приёмки. На уровне М4 верхняя оценка ускорения по суммарной занятости потоков равна  $P/[C(P)\bar{a}_w + \gamma(P)\bar{h}_w]$ , а по человеческому ресурсу —  $1/[\gamma(P)\bar{h}_w]$ . Эти два агрегата полезны для быстрой диагностики, но не учитывают критический путь и очередь к человеку.

## 4.8 Cross-Level Properties

При согласованной параметризации последовательные уточнения сохраняют ограничения предыдущих уровней. Поскольку  $M_N \leq L$ ,  $C(P) \geq 1$  и  $\gamma(P) \geq 1$ , имеем  $W_3(P) \geq W$ ,  $L_3(P) \geq L$ ,  $W_4(P) \geq W_3(P)$  и  $L_4(P) \geq L_3(P)$ . Поэтому нижние оценки упорядочены следующим образом:

$$B_0 \leq B_1 \leq B_2 \leq B_3 \leq B_4,$$

Поэтому верхние оценки достижимого ускорения  $\bar{S}^{(j)} := T_h/B_j$  монотонно убывают с ростом  $j$ . На уровне М0 время в точности совпадает с оценкой ( $T_{ai}^{(0)} = B_0$ ), а Наблюдение 1 задаёт алгебраически эквивалентный критерий ускорения в форме необходимого и достаточного условия. Начиная с уровня М1 величина  $B_j$  служит только нижней границей. Фактическое время определяется распределением задач между исполнителями и может строго превышать  $B_j$  даже при оптимальном расписании; пример такого расхождения для  $B_1$  приведён в разделе с расчётами. Условие ускорения по линейной составляющей при этом остаётся необходимым, но уже не является достаточным (Замечание 4).

Связь с обозначениями вероятностного расширения модели: линейной части уровня М3 соответствует  $T_{ai}^{\text{lin}} := W_3(P)/P$ , а коррекция на неделимость требует использовать максимальный вес  $\max_i w_i^{(3)}(P)$ . Граф зависимостей, фазовая очередь и общий человеческий ресурс в вероятностном расширении пока не рассматриваются.



## 4.9 Scale Invariance and Comparative Statics

**Proposition 5** (Scale Invariance of Configuration Comparison). Зафиксируем декомпозицию проекта — набор задач со сложностями  $Z_i$  и граф зависимостей  $G$  — и конфигурацию  $\sigma$  с целым числом исполнителей  $P$ . На уровне М4 дополнительно фиксируются для каждой задачи число, типы и порядок фаз, отношения предшествования, правило локальной блокировки,  $C(P)$  и  $\gamma(P)$ . Масштабное преобразование умножает длительность *каждой* агентной и человеческой фазы на один общий коэффициент  $\kappa > 0$ ; следовательно,  $a_i$  и  $h_i$  масштабируются по отдельности, а их разбиение и фазовый профиль сохраняются. Пусть  $T^*(\kappa)$  — оптимальное время завершения после такого преобразования. Тогда:

1.  $T^*(\kappa) = \kappa T^*(1)$  (однородность первой степени); тем же свойством обладают нижние оценки  $B_0$ – $B_4$ , если  $C(P)$  и  $\gamma(P)$  от  $\kappa$  не зависят. Оптимальное распределение задач между исполнителями, порядок обслуживания очереди, активный член  $\max\{W_4(P)/P, L_4(P), \gamma(P)H\}$  и состав критического пути от  $\kappa$  не зависят;
2. если для двух вариантов организации работы  $A$  и  $B$  полные векторы длительностей соответствующих агентных и человеческих фаз известны с точностью до одного и того же общего множителя  $\kappa > 0$ , то есть  $\mathbf{d}^A = \kappa \hat{\mathbf{d}}^A$  и  $\mathbf{d}^B = \kappa \hat{\mathbf{d}}^B$ , то отношение  $T_A^*/T_B^*$  и, следовательно, ранжирование вариантов не зависят от  $\kappa$ . В частности,  $a_i$ ,  $h_i$  и  $k_i = a_i + h_i$  должны масштабироваться согласованно; одной пропорциональности суммарных  $k_i$  при меняющемся разбиении на фазы недостаточно.

*Доказательство.* При умножении длительностей всех фаз на  $\kappa$  длительности задач, человеческое обслуживание и возникающие при том же порядке событий интервалы ожидания умножаются на  $\kappa$ . Любое допустимое расписание преобразуется в допустимое расписание новой задачи умножением всех моментов начала и завершения на  $\kappa$ . Порядок фаз, отношения предшествования, локальное закрепление агентов и ограничения ресурсов сохраняются. Преобразование обратимо и задаёт взаимно однозначное соответствие между множествами допустимых расписаний. Поэтому  $T^*(\kappa) = \kappa T^*(1)$ , а оптимальное расписание переходит в оптимальное. Однородность  $B_j$  следует из линейности  $W$ ,  $M_N$ ,  $L$ ,  $W_3$ ,  $L_3$ ,  $W_4$ ,  $L_4$  и  $H$  по  $\kappa$ . Утверждение (ii) следует из (i):  $T_A^*(\kappa)/T_B^*(\kappa) = T_A^*(1)/T_B^*(1)$ . Ч.Т.Д.  $\square$

**Remark 12** (Comparative Statics without Exact Calibration). Предложение 5 уточняет возможности модели. Абсолютный прогноз — значение  $T_{ai}$  в часах или  $S_E$  — требует знания уровня длительностей и в том же масштабе наследует ошибку калибровки (Замечание 14). Однако *выбор между вариантами декомпозиции и организации работы* не зависит от общего масштаба, если полные относительные профили агентных и человеческих фаз известны с точностью до одного общего множителя.

Направление выгодных изменений модель также указывает без точной калибровки. Нижняя оценка  $B_2$  кусочно-линейна по вектору  $k = (k_1, \dots, k_N)$ . Если активна ветвь  $L > W/P$  и критический путь  $\pi^*$  единственен, её эластичности по отдельным

коэффициентам вычисляются явно:

$$\frac{\partial B_2}{\partial k_i} \cdot \frac{k_i}{B_2} = \begin{cases} \frac{Z_i k_i}{\sum_j Z_j k_j}, & \text{если } W/P > L, \\ \frac{Z_i k_i X}{L}, & \text{если } L > W/P, \pi^* \text{ единственен и } i \in \pi^*, \\ 0, & \text{если } L > W/P, \pi^* \text{ единственен и } i \notin \pi^*, \end{cases}$$

В точках переключения активных членов и при нескольких критических путях  $B_2$  может быть недифференцируема; тогда используются направленные производные, определяемые всеми активными путями. При активной ветви  $L$  увеличение  $P$  не снижает саму нижнюю оценку  $B_2$ , однако срок  $T^*$  может уменьшиться, пока не достигнет этой границы. Аналогично, при активной ветви  $\gamma(P)H$  доля задачи в  $H$  описывает чувствительность  $B_4$ , но переносить её на  $T^*$  можно только при доказанной тесноте  $T^* = B_4$ .

Инвариантность имеет три ограничения. Во-первых, она относится только к *общей мультипликативной* ошибке всех фаз. Если ошибки оценивания искажают относительные длительности задач, разбиение между  $a_i$  и  $h_i$  либо внутренний фазовый профиль, состав критического и ресурсно-дополненного путей и ранжирование вариантов могут измениться. Во-вторых, множитель  $\kappa$  должен быть общим для всех фаз и всех сравниваемых вариантов. Иными словами, систематическое смещение процедуры оценки (Замечание 14) предполагается одинаковым для конфигураций одной команды при едином протоколе измерения. В-третьих, сравнение разных режимов требует знания относительных режимных профилей отдельно для агентных и человеческих фаз. Это требование существенно слабее знания их абсолютных длительностей, но сильнее знания только суммарных коэффициентов  $k_i$ .

## 4.10 Calibration and Uncertainty

**Remark 13** (Stochastic Nature of the Coefficients  $k_i$ ). На практике влияние ИИ на выполнение задачи недетерминировано. Коэффициент  $k_i$  целесообразно рассматривать как случайную величину, распределение которой отражает неопределённость генерации кода, вероятность дефектов и затраты времени на их исправление. Тогда  $T_{ai}$  также становится случайной величиной, а условие ускорения следует проверять для математического ожидания  $\mathbb{E}[T_{ai}] < T_h$  или выбранных квантилей распределения.

Отметим, что в силу линейности математического ожидания  $\mathbb{E}[T_{ai}^{(0)}] = \frac{X}{P} \sum_{i=1}^N Z_i \nu_i$ , где  $\nu_i = \mathbb{E}[k_i]$ , поэтому тождественный критерий уровня М0 для детерминированных коэффициентов (Наблюдение 1) переносится на ожидания дословно:

$$\mathbb{E}[T_{ai}^{(0)}] < T_h \iff P > \frac{\sum_{i=1}^N Z_i \nu_i}{\sum_{i=1}^N Z_i}.$$

Систематическое развитие стохастической постановки (дисперсия, квантильные гарантии) вынесено в отдельную вероятностную модель.

**Remark 14** (Estimating  $k_i$  before Project Start). Операциональное определение  $k_i^{(r)}$  (Определение 1) носит ретроспективный характер: коэффициент вычисляется по уже наблюдавшемуся времени  $T_i^{ai}(r)$  и сам по себе не является прогнозом. Чтобы применять критерии ускорения (Наблюдение 1, Замечание 4, Теорема 1) до начала проекта, требуется внешняя оценка  $\hat{k}_i^{(r)}$ . Её можно получить следующим образом:

1. задачи классифицируются по типам (CRUD-операции, интеграция с внешним API, тесты, инфраструктура и т. п.); тип задачи  $i$  обозначим  $c(i)$ ;
2. по задачам команды накапливается выборка значений  $k$  для пар «тип  $\times$  режим»; отдельный хронометраж позволяет одновременно фиксировать человеческую составляющую  $h$  (Определение 1). Неуспешные запуски, тайм-ауты и непринятые результаты должны учитываться явно либо как наблюдения, либо как цензурированные случаи: выборка только завершённых задач оценивает условную длительность и подвержена смещению выжившего;
3. оценка  $\hat{k}_i^{(r)}$  берётся как эмпирическое среднее по паре  $(c(i), r)$  — для критерия в терминах ожиданий (Замечание 13) — либо как верхний квантиль выборки для консервативного решения.

На уровне M0 чувствительность результата к ошибке оценки следует непосредственно из  $S_E = P/\bar{k}_w$ : относительная ошибка  $\bar{k}_w$  в том же масштабе переносится в  $S_E$  (завышение  $\bar{k}_w$  в  $(1 + \varepsilon)$  раз занижает  $S_E$  в  $(1 + \varepsilon)$  раз). Для сравнения конфигураций на M0 абсолютный уровень  $k$  менее важен: общая мультипликативная ошибка не меняет их ранжирование. На M4 тот же вывод требует более сильной предпосылки Предложения 5: один множитель должен масштабировать все агентные и человеческие фазы, сохраняя их типы и порядок. Следовательно, модель имеет двойной статус. При ретроспективном применении она задаёт точное тождество, а при использовании для принятия решений точность абсолютных прогнозов  $T_{ai}$  и  $S_E^{\text{ach}}$  определяется качеством оценки  $\hat{k}$  и фазового профиля. Сравнительные выводы устойчивы только к общей систематической ошибке в смысле Замечания 12.

## 5 Computational Verification Method

Вычислительные эксперименты предназначены для проверки внутренней логики М4, поиска контрпримеров к слишком сильным интерпретациям и анализа чувствительности. Они не оценивают производительность конкретной команды или ИИ-инструмента: длительности, топологии и функции издержек в основном синтетические. Все размеры выборок ниже относятся к полностью перечислимым замороженным матрицам либо к воспроизводимому генератору, а не к случайной выборке реальных проектов.

### 5.1 Experimental Questions and Reporting Rules

Каждая серия отвечает на заранее заданный вопрос и использует собственный estimand. Таблица 2 фиксирует эту связь до изложения результатов и не позволяет смешивать доказанный оптимум, срок фиксированной политики и наблюдение на конечной синтетической сетке.

Таблица 2: Связь вычислительных серий с проверяемыми утверждениями

| Серия     | Вопрос                                                         | Изменяемые факторы                            | Estimand и правило                                        | Метод        |
|-----------|----------------------------------------------------------------|-----------------------------------------------|-----------------------------------------------------------|--------------|
| EXP-00    | Воспроизводятся ли варианты иллюстративного сценария?          | Конфигурации 2–4                              | $B_4, T^*$ ; сертификат $T^* = B_4$                       | exact        |
| EXP-01    | Всегда ли достижима граница $B_4$ ?                            | DAG, фазы, $P$                                | $T^*/B_4 - 1$ ; доказанный положительный разрыв           | exact        |
| EXP-02    | Чем различаются доступная мощность и настроенная конфигурация? | $P, C(P), \gamma(P)$ , human-share            | Профиль $T^*(P)$                                          | exact        |
| EXP-03    | Идентифицируют ли агрегаты точный срок?                        | Фазовый профиль при равных агрегатах          | Парная разность $T^*$                                     | exact        |
| EXP-04–05 | Как гранулярность взаимодействует с $P$ и overhead?            | Декомпозиция, $P$ , тип и величина overhead   | Оптимум по гранулярности и interaction contrast           | exact        |
| EXP-06    | Устойчивы ли масштабные и сравнительные выводы?                | Общий масштаб и замороженные семейства ошибок | Инвариантность и сохранение ранжирования                  | exact/stress |
| EXP-07    | Переносятся ли направления эффектов на синтетические DAG?      | $N$ , топология, веса, размещение human-work  | Paired contrasts для $T(\pi)$ ; отдельный exact-аудит     | mixed        |
| EXP-08    | Что меняет расположение фиксированного $H$ ?                   | Концентрация на пути, фазы, $P, \gamma/C$     | Парные изменения агрегатов, пути и $T(\pi)$ ; exact-аудит | mixed        |

Во всех отчётах различаются четыре величины: структурная граница  $B_4$ , доказанный оптимум  $T^*$ , найденное допустимое решение без доказательства оптимальности и срок фиксированной политики  $T(\pi)$ . Обозначение  $T^*$  используется только для запусков со статусом OPTIMAL. Допустимое расписание длины  $B_4$  доказывает достижимость границы, но совпадение нижней оценки с лучшим найденным решением без закрытого solver gap недостаточно.

### 5.2 Common Semantics and Experimental Quantities

Во всех сериях действует одна семантика М4: начатая задача удерживает один агентный поток до последней фазы, в том числе во время ожидания человека; все человеческие фазы используют один непрерываемый ресурс мощности 1; преемник становится доступен после завершения всех предшественников. Множитель  $C(P)$  применяется к агентным фазам,  $\gamma(P)$  — к человеческим. Решатель минимизирует только

makespan; очередь и суммарная блокировка сообщаются как производные характеристики, но не служат вторичной целью. Поэтому сообщаемое  $Q$  относится к одному возвращённому оптимуму, а не к минимуму  $Q$  среди всех makespan-оптимальных расписаний. В точной модели задача назначается при начале первой фазы; фиксированная имитационная политика резервирует поток заранее при выборе готовой задачи. Её очередь и срок относятся именно к этой политике.

Для серий с равной долей человеческой работы обозначим

$$r_h := \frac{h_i}{k_i}.$$

В канонических объектах EXP-01 и EXP-03 имеем  $k_i = 1$  и одинаковое  $r_h$  у всех задач; в иллюстративном объекте EXP-02 сохраняется исходное  $k_i$ , а  $h_i = r_h k_i$ . Профиль `io` делит  $H_i$  пополам вокруг единственной agent-фазы:  $H_i/2, A_i, H_i/2$ . Профиль `front_loaded` использует  $0.9H_i, A_i, 0.1H_i$ . В профиле с равным чередованием четыре human-фазы имеют длины  $H_i/4$ , а три agent-фазы —  $A_i/3$ . Историческая метка `alternating_sync` обозначает именно это внутризаточное равенство и *не* навязывает совпадение запросов разных задач по wall-clock. В `alternating_staggered` human-фазы те же, а agent-доли  $\{1/6, 2/6, 3/6\}$  циклически переставляются между задачами. В EXP-08 нейтральное имя `equal_alternating` используется для равного чередования. Только в EXP-08 номинальные половины, четверти и трети реализуются на allocation-сетке 0.001: целое число тиков делится с остатком, который распределяется по seeded-перестановке. Сумма фаз сохраняется точно, а части одного типа могут различаться не более чем на один тик.

Экспериментальная гранулярность  $m$  — число параллельных подзадач, которыми заменяется выбранная задача. В каноническом операторе EXP-04–EXP-05 её исходная входная human-фаза и полезная agent-работа делятся поровну между  $m$  подзадачами; все они наследуют предшественников исходной задачи. После них создаётся join, содержащий исходную финальную приёмку, сохраняющий исходный quality gate и становящийся предшественником всех прежних successors. При  $m = 1$  это семантически тот же объём полезной работы, хотя DAG уже содержит явный join. Поэтому contrasts EXP-04–EXP-05 относятся к преобразованному оператору «части плюс join»; СЗ в EXP-07 использует исходную задачу при  $m = 1$ , и численные величины этих двух серий напрямую не сопоставляются.

Пусть  $E_0$  — исходная полная длительность выбранной задачи,  $r_o$  — ставка издержек одной дополнительной границы, а  $\lambda_h$  — человеческая доля этих издержек. Тогда

$$O(m, r_o) = (m - 1)r_o E_0, \quad O_h = \lambda_h O, \quad O_a = (1 - \lambda_h)O.$$

Половина  $O_h$  и  $O_a$  поровну добавляется ко входам подзадач, вторая половина — к join. При  $r_o = 0$  полезные  $A$  и  $H$  сохраняются точно. Для двух уровней параллелизма complementarity декомпозиции измеряется contrast

$$I(P, m) = [T^*(P, 1) - T^*(P, m)] - [T^*(1, 1) - T^*(1, m)]. \quad (7)$$

Положительное  $I$  означает, что выигрыш от дробления при  $P$  больше, чем при  $P = 1$ . Contrast вычисляется только когда все четыре objectives доказаны как OPTIMAL. Для заранее фиксированной политики  $\pi$  отдельно обозначим

$$I_\pi(P, m) = [T_\pi(P, 1) - T_\pi(P, m)] - [T_\pi(1, 1) - T_\pi(1, m)],$$

где  $T_\pi$  — срок этой политики, а не доказанный оптимум. В замороженных матрицах EXP-04–EXP-05 используется  $\text{tolerance} = 10^{-4}$ :  $\text{contrast}$  считается положительным при  $I > \text{tolerance}$ , отрицательным при  $I < -\text{tolerance}$  и нулевым в остальных случаях. На конечной сетке EXP-04 кривая называется U-образной, если все её точки доказаны оптимальными, минимум достигается хотя бы при одном внутреннем  $m \notin \{1, 8\}$  и оба края  $T^*(1)$  и  $T^*(8)$  превосходят минимум больше  $\text{tolerance}$ . Это операциональное определение не утверждает U-образность вне исследованных целых  $m$ .

Для синтетических DAG используются две концентрации. Величина  $\rho_L = L/W_4$  задаёт долю структурной работы на longest path при  $C = \gamma = 1$ , а  $\rho_H^{\text{cp}} = H_{\text{cp}}/H$  — долю всего human-work, расположенную на выбранном структурном пути. Они не взаимозаменяемы: первая характеризует последовательность DAG, вторая — размещение фиксированного или приближённо фиксированного человеческого ресурса.

### 5.3 Frozen Exact Matrices EXP-00–EXP-05

EXP-00 решает три полностью заданных варианта иллюстративного сценария (варианты 2–4). EXP-01 использует полный факторный дизайн

$$5 \text{ топологий} \times N=4 \times P \in \{2, 4\} \times r_h \in \{0.10, 0.25, 0.50\} \times 3 \text{ профиля} = 90$$

точных точек. Топологии — chain, independent, fork-join, diamond и two-layer; веса задач циклически равны 6, 12, 18, 24. Исходная замороженная матрица содержала также  $N = 8, 12$  (270 точек), но заранее заданное правило требовало глобально уменьшить размер, если хотя бы одна точка не получает **OPTIMAL** за общий 60-секундный лимит. Первая такая точка при  $N = 8$  получила **FEASIBLE**, после чего без индивидуальной настройки лимитов была исполнена полная  $N = 4$ -матрица; исходная матрица и запись решения сохранены.

EXP-02 содержит два объекта — DAG иллюстративного варианта 4 и канонический fork-join при  $N = 4$  — и сетку  $P \in \{1, 2, 3, 4, 6, 8, 12\}$ . Для каждого объекта исполняются пять уникальных режимов: три значения  $r_h \in \{0.10, 0.25, 0.50\}$  при  $C = \gamma = 1$ , а также при  $r_h = 0.25$  отдельно

$$C(P) = 1 + 0.05(P - 1) + 0.005P(P - 1) \quad \text{и} \quad \gamma(P) = 1 + 0.10(P - 1).$$

Общий режим  $r_h = 0.25, C = \gamma = 1$  не дублируется, поэтому всего решается  $2 \times 5 \times 7 = 70$  точек.

EXP-03 скрещивает два DAG (fork-join и two-layer),  $P \in \{2, 4\}$ ,  $r_h \in \{0.25, 0.50\}$  и четыре профиля (io, front\_loaded, равное и переставленное чередование): всего 32 точные задачи. Для каждого из восьми наборов “DAG- $P$ - $r_h$ ” три профиля сравниваются с io, что даёт 24 парных contrasts при одинаковых  $A, H, W_4, L_4, B_4, C, \gamma$ .

EXP-04A повторно использует доказанную иллюстративную пару вариантов 3–4. EXP-04B применяет описанный оператор к sink-задаче  $E_0 = 24$  канонического two-layer DAG при  $N = 4$ ,  $P = 4$ ,  $r_h = 0.25$  и профиле io. Сетка

$$m \in \{1, 2, 3, 4, 6, 8\}, \quad r_o \in \{0, 0.025, 0.05, 0.10, 0.20\}, \quad \lambda_h \in \{0, 0.5, 1\}$$

содержит 90 формальных комбинаций и 66 семантически различных сценариев: при  $m = 1$  overhead нулевой для всех  $r_o, \lambda_h$ , а при  $r_o = 0$  значения  $\lambda_h$  совпадают. После восстановления общих точек анализируется 13 содержательных кривых.

EXP-05 использует тот же объект и оператор, но скрещивает  $P, m \in \{1, 2, 3, 4, 6, 8\}$ ,  $r_o \in \{0, 0.05, 0.10\}$  и  $\lambda_h \in \{0, 0.5\}$ . Из 216 формальных позиций получено 156 уникальных задач; их решения восстанавливают 180 аналитических позиций после объединения тождественных нулевых срезов. Первый общий 120-секундный проход дал 155 OPTIMAL и один FEASIBLE; предусмотренное матрицей единственное увеличение лимита до 300 секунд было затем одинаково применено ко всем 156 точкам, а не только к трудной.

## 5.4 Robustness, Synthetic DAGs, and Confirmatory Freeze

EXP-06 разделён на три части. EXP-06A умножает четыре заранее выбранных объекта (иллюстративные варианты 3–4 и две канонические точки) на  $\kappa \in \{0.25, 0.5, 1, 2, 4\}$ , одновременно масштабируя  $X$ , фазы, time unit и tolerance; это 20 точных точек с неизменной целочисленной моделью. EXP-06B сравнивает варианты 3–4 при пяти уровнях  $u \in \{0.05, 0.10, 0.20, 0.30, 0.50\}$  и 1000 seeds на уровень: 5000 пар, или 10000 точных задач. Agent- и human-множители выбираются независимо и равномерно в log-space на  $[\log(1 - u), \log(1 + u)]$ ; общие задачи получают одинаковые множители в двух вариантах, а декомпозированные тестовые задачи наследуют множитель исходной задачи с независимым ограниченным log-residual масштаба 0.25. EXP-06C проверяет два заранее заданных неблагоприятных направления на  $u = 0, 0.005, \dots, 0.95$ : исходная тестовая задача варианта 3 получает  $1 - u$ , а соответствующие декомпозированные задачи варианта 4 —  $1 + u$  на выбранном agent- либо human-ресурсе. Получаются 382 пары, или 764 точные задачи. Эти семейства ошибок синтетические и не являются доверительными интервалами эмпирической калибровки.

EXP-07 — стратифицированный пилот из 72 ячеек:

$$\begin{aligned} N &\in \{8, 16, 32, 64\}, & \text{topology} &\in \{\text{wide-layered, mixed, chain-like}\}, \\ \text{weights} &\in \{\text{homogeneous, lognormal}\}, & \rho_H^{cp} &\in \{\text{low, medium, high}\}. \end{aligned}$$

В каждой ячейке принимаются 20 последовательных observation seeds, всего 1440 сценариев: 480 общих структурных основ «граф плюс веса» повторяются для трёх уровней размещения human-work. Отклонённые попытки сохраняются, но наблюдениями не считаются. Три семейства топологии по построению соответствуют low, medium и high корзинам  $\rho_L$ , поэтому эффект topology family не идентифицируется отдельно от  $\rho_L$ . На полной выборке четыре замороженных направления C1–C4 оценивают фиксированную политику или ветви  $B_4$ : различие равного и переставленного чередования; сдвиг policy-optimal  $P$  на сетке  $\{1, 2, 4, 8\}$  при росте agent- или human-cost; policy-contrast  $I_\pi(4, 3)$  для  $m = 3$ ,  $P = 1/4$ ,  $r_o = 0/0.10$ ,  $\lambda_h = 0.5$ ; и частоту активности human- ветви при разных  $\rho_H^{cp}$ . При равенстве минимальных сроков policy-optimal параллелизм определяется как  $P_{\text{opt}} := \min \arg \min_P T_\pi(P)$ .

Знаки и правила решения были зафиксированы до расчёта. Для C1 ожидается неотрицательная медиана разности policy-tightness равного и переставленного чередования; решение требует неотрицательных point estimate и верхней границы 95%-го bootstrap-интервала. Для C2 ожидается неположительный медианный сдвиг policy-optimal  $P$ ; кроме этого доля положительных сдвигов не должна превышать 0.05. Для C3 ожидаются неотрицательные медианы  $I_\pi(4, 3)$  и ослабления выигрыша при переходе от  $r_o = 0$  к  $r_o = 0.10$ . Для C4 ожидаются положительные парная средняя разности индикаторов human-active branch и разность её частот между high и low концентрациями. Индивидуальные исключения сохраняются и сообщаются,

поэтому подтверждение медианного направления не означает универсальный знак на каждом DAG. Для интервалов используются 2000 кластерных bootstrap-перевыборок с seed 707071; единицей перевыборки служит общая структурная основа, чтобы три размещения human-work не считались независимыми.

После 20 принятых seeds в каждой ячейке замороженная матрица предписывала pilot-extension rule для двух ключевых ячеек. В первой  $N = 16$ , topology family — mixed, веса — lognormal и  $\rho_H^{cp}$  — medium; во второй  $N = 64$  при тех же topology family и типе весов, но  $\rho_H^{cp}$  — high. По протоколу выборка должна была увеличиться до 50 seeds, если bootstrap-интервал первичной медианы пересекает ноль либо Monte Carlo standard error первичной доли превышает 0.05. Реализованный runner проверил только пересечение нуля глобальными bootstrap-интервалами медиан; ветви key-cell и MCSE не были исполнены. Глобальный триггер не сработал, extension не запускался, поэтому EXP-07 следует интерпретировать как 20-seed pilot с частично реализованным правилом расширения, а не как полностью завершённую confirmatory-процедуру. Заранее выбранный exact-аудит содержит 12 DAG с  $N \in \{8, 16\}$ , homogeneous-весами, средней концентрацией и seeds 0–1 для трёх topology families. После единой ресурсной поправки, принятой до просмотра claim outcomes, exact-score ограничен baseline-gap и двумя профилями C1: 36 сценариев с лимитом 10 секунд. C2 и C3 в EXP-07 являются policy-only, C4 — lower-bound-only. FEASIBLE incumbents исключаются из доказанных пар.

EXP-08 исправляет смещение общего  $H$  и его расположения с помощью независимых development и confirmatory потоков. Общая сетка содержит 24 ячейки  $N \times \text{topology} \times \text{weight type}$ . Development namespace даёт по 10 основ на ячейку (240), confirmatory namespace — по 20 (480); raw seed равен первым 64 битам SHA-256 от namespace, ячейки, номера наблюдения и номера попытки. Confirmatory-графы генерируются только при наличии freeze-файла, а пересечения raw seeds с development и EXP-07 автоматически проверяются.

Для каждой основы сначала при  $C = \gamma = 1$  выбирается reference path; равные longest paths разрешаются seeded-рангом, не зависящим от task id, а их множественность сохраняется отдельным признаком. Общая доля human-work строго фиксируется как  $H/W_4 = 0.12$ , после чего на reference path размещается ровно 0.30 или 0.70 от  $H$ . Распределение внутри и вне пути пропорционально структурным весам, ограничено task-level долями  $[0.025, 0.85]$  и выполняется bounded largest-remainder allocator на сетке 0.001 с seeded tie-break. Для schedule-effect используются профили io и equal\_alternating,  $P \in \{1, 4, 8\}$  и  $C = \gamma = 1$ ; парная разность “high минус low” нормируется на общий  $B_4$ . Engineering equivalence margin равен 0.01, sensitivity margins — 0.005 и 0.02; 2000 bootstrap-перевыборок выполняются по graph base с seed 808071. Отдельный differentiated-slowdown срез использует  $C = 1$ ,  $\gamma = 1.3$ ,  $P = 4$ .

Scope exact-аудита EXP-08 выбран только по statuses и wall time development-прогонов, без сохранения их objectives. После экранов с общими лимитами 5 и 30 секунд до confirmatory-генерации были заморожены  $N = 8$ , homogeneous-веса, семейства mixed/chain-like, observation indices 0–1,  $P \in \{4, 8\}$  и обе allocations. Это 16 confirmatory-сценариев с лимитом 30 секунд; wide-layered и  $N = 16$  исключены единообразно по timing rule, а не по значению эффекта.



## 5.5 Synthetic DAG Generator

EXP-07 и EXP-08 используют один тип слоистого генератора. Число слоёв равно округлённому  $Nf$  с ограничением  $[2, N]$ . При двух слоях source образует первый слой, а все остальные вершины — второй; при трёх и более слоях первый и последний слои содержат по одной вершине, а остальные вершины распределяются между промежуточными слоями. Каждая вершина получает хотя бы одного предшественника из соседнего слоя, и каждая вершина предыдущего слоя получает хотя бы одного successor. Параметры  $(f, p_{adj}, p_{skip})$  равны  $(0.20, 0.30, 0)$  для wide-layered,  $(0.45, 0.25, 0.04)$  для mixed и  $(0.75, 0.15, 0)$  для chain-like. Принимаются только ациклические, слабо связанные графы, достижимые из source, с  $\rho_L \in [0.10, 0.30)$ ,  $[0.30, 0.60)$  или  $[0.60, 0.90]$  соответственно.

Homogeneous-веса равны 12. Для lognormal-весов сначала генерируется  $Y \sim \text{LogNormal}(0, 0.55)$ , затем множитель  $\min\{6, \max\{1, \text{round}(2Y)\}\}$  умножается на 12. В EXP-07 две task-level human-share величины — на структурном critical path и вне его — выбираются с сетки  $0.025, 0.05, \dots, 0.80$ : сначала минимизируется отклонение  $\rho_H^{\text{cp}}$  от целей 0.18, 0.45 или 0.75 в соответствующей корзине, затем отклонение общей human-share от 0.25. Это приближённое, а не строгое выравнивание общего  $H$ ; строго фиксированный  $H$  вводится только в EXP-08.

## 5.6 Exact Solver, Time Grid, and Fixed Policy

Точные задачи сформулированы как интервальная модель CP-SAT из OR-Tools 9.15.6755 [27]. Для фаз задаются целочисленные времена начала и окончания; порядок фаз и дуги DAG задаются ограничениями предшествования, человеческий ресурс — глобальным NoOverlap, а закрепление задачи и локальная блокировка — альтернативными интервалами, охватывающими весь span задачи на выбранном агентном потоке. Для каждого потока также задаётся NoOverlap; objective — максимум окончаний последних фаз.

Каждый сценарий задаёт положительный time\_unit. Эффективная длительность каждой фазы обязана делиться на него без остатка; иначе scenario validator завершает работу ошибкой. Поэтому CP-SAT получает точную целочисленную модель, а не округляет непрерывные длительности внутри решателя. Обратное преобразование умножает integer ticks на time\_unit. В EXP-06B/C возмущённые фазы заранее квантуются методом ROUND\_HALF\_UP минимум до одного тика, после чего последняя фаза каждого ресурса корректируется так, чтобы итог задачи оставался точно представимым. EXP-08 отдельно распределяет  $H$  на allocation-сетке 0.001, а решает расписание на сетке 0.0001.

Все точные запуски используют один search worker и solver seed 0. Статусы, objective, best bound, относительный gap и wall time сохраняются. Ограничение времени не является доказательством оптимальности: строки FEASIBLE сохраняют incumbent, но не получают обозначение  $T^*$ .

Для больших выборок используется заранее зафиксированная детерминированная политика bottom\_level\_fcfs\_v1. Пусть  $w_i^{(4)}$  — эффективный вес задачи, а

$$b_i = w_i^{(4)} + \max_{j \in \text{succ}(i)} b_j$$

— её bottom level, где максимум пустого множества равен нулю. В каждый момент события готовые неназначенные задачи сортируются по убыванию  $b_i$ , затем по убыванию  $w_i^{(4)}$  и лексикографически по task id; свободные потоки берутся по возрастанию

Таблица 3: Целочисленные сетки и лимиты точного решателя

| Серия              | Time unit                                         | Общий лимит на сценарий                                 |
|--------------------|---------------------------------------------------|---------------------------------------------------------|
| EXP-00             | 0.05                                              | 300 s                                                   |
| EXP-01             | 0.025                                             | 60 s                                                    |
| EXP-02             | 0.00005                                           | 60 s                                                    |
| EXP-03             | 0.025                                             | 60 s                                                    |
| EXP-04             | 0.00625                                           | 120 s                                                   |
| EXP-05             | 0.00625                                           | 120 s, затем единое замороженное<br>увеличение до 300 s |
| EXP-06A            | исходная сетка объекта,<br>умноженная на $\kappa$ | 60 s                                                    |
| EXP-06B/C          | 0.05                                              | 10 s                                                    |
| EXP-07 exact-аудит | 0.001                                             | 10 s                                                    |
| EXP-08 exact-аудит | 0.0001                                            | 30 s                                                    |

номера. Завершившаяся задача освобождает поток только после последней фазы. Запросы к человеку обслуживаются по времени поступления (FCFS); при одинаковом времени используются убывающий  $b_i$ , task id и индекс фазы. При одном timestamp сначала продолжают следующие фазы уже назначенных задач, затем назначаются вновь готовые задачи, после чего запускается первый human-запрос. Политика непрерываема и не использует будущую информацию сверх фиксированного DAG и длительностей. Её результаты относятся к  $T(\pi)$ , а не к  $T^*$ ; небольшие точные подвыборки служат аудитом и не подменяют основной estimand.

## 5.7 Reproducibility Environment and Integrity

Матрицы EXP-01–EXP-08, сценарии EXP-00, правила принятия решений и seeds хранятся вместе с кодом. Manifests записывают SHA-256 входов, реализации и выходных файлов, версии зависимостей, solver settings и статусы. Пакет требует Python  $\geq 3.11, < 3.14$ ; команды воспроизведения запрашивают Python 3.13, а `uv.lock` и `pyproject.toml` фиксируют OR-Tools 9.15.6755, PyYAML 6.0.3 и Matplotlib 3.11.1. Сохранённое локальное окружение, на котором проверялась эта редакция, сообщает Python 3.13.14 и macOS 26.5 arm64.

При этом исходные manifests не сохраняют patch-версию Python, ОС, процессор или объём памяти. Поэтому указанные Python patch и платформа характеризуют доступную локальную проверку, но не являются доказанными метаданными каждого первичного прогона; wall time нельзя воспроизводить как платформенно инвариантную величину. Численные выводы основаны на статусах и objectives, а не на сравнении wall time. Команды, порядок EXP-08 и полный список артефактов приведены в Приложении C.

## 6 Computational Results

### 6.1 Illustrative Planning Scenario and Reproduction by EXP-00

Рассмотрим синтетический иллюстративный сценарий разработки небольшого сервиса обработки заявок. Фиксированный объём работы включает API-контракт, слой

хранения, бизнес-логику, фоновую обработку, наблюдаемость, тесты и упаковку. В условной шкале он равен 38 story points, или 19 единицам  $Z$ ; нормировка  $X = 4$  часа даёт последовательный исходный срок  $T_h = 76$  часов. Эта связь используется только для прозрачности сценария и не является эмпирическим преобразованием story points в часы.

Task-DAG содержит последовательное ядро  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6$  и две боковые ветви: наблюдаемость с упаковкой и работы, которые становятся доступными после фиксации интерфейсов. Сравниваются четыре варианта организации работы. В вариантах 1–3 меняются режим и управляемый параллелизм, поэтому меняются и коэффициенты  $k_i, h_i$ . Вариант 4 сохраняет конфигурацию варианта 3, но дробит монолитную задачу тестирования на три части с точными зависимостями. Только сравнение 3→4 выделяет эффект декомпозиции при прочих равных.

Таблица 4: Сокращённое сравнение вариантов иллюстративного сценария

| Показатель   | Вариант 1    | Вариант 2    | Вариант 3    | Вариант 4        |
|--------------|--------------|--------------|--------------|------------------|
| $P$          | 1            | 2            | 4            | 4                |
| Режим        | интеракт.    | смешанный    | делегир.     | делегир.         |
| Декомпозиция | крупная      | крупная      | крупная      | дроблённые тесты |
| $W_4$ , ч    | 65.0         | 59.8         | 53.6         | 53.6             |
| $W_4/P$ , ч  | 65.0         | 29.9         | 13.4         | 13.4             |
| $L_4$ , ч    | 51.8         | 47.8         | 43.2         | 36.7             |
| $H$ , ч      | 65.0         | 38.4         | 19.0         | 19.0             |
| $B_4$ , ч    | 65.0         | 47.8         | 43.2         | 36.7             |
| $T^*$ , ч    | 65.0         | 47.8         | 43.2         | 36.7             |
| Ускорение    | $1.17\times$ | $1.59\times$ | $1.76\times$ | $2.07\times$     |

В серии EXP-00 для вариантов 2–4 точный решатель получил статус OPTIMAL. В каждом случае предъявлено допустимое расписание с  $T(\pi) = B_4$ , поэтому цепочка  $B_4 \leq T^* \leq T(\pi)$  доказывает равенство  $T^* = B_4$  именно для этих сценариев. Равенство не предполагается универсальным свойством M4.

### 6.1.1 Local Blocking and the Human Resource

В варианте 2 два агентных слота обслуживаются одним разработчиком. Наивное расписание, допустимое по DAG и числу агентов, содержит пересечения human-фаз и потому недопустимо в M4. Исправленное расписание на Рисунке 1 размещает постановки и проверки последовательно. Задача 7 ожидает проверку 22.2 часа и удерживает второй слот, но ожидание идёт параллельно критической ветви и не увеличивает срок проекта. Этот пример показывает, почему одна величина  $Q(\pi)$  не определяет makespan: важно положение ожидания в ресурсном расписании.

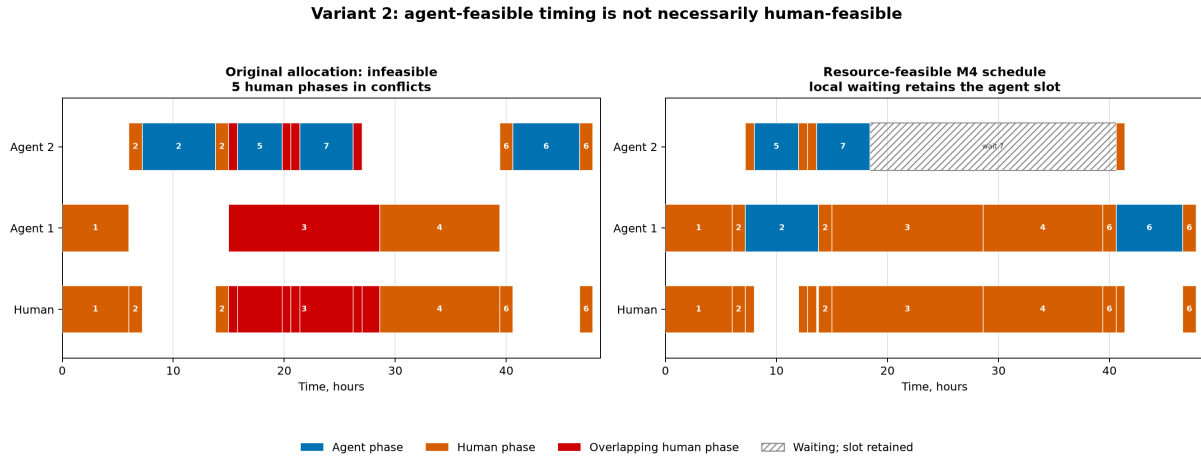


Рис. 1: Вариант 2 до и после явного учёта человеческого ресурса. Слева исходное расписание содержит пересекающиеся human-фазы и недопустимо в М4. Справа обслуживание разработчиком сериализовано; ожидающая задача удерживает свой агентный слот, но не увеличивает общий срок.

### 6.1.2 Critical-Path Decomposition

В варианте 3 вся задача тестирования ждёт завершения фонового обработчика. В варианте 4 модульные и интеграционные тесты запускаются после собственных предшественников, а в хвосте критического пути остаётся только проверка повторных попыток. При неизменных  $A = 34.6$ ,  $H = 19.0$  и  $W_4 = 53.6$  критический путь и точный makespan сокращаются с 43.2 до 36.7 часа. Эффект равен 6.5 часа, или примерно 15% срока варианта 3; ускорение относительно исходного сценария возрастает с  $1.76\times$  до  $2.07\times$ .

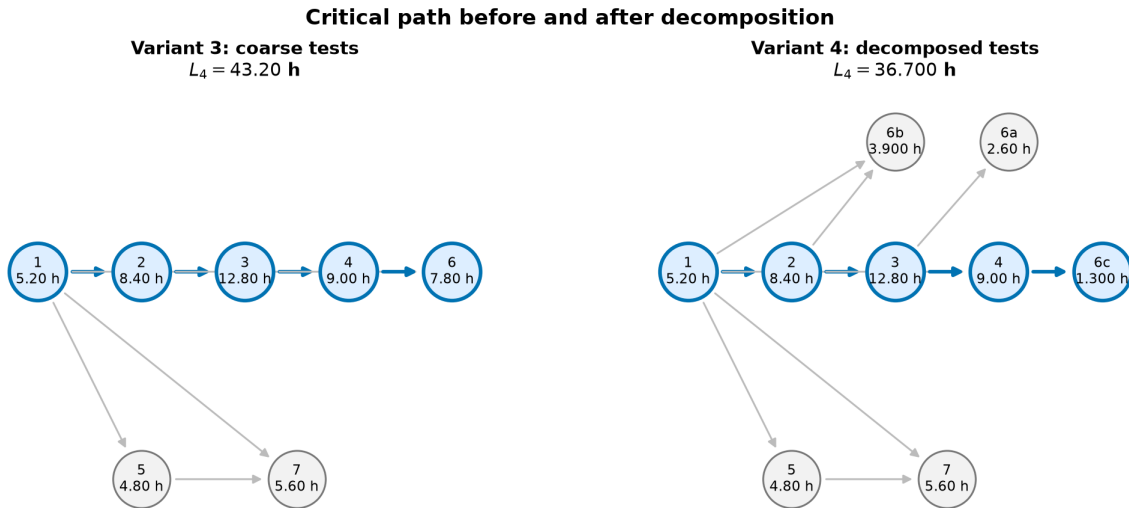


Рис. 2: Task-DAG и критический путь до и после уточнения зависимостей задачи тестирования. Суммарная работа сохраняется, сокращается лишняя сериализация.

Полные параметры задач, фазовые интервалы, проверка человеческого ресурса и элементарные примеры M0–M1 приведены в Приложении В. Ниже серии расположе-

ны по содержательным вопросам, а не по их числовым идентификаторам: сначала проверяется достижимость границы и роль фазового порядка, затем параллелизм и декомпозиция, после чего — масштабная устойчивость, перенос на синтетические DAG и контролируемое размещение human-work. Числовой идентификатор обозначает замороженный протокол, а не место серии в логике изложения.

## 6.2 Attainability of $B_4$

Равенство, полученное в EXP-00 для иллюстративного сценария, не является общим свойством модели. В EXP-01 все 90 сценариев решены до оптимальности, и в 30 из них найден строгий разрыв  $T^* > B_4$ ; максимальный относительный разрыв на этой сетке равен 18.333%. Выбранный четырёхзадачный контрпример на сетке EXP-01 — diamond при  $P = 2$ :  $B_4 = 48.00$ , но  $T^* = 49.25$  часа. Дополнительные 1.25 часа создаёт ресурсная цепочка повторных человеческих фаз, отсутствующая в трёх агрегатных ветвях  $B_4$ .

EXP-03 прямо проверяет достаточность агрегатов. В 14 из 24 пар одинаковые DAG,  $A$ ,  $H$ ,  $W_4$ ,  $L_4$ ,  $B_4$ ,  $C$  и  $\gamma$  дали разные  $T^*$ . В наиболее выраженной выбранной паре при общем  $B_4 = 36.0$  два фазовых профиля дали 40.5 и 37.75 часа. Следовательно,  $B_4$  корректно задаёт структурную границу, но точное значение определяет ресурсно-дополненный фазовый граф.

Рисунок 3 показывает такую цепочку явно на одной точке EXP-05 с  $P = 2$ ,  $m = 4$  и агентной долей overhead 0.05. В ней обычная последовательность фаз и зависимости DAG перемежаются порядком обслуживания единственного человеческого ресурса и порядком задач на агентном слоте. Поэтому её длина 36.45 часа совпадает с доказанным  $T^*$  и превышает агрегатную границу  $B_4 = 31.80$  часа.

**Resource-augmented critical path (EXP-05)**  
**36.45 h versus  $B_4=31.80$  h**

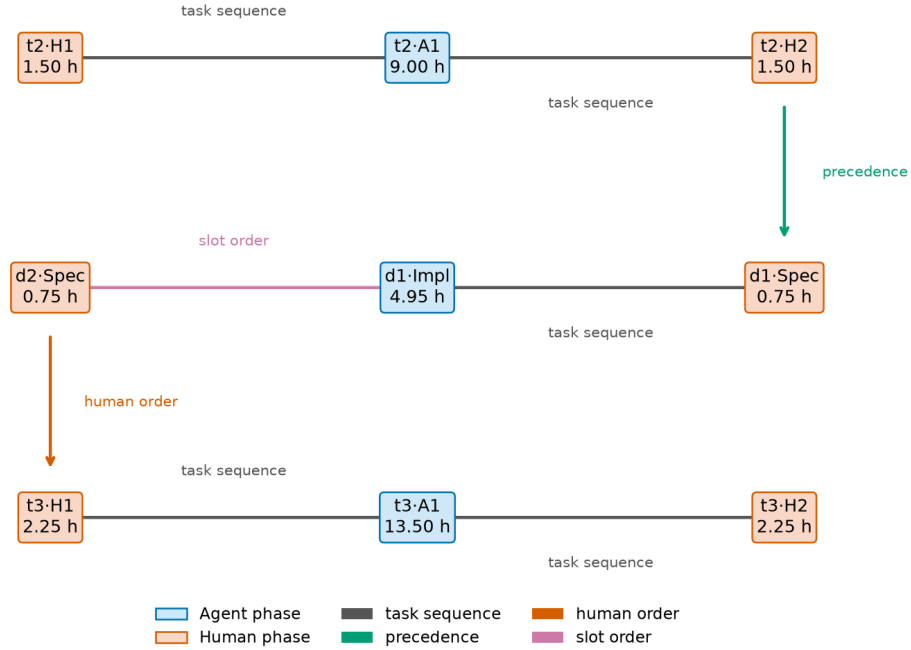


Рис. 3: Ресурсно-дополненный критический путь: task sequence — порядок фаз одной задачи, precedence — ребро DAG, human order — порядок на человеческом ресурсе, slot order — порядок задач на одном агентном слоте. Обозначения  $d1$ ,  $d2$  относятся к двум декомпозированным подзадачам.

### 6.3 Configured Parallelism

На 70 точных сценариях EXP-02 при постоянных  $C = \gamma = 1$  рост доступного  $P$  не ухудшал оптимум и быстро приводил к плато по критическому пути или человеческой ветви. Если же с  $P$  росли  $C(P)$  или  $\gamma(P)$ , во всех четырёх соответствующих исследованных кривых внутренний минимум находился при  $P = 2$ . Это согласуется с Предложениями 2 и 3: ухудшение создаёт не лишний свободный поток, а изменившаяся конфигурация процесса.

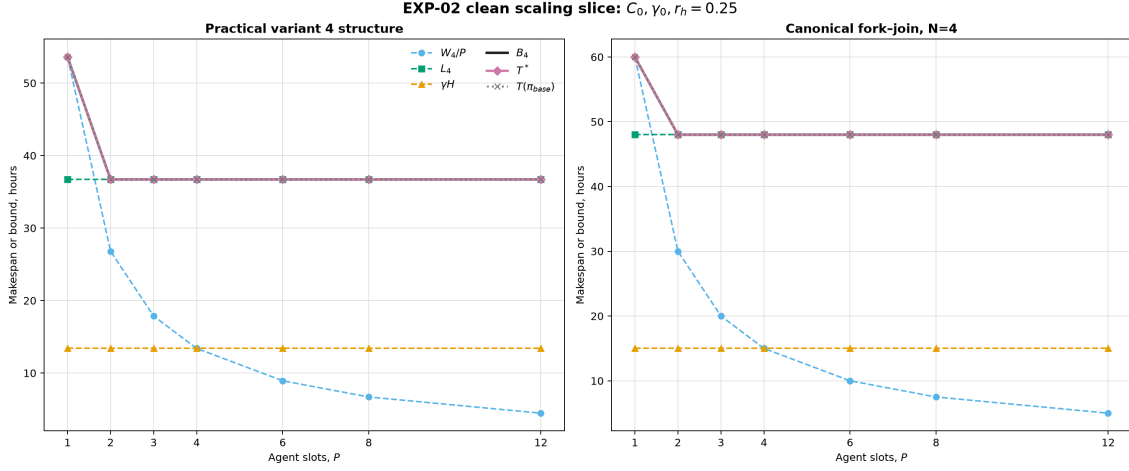


Рис. 4: Точный makespan при изменении числа агентных потоков. Постоянные издержки дают насыщение, растущие  $C(P)$  или  $\gamma(P)$  — внутренний минимум на исследованной сетке.

## 6.4 Decomposition and Its Interaction with $P$

В EXP-04 точный профиль  $T^*(m)$  оказался U-образным в 12 из 13 кривых — во всех вариантах с положительными издержками дробления. Общий нулевой срез после снятия сериализации вышел на плато. При небольшом overhead оптимум обычно достигался при  $m = 3$ , а при его росте смещался к  $m = 2$ . Поэтому декомпозиция не монотонно полезна: она сокращает структурную сериализацию, но добавляет спецификацию, приёмку и интеграцию.

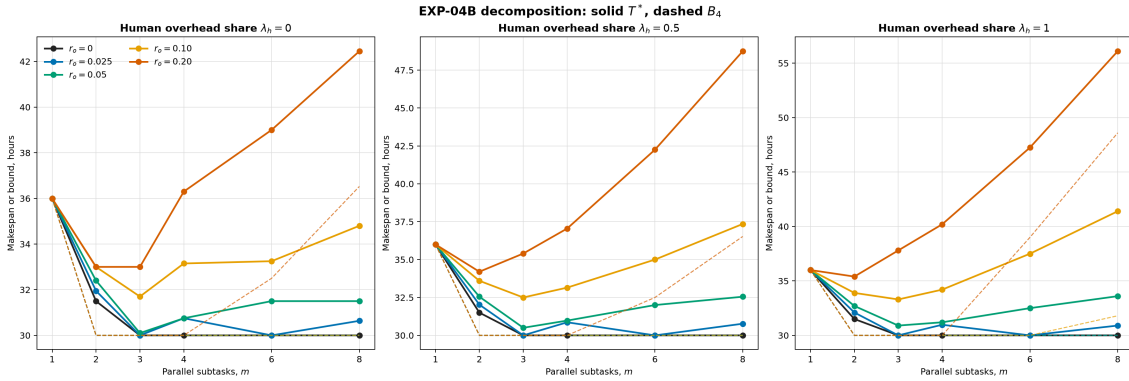


Рис. 5: Точные кривые декомпозиции. Положительные издержки создают конечную оптимальную гранулярность; нулевые издержки дают плато.

EXP-05 проверяет совместный эффект. Из 100 внутренних interaction contrasts при положительном overhead 96 положительны, четыре равны нулю и ни один не отрицателен. Однако абсолютный выигрыш от дробления при  $P > 1$  положителен лишь в 74 точках и отрицателен в 26; в 22 точках положительный interaction contrast сочетается с отрицательным абсолютным выигрышем. Следовательно, взаимодополняемость означает усиление эффекта относительно  $P = 1$ , а не безусловную полезность дробления. Увеличение человеческой составляющей overhead ослабляет выигрыш. Это результат выбранной сетки, а не универсальная теорема.

## 6.5 Scale and Parameter Robustness: EXP-06

EXP-06 воспроизвёл масштабную инвариантность как реализационную проверку на 20 из 20 точных точек. Преимущество иллюстративного варианта 4 над вариантом 3 сохранилось во всех 5000 случайных и 382 заранее заданных неблагоприятных парах. Этот результат означает устойчивость лишь внутри замороженных семейств ошибок, а не при произвольном возмущении всех параметров.

## 6.6 Transfer across Synthetic DAGs: EXP-07

На 1440 сценариях, построенных из 480 синтетических DAG с тремя размещениями human-work, EXP-07 в пилотном режиме поддержал два заранее заданных медианных направления для `bottom_level_fcfs_v1`. В C2 медианный сдвиг  $P_{\text{opt}} := \min \arg \min_P T_\pi(P)$  при введении agent- и human-cost был равен нулю; сдвиги в противоположную, положительную сторону встретились соответственно в 0.42% и 0.63% DAG, то есть ниже предзаданного порога 5%. В C3 медиана  $I_\pi(4, 3)$  равнялась 6.4 с 95%-м bootstrap-интервалом  $[6.0, 6.8]$ , а медиана ослабления выигрыша из-за overhead — 2.0 с интервалом  $[2.0, 2.0]$ . Отрицательные индивидуальные contrasts при этом составляли 6.53% и 3.47%. Следовательно, пилот поддержал агрегированные направления, а не универсальные знаки для каждого DAG и не утверждения о доказанном  $T^*$ . Из-за частичной реализации pilot-extension rule эти решения относятся к полученному 20-seed pilot и требуют повторной проверки с полным gate.

Гипотезы о «синхронном» профиле и о простой концентрации human-work не подтвердились; первая фактически не моделировала синхронизацию во времени, а вторая смешивала изменение общего  $H$  с его расположением. Поэтому ретроспективная интерпретация первой ограничена сравнением равного и переставленного асинхронных фазовых профилей, а вывод о расположении  $H$  проверяется заново в EXP-08. В exact-аудите EXP-07 только 17 из 36 запусков получили OPTIMAL; четыре полные пары C1 дали малую оценку противоположного знака. Этот аудит не охватывал C2–C3 и не заменяет основной policy-estimand.

## 6.7 Placement of Fixed Human Work: EXP-08

Исправленный парный дизайн EXP-08 сохранил DAG,  $A$ ,  $H$  и общий объём работы. При  $C = \gamma = 1$  перемещение человеческой работы между задачами не изменило  $H$ ,  $W_4$ ,  $L_4$ ,  $B_4$  и активные ветви во всех 2880 проверках. При  $\gamma > C$  перенос доли  $\Delta H$  на заранее выбранный путь изменяет его длину точно на  $(\gamma - C)\Delta H$ ; это тождество подтверждено в 480 из 480 пар. Изменение же срока фиксированной политики зависело от фаз и  $P$ . Для первичного профиля io нормированная парная медиана “high minus low” при  $P = 4$  была  $-0.0077$  с 95%-м bootstrap-интервалом  $[-0.0090, -0.0065]$  и по предзаданной margin 0.01 классифицирована как practically equivalent. При  $P = 8$  оценка составила  $-0.0117$  с интервалом  $[-0.0120, -0.0101]$  и получила решение negative.

Все 16 сценариев exact-аудита имели статус OPTIMAL, однако это не делает статистический вывод определённым: восемь доказанных пар дали для  $P = 4$  и  $P = 8$  одинаковую медиану 0.0010 с интервалом  $[-0.0150, 0.0080]$ , поэтому оба решения остались inconclusive. Тем самым важны две разные характеристики: общий объём человеческой работы задаёт ресурсный потолок; при  $\gamma \neq C$  её размещение меняет



веса путей и  $L_4$ , а при  $C = \gamma$  порядок фаз всё ещё может изменить очередь и срок фиксированной политики при неизменных агрегатах.

## 7 Discussion

Полученные результаты позволяют читать модель не как формулу «число агентов делит срок», а как модель планирования ограниченной человеко-агентной ячейки.

### 7.1 Bound, Optimum, and a Concrete Schedule

$B_4$  отвечает на вопрос, быстрее какого срока закончить невозможно из-за суммарной загрузки потоков, исходного критического пути или единственного человеческого ресурса.  $T^*$  отвечает на другой вопрос: каков лучший срок с учётом всех ресурсных порядков. Наконец,  $T(\pi)$  характеризует выбранную политику или фактическое расписание. Равенство  $T^* = B_4$  требует сертификата; в противном случае одинаковая граница не означает одинаковый срок.

### 7.2 Resource Chains as a Gap Mechanism

Повторные обращения нескольких задач к одному разработчику связывают фазы, которые не лежат на одном пути исходного DAG. Удержание агентного потока во время ожидания переносит эту конкуренцию дальше по расписанию. Ресурсно-дополненный фазовый граф делает механизм видимым: его максимальный путь содержит не только технологические, но и ресурсные дуги. Это объясняет, почему одни лишь  $A, H, W_4, L_4, B_4$  не идентифицируют  $T^*$ .

### 7.3 Parallelism as a Configuration Choice

Следует различать доступную и настроенную мощность. При неизменных длительностях дополнительный поток можно оставить свободным, поэтому оптимум не ухудшается. Но организационный переход к большему  $P$  может повысить стоимость интеграции  $C(P)$  и переключений  $\gamma(P)$ . Тогда сравниваются уже разные процессы, и конечный минимум возможен. Практически это означает, что  $P$  нельзя выбирать в отрыве от назначения режимов и функций издержек. Формальная конфигурация  $\sigma = (P, \rho)$  фиксирует число потоков и режимы задач; декомпозиция, DAG, фазовые профили и критерий приёма образуют остальные условия полностью заданного сценария. Если вместе с  $P$  меняется хотя бы одно из них, наблюдаемая разность относится к сравнению сценариев целиком, а не к изолированному эффекту дополнительного потока.

### 7.4 Decomposition and Parallelism

Декомпозиция полезна, пока она сокращает длинные последовательные участки и создаёт готовую работу для свободных потоков. После этого дополнительные границы увеличивают спецификацию, интеграцию и число вмешательств человека. В исследованном операторе декомпозиции поэтому оптимальная гранулярность зависит от  $P$ : без доступного параллелизма мелкие независимые задачи почти не дают выигрыша, а при большом  $P$  именно гранулярность может ограничивать использование мощности.

Эксперименты поддерживают такую взаимодополняемость на выбранных сетках, но не доказывают её для произвольного DAG или правила дробления; её направление и величина требуют калибровки на реальных процессах.

## 7.5 Volume and Placement of Human Work

Общий  $H$  задаёт безусловную последовательную нагрузку и потолок ускорения. Его расположение — отдельная характеристика. При одинаковом масштабировании агентных и человеческих фаз перенос  $H$  между задачами не обязан менять агрегатную границу; при  $\gamma > C$  человеческая работа на критическом пути дороже агентной и удлиняет этот путь. Даже при неизменном  $L_4$  фазовый порядок может менять очередь и  $T^*$ . Поэтому для проектирования процесса недостаточно сократить часы ревью: важно также решить, когда и в каких задачах они нужны.

## 7.6 Relation to Prior Work

Потолок по человеческому ресурсу является структурным переносом логики закона Амдала: последовательной долей здесь служит не участок вычислительной программы, а измеренное участие разработчика [1]. Внутренний минимум по настроенному  $P$  согласуется с универсальным законом масштабируемости в том ограниченном смысле, что растущие конкуренция и согласование способны сменить насыщение отрицательной отдачей [18]. Численная форма  $C(P)$  для агентов при этом остаётся гипотезой модели, а не следствием этих классических результатов.

Разрыв между  $B_4$  и  $T^*$  связывает постановку с теорией расписаний и RCPSP: исходного графа зависимостей задач и суммарной нагрузки недостаточно, когда единственный обслуживающий ресурс создаёт дополнительные ресурсные цепочки [16, 8]. Дуги ресурсного порядка и общий сервер являются стандартными механизмами теории расписаний [28, 19]. Модели загрузки и выгрузки уже содержат сервис до и после обработки и могут удерживать машину до выгрузки [14]; возвратная модель направляет работу после удалённого сервера на ту же основную машину, но освобождает её на время сервиса [10]. Следовательно, новизна не заявляется ни для DAG, ни для общего ресурса, повторного сервиса или ресурсно-дополненного графа.

Близкая идея конечного числа эффективно контролируемых исполнителей возникает в моделях числа роботов на одного оператора, где интервалы обслуживания должны укладываться в окна автономной работы остальных систем [25, 12]. Эксперимент Cummings и Mitchell дополнительно показывает, что очередь, ожидание восстановления ситуационной осведомлённости и переориентация снижают расчётную пропускную способность оператора, но численно переносить результат имитации UAV на разработку нельзя [13]. Среди агентов программирования MAGIS реализует автоматизированный цикл «разработка — контроль качества — доработка», а CAID — конкурентные рабочие деревья с планом зависимостей, слиянием и тестированием [30, 15]. Их показатели качества и успешности не свидетельствуют о сокращении makespan; в основных сравнениях CAID календарное время выполнения и стоимость росли вместе с показателем качества.

Настоящая модель объединяет известные конструкции в отображении на программный процесс: коэффициент  $k$ , зависящий от задачи и режима, отдельные  $a_i$  и  $h_i$ , повторные человеческие фазы, локальное удержание агентного потока, интеграционные издержки и издержки переключения, а также makespan принятого

результата. В основном корпусе, сформированном по поисковому протоколу с датой завершения 2026-08-09, не обнаружена их совместная эмпирическая калибровка на программных задачах; это не универсальное утверждение обо всей литературе. Сама статья также не закрывает этот эмпирический пробел: вычислительные результаты уточняют механизм процессного ограничения на синтетических сценариях, но не заменяют измерения конкретных инструментов и команд.

## 7.7 Using the Model

Сначала следует оценить задачи и режимы в общей шкале, построить DAG и фазовые профили, затем вычислить  $B_4$  как быструю диагностику узкого места. Для нескольких конкурирующих конфигураций строится точное или эвристическое расписание и явно фиксируется, получен ли  $T^*$  или только  $T(\pi)$ . Наконец, проверяется чувствительность к  $k$ ,  $h$ ,  $C$  и  $\gamma$ . Модель полезнее для сравнения вариантов организации работы, чем как обещание точного календарного срока без локальной калибровки. Активная ветвь  $B_4$  характеризует ограничение нижней границы, но сама по себе не доказывает, что фактический  $T^*$  определяется той же ветвью: для такого вывода требуется достижимость границы либо анализ ресурсно-дополненного расписания. Аналогично, автономная агентная фаза означает только отсутствие участия разработчика на данном интервале; задача с такой фазой остаётся делегированной, если постановка или приёмка требуют человека.

## 8 Limitations and Threats to Validity

В формальной модели предполагается, что ориентированный ациклический граф  $G = (V, E)$  известен до начала планирования и остаётся неизменным в ходе исполнения. Это допущение приемлемо для заранее специфицированного набора работ, но ограничивает применение модели к исследовательским задачам, архитектурным разведкам и другим R&D-процессам, в которых новые задачи и зависимости обнаруживаются по мере получения результатов. В таких процессах оценки  $L_4(P)$  и  $B_4$  относятся только к текущему снимку графа  $G_t$ ; однажды вычисленный критический путь нельзя интерпретировать как постоянную характеристику всего проекта.

Все задачи считаются доступными в момент начала планирования, их фазы — непрерываемыми, а ограниченными ресурсами — только  $P$  агентных потоков и один разработчик. Release dates, отдельные мощности CI/API и полностью ручные задачи требуют другой постановки; M4 описывает только заранее выбранный AI-поддержанный набор.

Модель также не описывает механизм раскрытия новых вершин, стоимость перепланирования и выбор способа дальнейшей декомпозиции. Дробление задачи может выявить только содержательно допустимый параллелизм: оно не устраняет реальные отношения предшествования и обычно увеличивает затраты на постановку, проверку и интеграцию. Поэтому приведённые результаты позволяют оценивать уже заданный граф или его текущий снимок, но сами по себе не определяют оптимальную политику построения динамического графа.

Абсолютный прогноз времени требует калибровки  $X$ ,  $k_i$ ,  $h_i$ ,  $C(P)$  и  $\gamma(P)$  для конкретных инструментов и режима работы; при их смене такая калибровка может быстро устаревать. Для сравнительного выбора между двумя вариантами требования слабее: объёмы и длительности можно задать в условных единицах одной базовой

задачи, если обе конфигурации оценены в общей шкале. Предложение 5 гарантирует устойчивость ранжирования только к одному общему множителю, одинаково применённому ко всем agent- и human-фазам сравниваемых вариантов. Неодинаковые ошибки по задачам или режимам, искажение разбиения между  $a_i$  и  $h_i$ , внутреннего фазового профиля, а также разные значения  $C(P)$  и  $\gamma(P)$  способны изменить ресурсные пути и порядок вариантов.

**Construct validity.** Коэффициенты  $k$ ,  $a$  и  $h$  агрегируют качество постановки, возможности инструмента, проверку и исправление результата. Один и тот же численный профиль может скрывать разные механизмы, а makespan не измеряет качество кода, понимание системы, стоимость вычислений и последующие дефекты. Термины «синхронный» и «концентрация» в ранних экспериментальных гипотезах оказались недостаточно операциональными; соответствующие выводы не включены в число подтверждённых.

**Internal validity.** Точные серии используют детерминированные длительности, целочисленную сетку и один решатель. Ошибки реализации снижаются независимыми проверками ограничений, статусами оптимальности, замороженными входами и воспроизводимыми manifests, но вычислительный эксперимент подтверждает свойства реализации формальной модели, а не истинность её допущений о реальном процессе.

**Conclusion validity.** Точные конечные сетки не доказывают универсальные закономерности за пределами выбранных диапазонов. Числа 30/90, 12/13 и 96/100 описывают именно исследованные матрицы, а не частоты в генеральной совокупности проектов. В больших сериях EXP-07 и части EXP-08 оценивается фиксированная политика; её эффекты нельзя переносить на  $T^*$  без отдельного точного доказательства.

**External validity.** Большинство DAG, длительностей и функций  $C(P)$ ,  $\gamma(P)$  синтетические и не калиброваны на продольных данных команд. Рассматривается один разработчик, однородные агентные потоки и локальная блокировка с удержанием потока. Другие планировщики могут переиспользовать ожидающую мощность, несколько людей могут обслуживать запросы параллельно, а реальные задачи раскрывают зависимости и повторную работу динамически.

**Parameter uncertainty.** EXP-06 проверяет только заранее заданные случайные и неблагоприятные семейства ошибок. Наблюдаемая устойчивость ранжирования не является гарантией против произвольной совместной ошибки оценок. Перед практическим решением необходимы локальная калибровка, интервальный анализ и повторное планирование по мере появления фактических данных.

## 9 Future Work

Для фиксированного графа предложенная модель решает прямую задачу: по структуре работ и параметрам человеко-агентной ячейки оценивается достижимое ускорение и строится расписание. Естественным продолжением является обратная задача. Если в момент  $t$  известна только часть графа  $G_t$ , дальнейшую декомпозицию можно выбрать с учётом критического пути, требуемого параллелизма и ограниченного ресурса человеческого внимания. Тем самым модель становится не только средством оценки готового плана, но и критерием формирования ещё не раскрытой части проекта.

Такую политику можно строить по принципу скользящего горизонта. После результата, который обнаруживает новые работы или зависимости, обновляются  $G_t$ , множество готовых задач и оценки фаз, после чего расписание незавершённой части

строится заново. Исследовательские задачи при этом можно представить вершинами-разведками, результат которых раскрывает множество допустимых продолжений  $\mathcal{D}_t$ . Для каждого  $d \in \mathcal{D}_t$  можно использовать суррогатную оценку

$$\Phi_t(d) = \max \left\{ \frac{W_{4,t}(d) + \hat{Q}_t(d)}{P}, L_{4,t}(d), \gamma(P)H_t(d) \right\}, \quad d_t^* \in \arg \min_{d \in \mathcal{D}_t} \Phi_t(d),$$

где  $\hat{Q}_t(d)$  — оценка будущего ожидания, не сводимого к уже известным структурным границам. Вариант декомпозиции должен сохранять исходный объём работ и критерий приёма, а его дополнительные спецификации, проверки и интеграционные издержки должны входить в  $W_{4,t}(d)$  и  $H_t(d)$ . Если эмпирически известен комфортный предел  $P_{\max}^{\text{attn}}$  одновременно сопровождаемых потоков, его превышение следует запретить, а близость полезного параллелизма к  $P_{\max}^{\text{attn}}$  использовать как вторичный критерий после минимизации  $\Phi_t$ . Построение допустимых преобразований графа, оценивание  $\hat{Q}_t(d)$  и получение гарантий для такой адаптивной политики образуют отдельную задачу совместного выбора декомпозиции и расписания; в настоящей работе этот критерий приводится только как эскиз будущей теории.

Отдельное направление связано с распространением модели на другие архитектуры многоагентных и роевых систем. Ближайшим к текущей постановке является иерархический вариант, в котором агент-оркестратор заменяет разработчика во внутреннем контуре: декомпозирует цель, ставит задачи рабочим агентам, обслуживает их запросы и принимает результаты. Структурная аналогия здесь прямая: оркестратор также является разделяемым ресурсом конечной мощности, а обращения рабочих агентов образуют очередь и могут локально блокировать их потоки. При этом человеческое внимание может сохраниться только на границах всего процесса — при постановке общей цели и итоговой приёме.

Иная постановка возникает при алгоритмической оркестрации с аудитом и самопроверкой. Макрограф задач может оставаться детерминированным, но каждая его вершина раскрывается в локальный контур «исполнение — аудит — условная доработка — повторный аудит». Если такие контуры самодостаточны и не используют общий оркестратор, они не блокируют друг друга, а внутренний человеческий ресурс мощности 1 исчезает. В терминах настоящей модели аудит и доработка переходят в агентные фазы:  $h_i$  для внутренних вершин уменьшается, а  $a_i$  растёт; знак изменения  $k_i = a_i + h_i$  зависит от баланса сэкономленной человеческой и добавленной агентной работы. При фиксированном числе проходов такой контур можно развернуть в обычный фазовый DAG; условное завершение и неизвестное число доработок потребуют стохастической модели повторной работы.

Более общее расширение должно описывать архитектуру не числом агентов, а графом ресурсов и взаимодействий. Иерархические оркестраторы, децентрализованные рои, взаимный аудит, специализированные планировщики и верификаторы, общая память и гетерогенные ресурсы создают разные узкие места и разные формы координационных издержек. Сравнение таких систем требует заменить звездообразную схему «один разработчик —  $P$  агентов» гетерогенной многоуровневой моделью расписаний и оценивать конфигурацию целиком: топологию, правила маршрутизации, ресурсы аудита, качество результата и стоимость дополнительных циклов самоконтроля. Несколько человеко-агентных ячеек требуют дополнительных межъячеечных ресурсов и потому не сводятся к увеличению  $P$  в М4.

Другим направлением является относительная и интервальная калибровка для организаций, не располагающих историей точных измерений. В режиме холодного старта единицу  $X$  можно трактовать не как час, а как условную базовую задачу, после чего задавать  $Z_i$ ,  $k_i$  и  $h_i$  экспертно относительно неё. Если две конфигурации  $A$  и  $B$  описаны в одной шкале, а все их агентные и человеческие фазы содержат один неизвестный общий множитель  $\kappa > 0$ , то из Предложения 5 следует

$$\frac{T_A^*(\kappa)}{T_B^*(\kappa)} = \frac{T_A^*(1)}{T_B^*(1)}.$$

Тем самым даже грубые количественные относительные оценки могут быть полезны для выбора более быстрой схемы организации работ без обещания точного срока в часах. Дальнейшее исследование должно заменить точечные оценки интервалами для  $k_i$  и  $h_i$ , определить условия устойчивого доминирования одного варианта над другим и выделить случаи, в которых неопределённость относительного профиля способна изменить ранжирование.

Ещё одно направление — стохастическая и робастная версия с повторной работой, отказами, обучением и интервальными оценками  $k$ ,  $h$ ,  $C$  и  $\gamma$ . Она должна давать не только ожидаемый срок, но и квантили, вероятность нарушения дедлайна и условия устойчивого доминирования одной конфигурации над другой.

Наконец, необходима эмпирическая калибровка на трассах реальной программной работы. Она должна отдельно измерять автономные и человеческие интервалы, ожидание, повторную работу и качество результата, чтобы проверить форму  $C(P)$  и  $\gamma(P)$  и отделить эффективный параллелизм от номинального числа запущенных агентов.

## 10 Conclusion

В работе исследован переход от локальной длительности задачи с ИИ к сроку фиксированного набора связанных программных работ. Для одной человеко-агентной ячейки ответ на поставленный во введении вопрос состоит в следующем: конфигурация сокращает срок относительно последовательной работы без ИИ не в силу самого наличия нескольких агентов, а только тогда, когда существует допустимое расписание, завершающее тот же принятый объём работ быстрее исходного сценария. Достижимое ускорение определяется совместно нормированными длительностями задач, их зависимостями и фазовыми профилями, числом и правилами использования агентных потоков, интеграционными издержками и последовательным ресурсом человеческого внимания. Тем самым подтверждён основной тезис: ускорение является свойством полностью заданного процесса, а не изолированной характеристикой ИИ-инструмента или номинального числа агентов.

Первый результат работы — операциональная рамка сравнения — задаёт границы такого вывода. Коэффициент  $k_i^{(r)}$  описывает длительность принятого результата для конкретной тройки “задача — инструмент — режим” и не имеет причинной интерпретации без отдельного дизайна сравнения. Конфигурация  $\sigma = (P, \rho)$  фиксирует число потоков и назначение режимов, но корректное сопоставление сценариев требует также сохранять либо явно изменять декомпозицию, DAG, фазовые профили, критерий приёма и функции издержек. Это разделение не позволяет переносить измеренный  $k$  из интерактивного режима в делегированную конфигурацию с другим

$P$  и отличает автономную агентную фазу, во время которой человек свободен, от полностью автономного процесса.

Второй результат — иерархия M0–M4 — показывает, как по мере снятия идеализаций меняется сила ответа. На M0 ускорение достигается тогда и только тогда, когда  $P > \bar{k}_w$ . На M1–M2 это условие остаётся необходимым, но неделимость и зависимости создают дополнительный зазор; при целом числе одинаковых потоков и  $C(P) = 1$  достаточную гарантию жадного расписания даёт  $\bar{k}_w < P - (P - 1)\mu$ , где  $\mu = M_N/T_h$  для M1 и  $\mu = L/T_h$  для M2. На M3 эффективные веса включают межагентные интеграционные издержки. На M4 полностью заданный сценарий ускоряет работу тогда и только тогда, когда его оптимальный makespan удовлетворяет  $T^* < T_h$ . Если структурная граница  $B_4 \geq T_h$ , ускорение невозможно; если  $B_4 < T_h$ , этого ещё недостаточно. Достаточным свидетельством ускорения служит допустимое расписание  $T(\pi) < T_h$ , а точный решатель устанавливает существование и оптимальность такого расписания.

Третий результат уточняет структуру M4. Нижняя граница

$$B_4 = \max \left\{ \frac{W_4(P)}{P}, L_4(P), \gamma(P)H \right\}$$

объединяет суммарную загрузку агентных потоков, критический путь исходного task-DAG и единственный человеческий ресурс. Однако повторные обращения задач к разработчику и удержание ожидающими задачами агентных слотов создают смешанные ресурсные цепочки, не содержащиеся ни в одной из трёх ветвей. Ресурсно-дополненный фазовый граф представляет точный срок как минимальную длину максимального пути по допустимым назначениям и ресурсным порядкам. Тем самым строго разведены структурная граница  $B_4$ , доказанный оптимум  $T^*$  и срок выбранной политики  $T(\pi)$ . Последовательная human-work одновременно задаёт независимый потолок  $S_E^{\text{ach}} \leq 1/[\gamma(P)\bar{h}_w] \leq 1/\bar{h}_w$ : после его достижения добавление агентов не заменяет сокращение человеческих фаз, автоматизацию проверки или более формальный критерий приёмки.

Четвёртый результат — серия EXP-00–EXP-08 — проверяет эти различия на воспроизводимых сценариях. Иллюстративные варианты 2–4 получили доказанные оптимумы, но равенство  $T^* = B_4$  оказалось частным свойством: в EXP-01 строгий разрыв найден в 30 из 90 точных сценариев. В EXP-03 разные фазовые профили при одинаковых  $A, H, W_4, L_4$  и  $B_4$  дали разные  $T^*$  в 14 из 24 пар. При неизменных длительностях дополнительная доступная мощность не ухудшала оптимум, тогда как растущие  $C(P)$  или  $\gamma(P)$  создавали внутренний минимум по настроенному  $P$  на исследованных кривых EXP-02. Положительные издержки декомпозиции дали конечную оптимальную гранулярность в 12 из 13 кривых EXP-04. В EXP-05 interaction contrast параллелизма и дробления был положительным в 96 из 100 внутренних сравнений и нулевым в четырёх, но абсолютный эффект декомпозиции оставался отрицательным в 26 точках. Следовательно, взаимодополняемость параллелизма и гранулярности не означает безусловной полезности дробления.

Проверки устойчивости ограничивают силу этих выводов. Масштабная инвариантность воспроизведена во всех точках EXP-06A, а преимущество иллюстративного варианта 4 над вариантом 3 сохранилось в заданных случайных и неблагоприятных семействах ошибок EXP-06B–C. Синтетический пилот EXP-07 поддержал агрегированные направления для фиксированной политики, но не универсальные знаки для каждого DAG и не утверждения о  $T^*$ . Исправленный парный дизайн EXP-08

показал, что общий объём человеческой работы задаёт ресурсный потолок, тогда как её размещение влияет на пути только при различии масштабов agent- и human-фаз; фазовый порядок при этом может менять очередь и срок даже при неизменных агрегатах. Все частоты относятся к замороженным синтетическим матрицам и не являются оценками распространённости эффектов в реальных проектах.

Практическое применение модели поэтому начинается не с выбора максимального числа агентов. Для сравниваемых сценариев необходимо в общей шкале оценить задачи и режимы, зафиксировать критерий приёма, построить DAG и фазовые профили, вычислить  $B_4$  как диагностику обязательных ограничений, а затем построить точное или эвристическое ресурсно-допустимое расписание и проверить чувствительность к  $k$ ,  $h$ ,  $C$  и  $\gamma$ . Активная ветвь  $B_4$  показывает, что ограничивает нижнюю границу, но только достижимость границы или анализ ресурсного пути позволяет перенести этот вывод на фактический оптимум. В этом смысле модель предназначена прежде всего для сравнения способов организации работы и поиска рычагов изменения срока, а не для обещания абсолютного прогноза без локальной калибровки.

Полученный ответ ограничен принятой архитектурой: фиксированным заранее известным DAG, одним разработчиком, однородными агентными потоками, непрерываемыми фазами и локальным удержанием слота. Модель не оценивает качество кода и стоимость как самостоятельные цели и не заменяет причинное исследование эффективности ИИ. Следующие шаги — эмпирически измерить agent/human-интервалы, очередь, повторную работу,  $C(P)$  и  $\gamma(P)$  на реальных трассах; перейти к интервальной и стохастической калибровке; а затем расширить постановку на динамическое раскрытие DAG, адаптивную декомпозицию и другие архитектуры оркестрации. До такой проверки главный вывод остаётся структурным, но уже операциональным: полезный параллелизм возникает не из числа запущенных агентов, а из согласованного проектирования задач, зависимостей, фаз, ресурсов и правил приёма всего человеко-агентного процесса.

## 11 Code and Data Availability

Исследовательский артефакт распространяется вместе с текущей рабочей версией рукописи в каталоге `experiments`. Он включает исходный код симулятора и точного решателя, тесты, замороженные матрицы сценариев EXP-00–EXP-08, manifests с контрольными суммами, исходные CSV/JSON-результаты, отчёты и код построения рисунков. Файлы `pyproject.toml` и `uv.lock` фиксируют Python-зависимости вычислительного пакета. Поисковый протокол обзора и evidence matrix входят в комплект как `../tmp_docs/simple_model_full_literature_search_protocol.md` и `../tmp_docs/simple_model_full_evidence_matrix.md`.

В дополнение к пакету воспроизводимости создан интерактивный companion artifact M4 Human-Agent Workbench, доступный по адресу <https://m4.articles.rexarrior.fun/>. Его исходный код и конфигурация развёртывания находятся в каталоге `calculator`. Интерфейс позволяет импортировать AI-декомпозицию проекта в JSON, редактировать DAG и параметры M4, а затем локально в браузере вычислить нижнюю границу, ресурсно-допустимое расписание и сценарное ускорение. Русская и английская версии, а также автоматическая, светлая и тёмная темы доступны в одной публикации. Пока пользователь явно не включит добровольную передачу расчёта, конфигурация и результат не отправляются на сервер.

Публичный калькулятор является демонстрационным и прикладным интерфейсом



к модели, но не заменяет воспроизводимый комплект `experiments`. Код вычислительного пакета и калькулятора распространяется по лицензии MIT; текст рукописи, документация, созданные автором рисунки, сценарии и результаты — по лицензии CC BY 4.0. Точные области действия и исключения зафиксированы в файле `LICENSE.md`. Публичный репозиторий, версионированный релиз и DOI будут указаны после архивирования версии, сопровождающей препринт. Рабочий URL калькулятора не считается архивным постоянным идентификатором.

Команды проверки и воспроизведения приведены в Приложении С. При распространении статьи отдельно от репозитория каталог `experiments` и два указанных файла обзора должны передаваться как единый supplementary artifact: без входных матриц, manifests, lockfile и поискового протокола одной PDF-рукописи недостаточно для независимой проверки вычислений и литературного поиска.

## Declarations

The author is employed by Yandex. This research was conducted independently in the author’s personal time and received no dedicated funding from Yandex. Some computational work used Yandex equipment and computing infrastructure. The views expressed are solely those of the author and do not necessarily represent those of Yandex. The author declares no other competing interests.

## Список литературы

- [1] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, pages 483–485, 1967. DOI: <https://doi.org/10.1145/1465482.1465560>.
- [2] Nishant Balepur, Connor Baumler, Valerie Chen, Eunsol Choi, Rachel Rudinger, and Jordan Lee Boyd-Graber. (Im)Paired Programming: Coding agents improve productivity but harm understanding, 2026. Version 1, submitted 29 July 2026; DOI: <https://doi.org/10.48550/arXiv.2607.26375>.
- [3] Shraddha Barke, Michael B. James, and Nadia Polikarpova. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):78:1–78:27, 2023. DOI: <https://doi.org/10.1145/3586030>.
- [4] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv preprint arXiv:2507.09089, 2025. DOI: <https://doi.org/10.48550/arXiv.2507.09089>.
- [5] Joel Becker, Nate Rush, Tom Cunningham, David Rein, and Khalid Mahamud. We are changing our developer productivity experiment design. METR, February 2026. <https://metr.org/blog/2026-02-24-uplift-update/>; accessed 2026-08-03.
- [6] Barry Boehm, Chris Abts, and Sunita Chulani. Software development cost estimation approaches—a survey. *Annals of Software Engineering*, 10(1–4):177–205, 2000. DOI: <https://doi.org/10.1023/A:1018991717352>.

- [7] Frederick P. Brooks, Jr. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, Boston, anniversary edition, 1995.
- [8] Peter Brucker, Andreas Drexler, Rolf Möhring, Klaus Neumann, and Erwin Pesch. Resource-constrained project scheduling: Notation, classification, models, and methods. *European Journal of Operational Research*, 112(1):3–41, 1999. DOI: [https://doi.org/10.1016/S0377-2217\(98\)00204-5](https://doi.org/10.1016/S0377-2217(98)00204-5).
- [9] Erik Brynjolfsson, Danielle Li, and Lindsey R. Raymond. Generative AI at work. *The Quarterly Journal of Economics*, 140(2):889–942, 2025. DOI: <https://doi.org/10.1093/qje/qjae044>.
- [10] Konstantin Chakhlevitch and Celia A. Glass. Scheduling reentrant jobs on parallel machines with a remote server. *Computers & Operations Research*, 36(9):2580–2589, 2009. DOI: <https://doi.org/10.1016/j.cor.2008.11.007>.
- [11] Jacob W. Crandall, Mary L. Cummings, Mauro Della Penna, and Paul M. A. de Jong. Computing the effects of operator attention allocation in human control of multiple robots. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, 41(3):385–397, 2011. DOI: <https://doi.org/10.1109/TSMCA.2010.2084082>.
- [12] Jacob W. Crandall, Michael A. Goodrich, Dan R. Olsen, Jr., and Curtis W. Nielsen. Validating human-robot interaction schemes in multitasking environments. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, 35(4):438–449, 2005. DOI: <https://doi.org/10.1109/TSMCA.2005.850587>.
- [13] Mary L. Cummings and Paul J. Mitchell. Predicting controller capacity in supervisory control of multiple UAVs. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, 38(2):451–460, 2008. DOI: <https://doi.org/10.1109/TSMCA.2007.914757>.
- [14] Abdelhak Elidrissi, Rachid Benmansour, Keramat Hasani, and Frank Werner. Scheduling on parallel machines with a common server in charge of loading and unloading operations, 2023. Version 1, submitted 29 June 2023; DOI: <https://doi.org/10.48550/arXiv.2306.16669>.
- [15] Jiayi Geng and Graham Neubig. Effective strategies for asynchronous software engineering agents, 2026. Version 2, revised 8 July 2026; accepted at COLM 2026; DOI: <https://doi.org/10.48550/arXiv.2603.21489>.
- [16] R. L. Graham. Bounds for certain multiprocessing anomalies. *The Bell System Technical Journal*, 45(9):1563–1581, 1966. DOI: <https://doi.org/10.1002/j.1538-7305.1966.tb01709.x>.
- [17] R. L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969. DOI: <https://doi.org/10.1137/0117039>.
- [18] Neil J. Gunther. A general theory of computational scalability based on rational functions, 2008. DOI: <https://doi.org/10.48550/arXiv.0808.1431>.

- [19] Nicholas G. Hall, Chris N. Potts, and Chelliah Sriskandarajah. Parallel machine scheduling with a common server. *Discrete Applied Mathematics*, 102(3):223–243, 2000. DOI: [https://doi.org/10.1016/S0166-218X\(99\)00206-1](https://doi.org/10.1016/S0166-218X(99)00206-1).
- [20] Magne Jørgensen. Forecasting of software development work effort: Evidence on expert judgement and formal models. *International Journal of Forecasting*, 23(3):449–462, 2007. DOI: <https://doi.org/10.1016/j.ijforecast.2007.05.008>.
- [21] Magne Jørgensen and Martin Shepperd. A systematic review of software development cost estimation studies. *IEEE Transactions on Software Engineering*, 33(1):33–53, 2007. DOI: <https://doi.org/10.1109/TSE.2007.256943>.
- [22] Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, Yao Yan, Zhihan Zhang, Yuchen Zhuang, Yun Liu, Mark Malhotra, Paul Pu Liang, Hae Won Park, Yuzhe Yang, Xuhai Xu, Yilun Du, Shwetak Patel, Tim Althoff, Daniel McDuff, and Xin Liu. Towards a science of scaling agent systems, 2025. Version 3, revised 8 April 2026; DOI: <https://doi.org/10.48550/arXiv.2512.08296>.
- [23] S. A. Kravchenko and F. Werner. Parallel machine scheduling problems with a single server. *Mathematical and Computer Modelling*, 26(12):1–11, 1997. DOI: [https://doi.org/10.1016/S0895-7177\(97\)00236-7](https://doi.org/10.1016/S0895-7177(97)00236-7).
- [24] Banruo Liu, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, and Esha Choukse. Agentic coding in the wild: Characterizing GitHub Copilot traces at production scale, 2026. Version 1, submitted 30 July 2026; DOI: <https://doi.org/10.48550/arXiv.2608.00101>.
- [25] Dan R. Olsen, Jr. and Stephen Bart Wood. Fan-out: Measuring human control of multiple robots. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 231–238, 2004. DOI: <https://doi.org/10.1145/985692.985722>.
- [26] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The impact of AI on developer productivity: Evidence from GitHub Copilot. arXiv preprint arXiv:2302.06590, 2023. DOI: <https://doi.org/10.48550/arXiv.2302.06590>.
- [27] Laurent Perron, Frédéric Didier, and Steven Gay. The CP-SAT-LP solver. In *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*, volume 280 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 3:1–3:2. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2023. DOI: <https://doi.org/10.4230/LIPIcs.CP.2023.3>.
- [28] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 3 edition, 2008. DOI: <https://doi.org/10.1007/978-0-387-78935-4>.
- [29] Dario D. Salvucci and Niels A. Taatgen. Threaded cognition: An integrated theory of concurrent multitasking. *Psychological Review*, 115(1):101–130, 2008. DOI: <https://doi.org/10.1037/0033-295X.115.1.101>.
- [30] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. MAGIS: LLM-based multi-agent framework for GitHub issue resolution. In *Advances in Neural Information Processing Systems*, volume 37, 2024. NeurIPS 2024; DOI: <https://doi.org/10.52202/079017-1647>.

- [31] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, pages 1–7, 2022. DOI: <https://doi.org/10.1145/3491101.3519665>.

## A Full Glossary of Terms and Notation

В этом разделе зафиксированы термины и обозначения, используемые далее в статье. Приведённые пояснения служат кратким справочником; формальные определения и допущения вводятся в следующем разделе.

### A.1 Core Terms

#### Единица работы

Условный объём работы, трудоёмкость которого без ИИ равна  $X$ . В иллюстративном сценарии единицы работы связываются с оценкой в story points, однако модель не требует конкретной шкалы сложности.

#### Задача

Неделимая на выбранном уровне декомпозиции часть проекта. Задача имеет сложность  $Z_i$ , режим выполнения  $r$  и нормированную длительность  $k_i^{(r)}$ . При изменении уровня декомпозиции одна задача может быть заменена несколькими подзадачами.

#### Трудоёмкость

Объём работы, выраженный во времени выполнения без ИИ. В исходном сценарии с одним разработчиком трудоёмкость совпадает с длительностью; при параллельном выполнении суммарная трудоёмкость и календарный срок проекта различаются.

#### Календарное и активное время

Календарное время измеряется от начала до завершения задачи и включает ожидание. Активное время учитывает только интервалы непосредственной работы исполнителя. Эти показатели, как и пропускная способность потока задач, не являются взаимозаменяемыми.

#### Принятый результат

Результат, удовлетворяющий заданному критерию приёмки. Время проверки, исправления и повторной проверки включается в длительность задачи; простое завершение агентной генерации не считается завершением работы.

#### Исходный сценарий

Последовательное выполнение заданного объёма работ одним разработчиком без ИИ. Его продолжительность обозначается  $T_h$  и используется как база для расчёта ускорения. В англоязычной литературе употребляется термин *baseline*.

#### Нормированная длительность с ИИ

Отношение длительности задачи с ИИ к базовой оценке  $Z_i X$ . Обозначается  $k_i^{(r)}$  и зависит от задачи, инструмента и режима. Это описательная величина, а не

универсальная характеристика модели ИИ или причинная оценка без отдельного дизайна.

### **ИИ-агент для программирования**

Программная система на основе модели ИИ, способная получать задачу, работать с кодом и инструментами и возвращать результат для автоматической либо человеческой приёмки. Краткая форма в статье — *агент*.

### **Человеко-агентная ячейка**

Рассматриваемая в статье организационная единица: один разработчик ставит задачи одному или нескольким агентам и принимает их результаты. Модель нескольких взаимодействующих ячеек остаётся направлением дальнейшего развития.

### **Режим работы**

Способ взаимодействия человека и агента при выполнении задачи. Различаются интерактивный, делегированный и полностью автономный режимы. Режим определяет не продукт сам по себе, а порядок постановки, выполнения, проверки и исправления задачи.

### **Интерактивный режим**

Разработчик и агент совместно выполняют одну задачу, последовательно обмениваясь запросами и результатами. В предельном случае человеческая составляющая занимает всё время задачи:  $h_i = k_i$ .

### **Делегированный режим**

Разработчик ставит задачу, агент выполняет её автономно, после чего человек проверяет результат, принимает его или возвращает на доработку.

### **Полностью автономный режим**

Агент самостоятельно обнаруживает задачу, выполняет её и подтверждает результат без обязательного участия человека. Этот режим описывает границу применимости модели и подробно не исследуется. Его не следует смешивать с автономной agent-фазой делегированной задачи: такая фаза освобождает разработчика на время исполнения, но постановка и приёмка результата по-прежнему требуют human-фаз.

### **Конфигурация процесса**

Полное описание организации работы: настроенный параллелизм  $P$ , назначение режимов задачам, используемый инструмент и протокол постановки и приёмки. При фиксированных инструменте и протоколе применяется сокращённая запись  $\sigma = (P, r)$  для единого режима либо  $\sigma = (P, \rho)$  для разных режимов. Сравнить конфигурации целиком, поскольку изменение любого из этих элементов может изменить коэффициенты  $k_i^{(r)}$ , фазовые профили и издержки.

### **Декомпозиция**

Разбиение проекта на задачи и подзадачи с явными границами и зависимостями. Декомпозиция определяет доступный параллелизм и состав критического пути; чрезмерное дробление может увеличить издержки постановки и приёмки.

## Параллелизм

Число одновременно доступных потоков выполнения с ИИ. Обозначается  $P$ . В задачах составления расписания  $P$  равно целому числу исполнителей, а в аналитических оценках может трактоваться как эффективное, усреднённое за период значение. Число запущенных агентов не обязательно совпадает с фактически достижимым параллелизмом.

## Граф зависимостей

Ориентированный ациклический граф  $G = (V, E)$ , вершины которого соответствуют задачам, а дуги задают отношения предшествования. Англоязычное сокращение — DAG (*directed acyclic graph*).

## Расписание

Назначение задач исполнителям и моментов их начала и завершения. Расписание называется допустимым, если соблюдает отношения предшествования, порядок фаз, число агентных потоков, локальное закрепление агента за начатой задачей и ограничение человеческого ресурса.

## Фаза задачи

Непрерывный этап выполнения задачи с однозначно заданным типом ресурса. В модели различаются автономные агентные и человеческие фазы; последние требуют внимания разработчика и одновременно сохраняют закрепление соответствующего агентного потока.

## Критический путь

Самый длинный по времени путь в графе зависимостей. Его длина  $L$  задаёт нижнюю границу срока проекта: увеличение числа агентов не сокращает задачи, последовательно связанные на этом пути.

## Время завершения всех работ

Время от начала проекта до завершения последней задачи с принятым результатом. Это основной временной показатель статьи. В теории расписаний ему соответствует термин *makespan*.

## Суммарная работа

Сумма длительностей всех задач с ИИ при выполнении в отдельных потоках. Обозначается  $W$ . Величина  $W/P$  задаёт идеализированную оценку при равномерном распределении нагрузки.

## Агентная составляющая

Интервалы выполнения задачи, в течение которых агент работает автономно, а разработчик свободен. Нормированная продолжительность обозначается  $a_i^{(r)}$ .

## Человеческая составляющая

Интервалы, в течение которых разработчик занят постановкой, чтением, проверкой или направлением исправлений по задаче. Нормированная продолжительность обозначается  $h_i^{(r)}$ .

## Человеческий ресурс

Время и внимание одного разработчика, общие для всех потоков. Ресурс имеет мощность 1: человеческие интервалы разных задач не могут пересекаться.

### **Координационные издержки**

Дополнительные затраты, возникающие при параллельной работе нескольких потоков. Межагентные интеграционные издержки учитываются функцией  $C(P)$ , а затраты разработчика на переключение между потоками — множителем  $\gamma(P)$ .

### **Локальная блокировка агентного потока**

Правило уровня М4, согласно которому начатая задача удерживает назначенного агента до принятия результата. Если задача запросила внимание занятого разработчика, агент прекращает работу и остаётся занятым ожиданием; остальные агенты продолжают свои задачи. Блокировка одного потока не означает остановки всей системы.

### **Очередь к человеческому ресурсу**

Множество запросов агентов, ожидающих обслуживания одним разработчиком. Ожидание не входит в полезное человеческое время  $H$ , но увеличивает занятость закреплённых агентов и может привести к превышению фактического времени над нижней оценкой  $B_4$ .

### **Переиспользование заблокированного потока**

Запуск другой задачи на месте агента, ожидающего человека. Такая организация, как и запуск приоритетного дополнительного потока сверх  $P$ , в статье не рассматривается.

### **Ускорение**

Отношение времени исходного сценария к времени выбранной конфигурации с ИИ. Значение больше единицы означает сокращение срока, единица — отсутствие временного эффекта, значение меньше единицы — замедление.

### **Нижняя оценка времени**

Значение, меньше которого время завершения быть не может. На уровнях М1–М4 нижняя оценка не обязательно достижима даже при оптимальном расписании.

### **Узкое место**

Задача, путь или ресурс, который в рассматриваемой конфигурации определяет нижнюю оценку времени. В модели узким местом может быть суммарная нагрузка  $W/P$ , самая длинная задача  $M_N$ , критический путь  $L$  или человеческий ресурс  $\gamma(P)H$ .

### **Уровни М0–М4**

Последовательные уточнения модели. Уровень М0 предполагает идеальную делимость работы; М1 учитывает неделимость задач, М2 — граф зависимостей, М3 переопределяет длительности с учётом координационных издержек, а М4 вводит фазовый порядок, ограниченный человеческий ресурс и локальную блокировку.

## **A.2 Notation**

- $X$  Базовая трудоёмкость единицы работы без ИИ.
- $N$  Число задач в проекте;  $i \in \{1, \dots, N\}$  — индекс задачи.

- $Z_i$  Сложность  $i$ -й задачи в единицах  $X$ .
- $R, r$  Множество допустимых режимов работы и выбранный режим.
- $\rho$  Отображение, назначающее каждой задаче собственный режим:  $\rho: \{1, \dots, N\} \rightarrow R$ .
- $T_i^{ai}(r)$  Наблюдаемое время выполнения  $i$ -й задачи с ИИ в одном потоке режима  $r$  при полной доступности разработчика.
- $k_i^{(r)}$  Нормированная длительность  $i$ -й задачи с ИИ в режиме  $r$ :  $k_i^{(r)} = T_i^{ai}(r)/(Z_i X)$ . Значения меньше, равные и больше единицы соответствуют меньшей, равной и большей длительности относительно базы.
- $\hat{k}_i^{(r)}$  Прогнозная оценка  $k_i^{(r)}$ , полученная до начала проекта по сопоставимым задачам и режимам.
- $a_i^{(r)}, h_i^{(r)}$  Агентная и человеческая составляющие коэффициента  $k_i^{(r)}$ , причём  $k_i^{(r)} = a_i^{(r)} + h_i^{(r)}$ .
- $P, P_h$  Параллелизм процесса с ИИ и параллелизм исходного сценария. В статье принимается  $P_h = 1$ .
- $\sigma$  Сокращённая запись конфигурации процесса при фиксированных инструменте и протоколе приёмки:  $\sigma = (P, r)$  или  $\sigma = (P, \rho)$ .
- $T_h$  Время последовательного выполнения всего проекта одним разработчиком без ИИ.
- $T_{ai}, T_{ai}^{(0)}$  Время выполнения проекта с ИИ и его идеализированное значение на уровне М0.
- $W$  Суммарная работа с ИИ:  $W = X \sum_{i=1}^N Z_i k_i$ .
- $w_i$  Вес, или длительность, отдельной задачи с ИИ:  $w_i = Z_i k_i X$ .
- $A$  Суммарная автономная агентная работа:  $A = X \sum_i Z_i a_i$ .
- $M_N$  Длительность самой длинной задачи:  $M_N = X \max_i (Z_i k_i)$ .
- $G = (V, E)$  Ориентированный ациклический граф зависимостей между задачами.
- $L$  Длина критического пути в  $G$  с весами  $w_i$ .
- $W_3(P), L_3(P)$  Суммарная занятость агентных потоков и длина критического пути на уровне М3 после применения  $C(P)$  только к автономным агентным фазам.



$W_4(P), L_4(P)$

Аналогичные величины уровня М4 после дополнительного учёта множителя  $\gamma(P)$  в длительности человеческих фаз.

$C(P)$

Множитель межагентных координационных и интеграционных издержек;  $C(1) = 1$ .

$H$  Суммарное человеческое время:  $H = X \sum_{i=1}^N Z_i h_i^{(r)}$ .

$\gamma(P)$

Множитель затрат на переключение внимания разработчика между параллельными потоками;  $\gamma(1) = 1$ .

$Q(\pi)$

Суммарное время блокировки назначенных агентов в очереди к разработчику в конкретном расписании  $\pi$ . Это эндогенная характеристика расписания, не входящая в  $k_i$  и  $H$ .

$T(\pi), T^*$

Время завершения конкретного допустимого расписания  $\pi$  и минимальное время среди всех допустимых расписаний соответственно.

$\bar{k}_w$  Средневзвешенная нормированная длительность:  $\bar{k}_w = \sum_i Z_i k_i / \sum_i Z_i$ .

$\bar{a}_w$  Средневзвешенная нормированная агентная работа:  $\bar{a}_w = \sum_i Z_i a_i^{(r)} / \sum_i Z_i$ .

$\bar{h}_w$  Средневзвешенная нормированная человеческая работа:  $\bar{h}_w = \sum_i Z_i h_i^{(r)} / \sum_i Z_i$ .

$B_0, \dots, B_4$

Нижние оценки времени завершения в последовательных уточнениях М0–М4. Переход между уровнями может добавлять ресурсное ограничение и переопределять эффективные длительности.

$S_E, S_E^{\text{ach}}$

Идеализированный и достижимый коэффициенты ускорения.

$S_{\text{target}}$

Целевой коэффициент ускорения.

$\mu$  Отношение длительности узкого места к исходному времени  $T_h$ :  $\mu = M_N / T_h$  на уровне М1 и  $\mu = L / T_h$  на уровне М2. Символ  $t$  в вычислительных экспериментах зарезервирован для числа частей декомпозиции.

$\nu_i$  Математическое ожидание случайной нормированной длительности:  $\nu_i = \mathbb{E}[k_i]$ .

$b, b_0$  Отношение длительности узкого места к средней нагрузке  $W/P$  и заданная верхняя граница этого отношения, используемая как правило декомпозиции.

$\kappa$  Общий положительный множитель, применяемый к длительностям всех agent- и human-фаз при анализе масштабной инвариантности. Отдельного масштабирования только суммарных  $k_i$  для вывода уровня М4 недостаточно.

## B Detailed Calculation Examples

### B.1 Elementary M0–M1 Examples

Сначала рассмотрим четыре самостоятельных расчётных примера. Во всех случаях базовая трудоёмкость единицы работы принимается равной  $X = 4$  часа.

Идеализированное время выполнения с ИИ вычисляется по формуле:

$$T_{ai}^{(0)} = \frac{1}{P} \sum_{i=1}^N Z_i k_i X.$$

Условие ускорения в идеализированной модели можно записать через порог параллелизма:

$$P > \frac{\sum_{i=1}^N Z_i k_i}{\sum_{i=1}^N Z_i} = \bar{k}_w.$$

Если доступный параллелизм  $P$  не превышает порог  $\bar{k}_w$ , ускорение не достигается. Условие  $P > \bar{k}_w$  необходимо во всех вариантах модели; достаточным оно является только в идеализированной модели идеально делимых задач.

Для сопоставления с более реалистичной моделью неделимых задач дополнительно используется нижняя оценка времени завершения всех работ:

$$T_{ai} \geq \max \left\{ \frac{1}{P} \sum_{i=1}^N Z_i k_i X, \max_{i \in \overline{1, N}} (Z_i k_i X) \right\}.$$

Расчёты выполнены при  $C(P) \equiv 1$  и  $\gamma(P) \equiv 1$ . Примеры 1–4 относятся к уровням M0, где работа считается идеально делимой, и M1, где учитываются независимые неделимые задачи. В иллюстративном сценарии дополнительно рассматриваются уровень M2 с графом зависимостей и уровень M4 с человеком как последовательным ресурсом. Для каждой задачи на уровне M4 выделяется человеческая составляющая  $h_i$  в разложении  $k_i = a_i + h_i$ , вычисляются суммарное человеческое время  $H = X \sum_i Z_i h_i$  и потолок ускорения  $1/\bar{h}_w$ . Делегированная задача разбивается на входную спецификацию, автономную агентную фазу и итоговую проверку. С момента постановки до приёмки она удерживает назначенного агента, в том числе при ожидании свободного разработчика. Расписания проверяются на отсутствие пересечений человеческих интервалов и переиспользования заблокированных потоков; чувствительность к  $\gamma(P) > 1$  обсуждается при сравнении вариантов. Уровень M3, описывающий межагентные координационные издержки  $C(P)$ , в расчётах не используется.

#### B.1.1 Example 1

##### Parameters.

Таблица 5: Параметры примера 1

| Параметр                   | Значение            |
|----------------------------|---------------------|
| $N$ (число задач)          | 3                   |
| $\{Z_i\}$ (сложности)      | $\{2, 3, 1\}$       |
| $\{k_i\}$ (коэфф. ИИ)      | $\{0.5, 0.7, 0.4\}$ |
| $P$ (параллелизм)          | 2                   |
| $X$ (базовая трудоёмкость) | 4 ч                 |

**Calculations.**

$$\begin{aligned}
\sum Z_i &= 2 + 3 + 1 = 6, \\
\sum Z_i k_i &= 2 \cdot 0.5 + 3 \cdot 0.7 + 1 \cdot 0.4 = 3.5, \\
T_h &= 6 \cdot 4 = 24 \text{ ч}, \\
T_{ai}^{(0)} &= \frac{1}{2} \cdot 3.5 \cdot 4 = 7 \text{ ч}, \\
S_E &= \frac{24}{7} \approx 3.43.
\end{aligned}$$

Для модели неделимых задач веса задач с ИИ равны:

$$\{Z_i k_i X\} = \{4.0, 8.4, 1.6\} \text{ ч.}$$

При оптимальном распределении первый исполнитель получает задачу длительностью 8.4 ч, а второй — две задачи общей длительностью  $4.0 + 1.6 = 5.6$  ч. Поэтому достижимое время завершения равно 8.4 ч, а ускорение составляет

$$S_E^{\text{ach}} = \frac{24}{8.4} \approx 2.86.$$

**Interpretation.** Во всех трёх задачах  $k_i < 1$ , поэтому ИИ снижает их трудоёмкость, а параллелизм  $P = 2$  усиливает этот эффект. Идеализированная модель даёт ускорение  $\times 3.43$ . После учёта неделимости задач оно снижается до  $\times 2.86$ , поскольку время завершения ограничено самой длинной задачей продолжительностью 8.4 ч.

**В.1.2 Example 2**

Таблица 6: Параметры примера 2

| Параметр                   | Значение                 |
|----------------------------|--------------------------|
| $N$ (число задач)          | 4                        |
| $\{Z_i\}$ (сложности)      | $\{5, 2, 4, 3\}$         |
| $\{k_i\}$ (коэфф. ИИ)      | $\{0.8, 0.6, 0.9, 0.5\}$ |
| $P$ (параллелизм)          | 3                        |
| $X$ (базовая трудоёмкость) | 4 ч                      |

**Parameters.**

### Calculations.

$$\begin{aligned}\sum Z_i &= 5 + 2 + 4 + 3 = 14, \\ \sum Z_i k_i &= 5 \cdot 0.8 + 2 \cdot 0.6 + 4 \cdot 0.9 + 3 \cdot 0.5 = 10.3, \\ T_h &= 14 \cdot 4 = 56 \text{ ч}, \\ T_{ai}^{(0)} &= \frac{1}{3} \cdot 10.3 \cdot 4 \approx 13.73 \text{ ч}, \\ S_E &= \frac{56}{13.73} \approx 4.08.\end{aligned}$$

Для модели неделимых задач веса задач с ИИ равны:

$$\{Z_i k_i X\} = \{16.0, 4.8, 14.4, 6.0\} \text{ ч}.$$

При оптимальном распределении нагрузки исполнители получают соответственно 16.0, 14.4 и  $6.0 + 4.8 = 10.8$  ч работы. Достижимое время завершения равно 16.0 ч, а ускорение составляет

$$S_E^{\text{ach}} = \frac{56}{16.0} = 3.50.$$

**Interpretation.** В примере 2 для всех задач выполняется  $k_i < 1$ , а коэффициент параллелизма равен  $P = 3$ . Идеализированное ускорение  $\times 4.08$  оказывается наибольшим среди первых трёх примеров. После учёта неделимости время завершения ограничивает задача длительностью 16.0 ч, поэтому достижимое ускорение снижается до  $\times 3.50$ .

### В.1.3 Example 3

Таблица 7: Параметры примера 3

| Параметр                   | Значение            |
|----------------------------|---------------------|
| $N$ (число задач)          | 3                   |
| $\{Z_i\}$ (сложности)      | $\{8, 5, 6\}$       |
| $\{k_i\}$ (коэфф. ИИ)      | $\{1.1, 0.7, 0.8\}$ |
| $P$ (параллелизм)          | 2                   |
| $X$ (базовая трудоёмкость) | 4 ч                 |

### Parameters.

### Calculations.

$$\begin{aligned}\sum Z_i &= 8 + 5 + 6 = 19, \\ \sum Z_i k_i &= 8 \cdot 1.1 + 5 \cdot 0.7 + 6 \cdot 0.8 = 17.1, \\ T_h &= 19 \cdot 4 = 76 \text{ ч}, \\ T_{ai}^{(0)} &= \frac{1}{2} \cdot 17.1 \cdot 4 = 34.2 \text{ ч}, \\ S_E &= \frac{76}{34.2} \approx 2.22.\end{aligned}$$

Для модели неделимых задач веса задач с ИИ равны:

$$\{Z_i k_i X\} = \{35.2, 14.0, 19.2\} \text{ ч.}$$

При оптимальном распределении один исполнитель получает задачу длительностью 35.2 ч, а второй — две задачи общей длительностью  $19.2 + 14.0 = 33.2$  ч. Достижимое время завершения равно 35.2 ч, а ускорение составляет

$$S_E^{\text{ach}} = \frac{76}{35.2} \approx 2.16.$$

**Interpretation.** В примере 3 ускорение сохраняется, хотя для одной задачи ИИ увеличивает трудоёмкость ( $k_i = 1.1$ ). Эта же задача длительностью 35.2 ч ограничивает расписание. Поэтому идеализированное ускорение  $\times 2.22$  лишь немного превышает достижимое значение  $\times 2.16$ ; оба результата ниже, чем в примерах 1 и 2.

#### B.1.4 Example 4

Таблица 8: Параметры примера 4

| Параметр                   | Значение            |
|----------------------------|---------------------|
| $N$ (число задач)          | 3                   |
| $\{Z_i\}$ (сложности)      | $\{4, 3, 3\}$       |
| $\{k_i\}$ (коэфф. ИИ)      | $\{2.4, 2.1, 2.3\}$ |
| $P$ (параллелизм)          | 2                   |
| $X$ (базовая трудоёмкость) | 4 ч                 |

**Parameters.**

**Calculations.**

$$\begin{aligned} \sum Z_i &= 4 + 3 + 3 = 10, \\ \sum Z_i k_i &= 4 \cdot 2.4 + 3 \cdot 2.1 + 3 \cdot 2.3 = 22.8, \\ \bar{k}_w &= \frac{22.8}{10} = 2.28, \\ T_h &= 10 \cdot 4 = 40 \text{ ч}, \\ T_{ai}^{(0)} &= \frac{1}{2} \cdot 22.8 \cdot 4 = 45.6 \text{ ч}, \\ S_E &= \frac{40}{45.6} \approx 0.88. \end{aligned}$$

Пороговое условие ускорения не выполняется:

$$P = 2 < \bar{k}_w = 2.28.$$

Следовательно, доступного параллелизма недостаточно, чтобы компенсировать рост трудоёмкости задач при использовании ИИ.

Для модели неделимых задач веса задач с ИИ равны:

$$\{Z_i k_i X\} = \{38.4, 25.2, 27.6\} \text{ ч.}$$

При  $P = 2$  оптимальное распределение имеет вид  $\{38.4\}$  и  $\{25.2 + 27.6 = 52.8\}$  ч. Поэтому достижимое время завершения равно 52.8 ч, а ускорение оказывается ниже идеализированной оценки:

$$S_E^{\text{ach}} = \frac{40}{52.8} \approx 0.76 < 1.$$

Нижняя оценка времени завершения равна  $\max\{45.6, 38.4\} = 45.6$  ч, но не достигается: оптимальное значение составляет 52.8 ч. Это единственный из четырёх примеров, в котором выражение  $\max\{W/P, \max_i Z_i k_i X\}$  строго меньше оптимума. Тем самым видно, что оно задаёт нижнюю границу, а не гарантированно достижимый результат. Расхождение остаётся в пределах гарантии Грэма:  $52.8 \leq (2 - 1/2) \cdot 45.6 = 68.4$  ч.

Достаточный критерий ускорения также не выполняется. Доля самой длинной задачи равна  $\mu = M_N/T_h = 38.4/40 = 0.96$ , а порог —  $P - (P - 1)\mu = 2 - 0.96 = 1.04$ , тогда как  $\bar{k}_w = 2.28 > 1.04$ . Более того, нарушено даже необходимое условие  $\bar{k}_w < P = 2$ , что согласуется с отсутствием ускорения.

**Interpretation.** В примере 4 все коэффициенты  $k_i$  существенно больше единицы, поэтому ИИ заметно увеличивает трудоёмкость каждой задачи. При  $\bar{k}_w = 2.28$  доступного параллелизма  $P = 2$  недостаточно, чтобы компенсировать это замедление. Даже идеализированная модель показывает рост времени с 40 до 45.6 ч, а после учёта неделимости и неравномерной нагрузки время завершения увеличивается до 52.8 ч.

## B.2 Summary Comparison

Таблица 9: Сводные результаты расчёта

| Показатель                              | Пример 1      | Пример 2      | Пример 3      | Пример 4      |
|-----------------------------------------|---------------|---------------|---------------|---------------|
| $T_h$ (без ИИ), ч                       | 24.0          | 56.0          | 76.0          | 40.0          |
| $T_{ai}^{(0)}$ (с ИИ), ч                | 7.0           | 13.73         | 34.2          | 45.6          |
| Идеализированное ускорение $S_E$        | $\times 3.43$ | $\times 4.08$ | $\times 2.22$ | $\times 0.88$ |
| Достижимое время завершения, ч          | 8.4           | 16.0          | 35.2          | 52.8          |
| Достижимое ускорение $S_E^{\text{ach}}$ | $\times 2.86$ | $\times 3.50$ | $\times 2.16$ | $\times 0.76$ |
| Выполнено условие ускорения?            | Да            | Да            | Да            | Нет           |

**Remark 15** (Final Interpretation). 1. В примерах 1–3 достигается ускорение: как по идеализированной формуле, так и при проверке реалистичной раскладки неделимых задач.

2. Различие между идеализированным и достижимым ускорением обусловлено самой длинной задачей: увеличение параллелизма не может сократить время завершения ниже её длительности.

3. Пример 2 даёт наибольшее ускорение благодаря сочетанию  $k_i < 1$  для всех задач и большего параллелизма  $P = 3$ .

4. Пример 3 показывает более умеренный эффект, поскольку одна из наиболее сложных задач имеет  $k_i > 1$  и становится критическим ограничителем расписания.

5. Пример 4 показывает отрицательный сценарий: даже при  $P = 2$  порог параллелизма не выполнен ( $P < \bar{k}_w$ ), поэтому  $T_{ai}^{(0)} > T_h$  и ускорение не достигается.

### B.3 Full Parameters of the Illustrative Scenario

Перейдём от абстрактных наборов задач к иллюстративному сценарию разработки небольшого микросервиса. Это синтетический сценарий с заданными авторами параметрами, а не эмпирическое исследование конкретного проекта. Один и тот же проект рассматривается в четырёх вариантах, чтобы разделить влияние двух факторов. В вариантах 1–3 при неизменном наборе задач меняется *конфигурация процесса*: параллелизм и режим работы агентов. В варианте 4 сохраняется лучшая из этих конфигураций, но меняется *декомпозиция*: крупная задача на критическом пути разбивается на подзадачи. Такое сравнение позволяет отдельно оценить эффект организации работы и эффект более точного разбиения проекта.

Базовая трудоёмкость единицы работы, как и ранее, принята  $X = 4$  часа.

#### B.3.1 Scenario Definition

Рассмотрим проект небольшого микросервиса для обработки пользовательских заявок. Сервис принимает заявку через HTTP API, проверяет входные данные, сохраняет состояние, запускает асинхронную обработку и предоставляет средства наблюдаемости. Проект достаточно компактен для полного разбора и в то же время содержит типичные элементы промышленной разработки: контракт API, хранение данных, бизнес-правила, фоновую обработку, тесты и инфраструктурную конфигурацию.

Для конкретности будем считать, что сервис должен поддерживать следующий минимальный набор возможностей:

- создание заявки по HTTP-запросу;
- проверку обязательных полей и единый формат ошибок;
- сохранение заявки и её статуса в базе данных;
- перевод заявки по состояниям: создана, принята в обработку, обработана, завершена с ошибкой;
- фоновую обработку с повторными попытками при временных ошибках;
- структурированные журналы, метрики, проверку работоспособности и базовую конфигурацию;
- набор модульных и интеграционных тестов;
- упаковку и минимальную конфигурацию запуска в среде непрерывной интеграции и развёртывания.

Для прозрачной параметризации иллюстративного сценария используем условные единицы сложности story points (SP). Примем, что 2 SP соответствуют одной единице  $Z_i$ . Тогда один story point соответствует  $X/2 = 2$  часам, а стоимость единицы  $Z_i$  по-прежнему равна  $X = 4$  часам. Подзадачам с нечётным числом SP соответствуют дробные значения сложности ( $1 \text{ SP} = 0.5 Z$ ), что согласуется с условием  $Z_i \in \mathbb{Q}^+$

в Определении 1. В вариантах 1–3 расчёты проводятся по крупным задачам, а в варианте 4 непосредственно используется разбиение на подзадачи.

Базовое разбиение проекта на крупные задачи приведено в Таблице 10.

Таблица 10: Базовое разбиение проекта на задачи

| $i$ | Задача                                                     | Оценка, SP | Сложность $Z_i$ |
|-----|------------------------------------------------------------|------------|-----------------|
| 1   | API-контракт и проверка запросов                           | 4          | 2               |
| 2   | Слой хранения (модель, миграции, репозиторий)              | 6          | 3               |
| 3   | Бизнес-логика (основные сценарии и переходы состояний)     | 8          | 4               |
| 4   | Фоновый обработчик (асинхронная обработка и повторы)       | 6          | 3               |
| 5   | Наблюдаемость и конфигурация (логи, метрики, health-check) | 4          | 2               |
| 6   | Тесты и интеграционные проверки                            | 6          | 3               |
| 7   | Упаковка и конфигурация развёртывания (Docker, CI)         | 4          | 2               |

Состав крупных задач раскрыт в Таблице 11. Такое уточнение важно для планирования: на уровне подзадач видно, какие работы можно независимо передавать агентам, а какие требуют заранее зафиксированного контракта или результата другой задачи. Детализация используется далее для выявления зависимостей, возникших из-за укрупнения графа, и в варианте 4, где задача тестирования разбивается на отдельные части.

Суммарная сложность проекта по крупным задачам:

$$\sum_{i=1}^7 Z_i = 2 + 3 + 4 + 3 + 2 + 3 + 2 = 19.$$

Эквивалентная сумма в story points равна 38 SP, что при стоимости 2 часа за SP также даёт 76 часов базовой работы.

Время выполнения без ИИ (последовательная работа одного разработчика):

$$T_h = 19 \cdot 4 = 76 \text{ ч.}$$

Задачи связаны отношениями предшествования. Для планирования параллельной работы зададим граф  $G = (V, E)$ , вершины которого соответствуют крупным задачам из Таблицы 10. Перечень зависимостей приведён в Таблице 12.

В виде рёбер граф можно записать так:

$$E = \{1 \rightarrow 2, 1 \rightarrow 3, 1 \rightarrow 5, 1 \rightarrow 7, 2 \rightarrow 3, 2 \rightarrow 4, 3 \rightarrow 4, 3 \rightarrow 6, 4 \rightarrow 6, 5 \rightarrow 7\}.$$

Из этого графа следует естественное волновое планирование:

1. сначала фиксируется API-контракт и базовые DTO;
2. затем параллельно можно брать хранение и наблюдаемость;
3. после хранения становится доступна полноценная бизнес-логика, а упаковку можно готовить параллельно;



Таблица 11: Подзадачи и story points

| <i>i</i> | Задача             | Подзадача                                               | SP |
|----------|--------------------|---------------------------------------------------------|----|
| 1        | API                | Описать эндпоинты, DTO и формат ошибок                  | 2  |
| 1        | API                | Реализовать проверку данных и обработку ошибок          | 2  |
| 2        | Хранение           | Спроектировать таблицы и миграции                       | 2  |
| 2        | Хранение           | Реализовать репозиторий и методы доступа                | 3  |
| 2        | Хранение           | Добавить базовые тестовые данные для локальной проверки | 1  |
| 3        | Бизнес-логика      | Описать состояния и допустимые переходы                 | 2  |
| 3        | Бизнес-логика      | Реализовать основные сценарии использования             | 4  |
| 3        | Бизнес-логика      | Обработать ошибки доменного уровня                      | 2  |
| 4        | Фоновый обработчик | Реализовать выборку задач в работу                      | 2  |
| 4        | Фоновый обработчик | Добавить повторные попытки и обработку отказов          | 3  |
| 4        | Фоновый обработчик | Синхронизировать статусы с основной моделью             | 1  |
| 5        | Наблюдаемость      | Добавить структурные логи и метрики                     | 2  |
| 5        | Наблюдаемость      | Настроить конфигурацию и проверку работоспособности     | 2  |
| 6        | Тесты              | Покрыть бизнес-логику модульными тестами                | 2  |
| 6        | Тесты              | Добавить интеграционные тесты API и хранения            | 3  |
| 6        | Тесты              | Проверить фоновую обработку и повторные попытки         | 1  |
| 7        | Упаковка           | Подготовить Docker/service-конфигурацию                 | 2  |
| 7        | Упаковка           | Добавить команды непрерывной интеграции и дымовой тест  | 2  |

Таблица 12: Граф зависимостей между крупными задачами

| Задача                              | Зависит от | Что можно делать параллельно                                          |
|-------------------------------------|------------|-----------------------------------------------------------------------|
| 1. API-контракт и проверка запросов | —          | Стартовая задача; задаёт интерфейсы для остальных работ               |
| 2. Слой хранения                    | 1          | Можно выполнять параллельно с наблюдаемостью после фиксации API       |
| 3. Бизнес-логика                    | 1, 2       | Частично проектируется после API, реализация требует хранения         |
| 4. Фоновый обработчик               | 2, 3       | Требует модели данных и правил переходов состояний                    |
| 5. Наблюдаемость и конфигурация     | 1          | Можно вести параллельно с хранением и бизнес-логикой                  |
| 6. Тесты и интеграционные проверки  | 1, 2, 3, 4 | Модульные тесты можно начинать раньше, интеграционные зависят от ядра |
| 7. Упаковка и развёртывание         | 1, 5       | Можно готовить параллельно после фиксации интерфейсов и конфигурации  |

4. фоновый обработчик зависит от бизнес-логики и модели данных;
5. интеграционные тесты и проверки фоновой обработки завершают критический путь.

Таким образом, в проекте сочетаются параллелизуемые участки и жёсткие зависимости. В вариантах 1–3 используются агрегированные значения  $Z_i$  и коэффициенты  $k_i$ , однако время завершения определяется уже не балансировкой независимых задач, а расписанием на графе из Таблицы 12.

**Actual Dependencies and Coarsening Artifacts.** Укрупнённый граф скрывает важное различие. Одни рёбра Таблицы 12 отражают *содержательные* зависимости: бизнес-логика не может опираться на ещё не созданную модель данных, а фоновый обработчик — на неопределённые переходы состояний. Другие рёбра возникают только из-за *укрупнения задач*: связь проведена между задачами целиком, хотя фактически зависимы лишь отдельные подзадачи. Например, зависимость задачи 6 от задач 1–4 означает, что набор тестов содержит проверки каждого компонента. При этом модульные тесты бизнес-логики можно запускать сразу после задачи 3, интеграционные тесты — после завершения API и слоя хранения, а только проверки повторных попыток требуют готового фонового обработчика. Сходным образом зависимость  $3 \rightarrow 4$  на уровне подзадач относится лишь к модели состояний и переходов объёмом 2 SP из 8; остальные сценарии бизнес-логики обработчику не нужны. Содержательную зависимость нельзя устранить перепланированием, тогда как зависимость, вызванную укрупнением, можно уточнить, заменив одну вершину несколькими подзадачами. Поэтому доминирование критического пути в последующих расчётах отчасти определяется выбранной гранулярностью графа. Вариант 4 показывает этот эффект количественно.

В вариантах 1–3 проект и набор крупных задач неизменны, но различаются конфигурации процесса  $\sigma = (P, r)$ : число параллельных потоков  $P$ , режимы работы агентов  $r$  и распределение задач. Поскольку коэффициент  $k_i = k_i^{(r)}$  зависит от тройки «задача  $\times$  инструмент  $\times$  режим», интерактивная работа в варианте 1, смешанный режим в варианте 2 и делегированная работа по подробным спецификациям в варианте 3 задают разные векторы  $\{k_i\}$  для одного набора задач. Инструмент и правила приёмки в сценарии фиксированы. Следовательно, сравниваются конфигурации целиком, а не изолированное изменение одного параметра. В варианте 4 воспроизводится конфигурация варианта 3 и меняется только декомпозиция, что позволяет отдельно оценить её влияние.

На уровне M4 явно учитывается конкуренция потоков за внимание разработчика. Для каждой задачи задана человеческая составляющая  $h_i$ , включающая постановку, проверку и направление исправлений. Расписание считается допустимым только при отсутствии пересечений между человеческими интервалами разных задач и при непрерывном закреплении одного агента за каждой начатой, но ещё не принятой задачей. Если человеческая фаза не может начаться немедленно, агент остаётся заблокированным в очереди; другие агенты продолжают работу. В интерактивном режиме  $h_i = k_i$ , поскольку разработчик сохраняет контекст задачи в течение всего времени её выполнения; в делегированном режиме  $h_i$  заметно меньше  $k_i$ , но не равно нулю из-за постановки и приёмки. Значения  $h_i$ , как и  $k_i$ , заданы экспертно. Сохраняются два упрощающих предположения:  $C(P) \equiv 1$ , то есть межагентные интеграционные

издержки не моделируются, и  $\gamma(P) \equiv 1$ . Влияние затрат на переключение контекста рассматривается отдельно при сравнении вариантов.

### B.3.2 Variant 1. Interactive Work in a Single Stream

Разработчик последовательно решает задачи с помощью ИИ-ассистента. Для каждой задачи он формулирует запрос, получает результат, проверяет и при необходимости исправляет его. Параллелизм отсутствует ( $P = 1$ ). Конфигурация имеет вид  $\sigma_1 = (P=1, \text{ интерактивный режим})$ .

Таблица 13: Параметры варианта 1

| $i$ | Задача                           | $Z_i$ | $k_i$ | $h_i$ |
|-----|----------------------------------|-------|-------|-------|
| 1   | API-контракт и проверка запросов | 2     | 0.80  | 0.80  |
| 2   | Слой хранения                    | 3     | 0.90  | 0.90  |
| 3   | Бизнес-логика                    | 4     | 0.85  | 0.85  |
| 4   | Фоновый обработчик               | 3     | 0.95  | 0.95  |
| 5   | Наблюдаемость и конфигурация     | 2     | 0.75  | 0.75  |
| 6   | Тесты                            | 3     | 0.80  | 0.80  |
| 7   | Упаковка и развёртывание         | 2     | 0.90  | 0.90  |

**Parameters.** Коэффициенты  $k_i$  отражают умеренное ускорение: ИИ помогает генерировать шаблонный код и тесты, но каждый результат проходит через ручную проверку. Для задач с высокой архитектурной сложностью (бизнес-логика, фоновый обработчик) коэффициент ближе к единице. Человеческая доля  $h_i = k_i$ : в интерактивном режиме разработчик удерживает контекст задачи всё её время (агентная составляющая  $a_i = 0$ ).

#### Calculations.

$$\begin{aligned} \sum_{i=1}^7 Z_i k_i &= 2 \cdot 0.80 + 3 \cdot 0.90 + 4 \cdot 0.85 + 3 \cdot 0.95 + 2 \cdot 0.75 + 3 \cdot 0.80 + 2 \cdot 0.90 \\ &= 1.60 + 2.70 + 3.40 + 2.85 + 1.50 + 2.40 + 1.80 = 16.25. \end{aligned}$$

Средневзвешенная нормированная длительность:

$$\bar{k}_w = \frac{16.25}{19} \approx 0.855.$$

Веса задач на графе равны  $w_i = Z_i k_i X$ :

$$\{w_i\}_{i=1}^7 = \{6.4, 10.8, 13.6, 11.4, 6.0, 9.6, 7.2\} \text{ ч.}$$

Полная работа:

$$W = \sum_{i=1}^7 w_i = 16.25 \cdot 4 = 65.0 \text{ ч.}$$

Идеализированное время при независимых задачах, то есть нижняя оценка по суммарной работе при  $P = 1$ :

$$T_{ai}^{(0)} = \frac{W}{P} = \frac{65.0}{1} = 65.0 \text{ ч.}$$

Идеализированное ускорение без учёта зависимостей:

$$S_E = \frac{76}{65.0} \approx 1.17.$$

Критический путь графа проходит через задачи  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6$ :

$$L = 6.4 + 10.8 + 13.6 + 11.4 + 9.6 = 51.8 \text{ ч.}$$

Нижняя оценка времени завершения с учётом графа зависимостей:

$$B_4 = \max \left\{ \frac{W}{P}, L, H \right\} = \max \{65.0, 51.8, 65.0\} = 65.0 \text{ ч.}$$

При  $P = 1$  расписание последовательно. Задачи можно выполнить, например, в топологическом порядке 1, 2, 3, 4, 5, 6, 7. Время завершения равно полному объёму работы:

$$T(\pi_1) = T^* = B_4 = 65.0 \text{ ч.}$$

Достижимое ускорение с учётом графа:

$$S_E^{\text{ach}} = \frac{76}{65.0} \approx 1.17.$$

Человеческий ресурс (уровень М4): поскольку  $h_i = k_i$ ,

$$H = X \sum_{i=1}^7 Z_i h_i = 65.0 \text{ ч} = W, \quad \bar{h}_w = \bar{k}_w \approx 0.855, \quad \frac{1}{\bar{h}_w} \approx 1.17.$$

**Interpretation.** Интерактивный режим обеспечивает непосредственный контроль результата. ИИ снижает трудоёмкость каждой задачи ( $\bar{k}_w \approx 0.855$ ), но отсутствие параллелизма ограничивает общий эффект. Критический путь не становится активным ограничением: при  $P = 1$  величина  $W/P = 65.0$  ч превышает  $L = 51.8$  ч. Ускорение составляет  $\times 1.17$ , что соответствует экономии примерно 11 часов из исходных 76.

На уровне М4 достигнутое ускорение  $\times 1.17$  в точности совпадает с потолком по человеческому ресурсу  $1/\bar{h}_w$ . В интерактивном режиме время завершения равно суммарному человеческому времени  $H$ . Следовательно, дальнейшая реорганизация процесса не даст выигрыша без снижения  $h$ , то есть без передачи части работы агентам для автономного выполнения.

### B.3.3 Variant 2. Interactive and Delegated Tasks

В этом варианте доступны два агентных потока ( $P = 2$ ). Задачи, требующие архитектурных решений, выполняются интерактивно, а более формализованные задачи делегируются агенту и принимаются разработчиком после автономной фазы. Режим относится к задаче, а не закрепляется за конкретным агентом: свободный агент может получить следующую готовую задачу любого предусмотренного режима.

Задачи распределяются по режимам следующим образом:

- **Интерактивный режим:** API-контракт, бизнес-логика, фоновый обработчик — задачи, требующие контекста и архитектурных решений.
- **Делегированный режим:** слой хранения, наблюдаемость, тесты, упаковка — задачи с более формализованными требованиями.

Конфигурация:  $\sigma_2 = (P=2, \rho)$  со смешанным назначением режимов  $\rho$  — задачи потока 1 выполняются в интерактивном режиме, задачи потока 2 — в делегированном.

Таблица 14: Параметры варианта 2

| $i$ | Задача                           | $Z_i$ | $k_i$ | $h_i$ | Режим  |
|-----|----------------------------------|-------|-------|-------|--------|
| 1   | API-контракт и проверка запросов | 2     | 0.75  | 0.75  | интер. |
| 2   | Слой хранения                    | 3     | 0.75  | 0.20  | делег. |
| 3   | Бизнес-логика                    | 4     | 0.85  | 0.85  | интер. |
| 4   | Фоновый обработчик               | 3     | 0.90  | 0.90  | интер. |
| 5   | Наблюдаемость и конфигурация     | 2     | 0.70  | 0.20  | делег. |
| 6   | Тесты                            | 3     | 0.70  | 0.20  | делег. |
| 7   | Упаковка и развёртывание         | 2     | 0.80  | 0.20  | делег. |

**Parameters.** В делегированном режиме коэффициенты  $k_i$  несколько ниже, чем в варианте 1. Это следствие смены режима: агент работает самостоятельно и не ожидает промежуточных ответов разработчика. Такой эффект проявляется уже в одном потоке и согласуется с определением  $k_i^{(r)}$  как характеристики тройки «задача  $\times$  инструмент  $\times$  режим»; инструмент в сценарии фиксирован. Для интерактивных задач коэффициенты близки к варианту 1, а  $h_i = k_i$ , поскольку разработчик занят в течение всего времени выполнения. Для делегированных задач принимается  $h_i = 0.20$ ; эта доля включает постановку и проверку результата. В расписании человеческое время делится поровну между входной спецификацией и итоговой проверкой, а агентный поток удерживается на всём интервале от начала спецификации до завершения проверки.

#### Calculations.

$$\begin{aligned} \sum_{i=1}^7 Z_i k_i &= 2 \cdot 0.75 + 3 \cdot 0.75 + 4 \cdot 0.85 + 3 \cdot 0.90 + 2 \cdot 0.70 + 3 \cdot 0.70 + 2 \cdot 0.80 \\ &= 1.50 + 2.25 + 3.40 + 2.70 + 1.40 + 2.10 + 1.60 = 14.95. \end{aligned}$$

Средневзвешенная нормированная длительность:

$$\bar{k}_w = \frac{14.95}{19} \approx 0.787.$$

Веса задач на графе равны  $w_i = Z_i k_i X$ :

$$\{w_i\}_{i=1}^7 = \{6.0, 9.0, 13.6, 10.8, 5.6, 8.4, 6.4\} \text{ ч.}$$

Полная работа:

$$W = \sum_{i=1}^7 w_i = 14.95 \cdot 4 = 59.8 \text{ ч.}$$

Идеализированное время при независимых задачах, то есть нижняя оценка по суммарной работе при  $P = 2$ :

$$T_{ai}^{(0)} = \frac{W}{P} = \frac{59.8}{2} = 29.9 \text{ ч.}$$

Идеализированное ускорение без учёта зависимостей:

$$S_E = \frac{76}{29.9} \approx 2.54.$$

Критический путь графа проходит через задачи  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6$ :

$$L = 6.0 + 9.0 + 13.6 + 10.8 + 8.4 = 47.8 \text{ ч.}$$

Человеческое время (уровень M4):

$$\sum_{i=1}^7 Z_i h_i = 2 \cdot 0.75 + 3 \cdot 0.20 + 4 \cdot 0.85 + 3 \cdot 0.90 + 2 \cdot 0.20 + 3 \cdot 0.20 + 2 \cdot 0.20 = 9.6,$$

$$H = 9.6 \cdot 4 = 38.4 \text{ ч}, \quad \bar{h}_w = \frac{9.6}{19} \approx 0.505, \quad \frac{1}{\bar{h}_w} \approx 1.98.$$

Нижняя оценка времени завершения с учётом графа зависимостей и человеческого ресурса:

$$B_4 = \max \left\{ \frac{W}{P}, L, H \right\} = \max \{ 29.9, 47.8, 38.4 \} = 47.8 \text{ ч.}$$

Раскладка по потокам из Таблицы 14, соблюдающая зависимости:

| Поток | Интервалы задач, ч                                                | Простой, ч                          |
|-------|-------------------------------------------------------------------|-------------------------------------|
| 1     | 1: [0, 6.0]; 3: [15.0, 28.6]; 4: [28.6, 39.4]                     | [6.0, 15.0], ожидание слоя хранения |
| 2     | 2: [6.0, 15.0]; 5: [15.0, 20.6]; 7: [20.6, 27.0]; 6: [39.4, 47.8] | [27.0, 39.4], ожидание зависимостей |

Если учитывать только два агентных потока и отношения предшествования, эта раскладка имеет срок 47.8 ч и совпадает с нижней оценкой. Однако допустимым расписанием уровня M4 она не является, как показывает следующая проверка.

$$T_{\text{agents}} = B_4 = 47.8 \text{ ч.}$$

**Human-Resource Check (Level M4).** Приведённое расписание допустимо по потокам и зависимостям, но *невыполнимо по человеческому ресурсу*. Разработчик непрерывно занят интерактивными задачами на интервале [15.0, 39.4]: сначала задач 3, затем задач 4. В то же время делегированные задачи 5 [15.0, 20.6] и 7 [20.6, 27.0] требуют по 1.6 ч участия человека для постановки и проверки. Эти интервалы пересекаются с интерактивной работой, что невозможно при одном разработчике. Следовательно, формально допустимое по агентным потокам расписание не предусматривает ресурса для своевременной приёмки делегированных результатов.

Конфликт устраним перестройкой расписания: делегированные задачи и их человеческие фазы размещаются в окнах, когда разработчик свободен. При этом начатая задача непрерывно удерживает назначенного агента, включая ожидание итоговой проверки. Скорректированное расписание:

- Агент 1: задача 1 [0, 6.0]; задача 2 [6.0, 15.0]; задача 3 [15.0, 28.6]; задача 4 [28.6, 39.4]; задача 6 [39.4, 47.8].
- Агент 2: задача 5 [7.2, 12.8]; задача 7 [12.8, 41.4]. В задаче 7 спецификация занимает [12.8, 13.6], автономная работа — [13.6, 18.4], ожидание проверки — [18.4, 40.6], проверка — [40.6, 41.4].
- Человек: задача 1 [0, 6.0]; спецификация задачи 2 [6.0, 7.2]; спецификация задачи 5 [7.2, 8.0]; проверка задачи 5 [12.0, 12.8]; спецификация задачи 7 [12.8, 13.6]; проверка задачи 2 [13.8, 15.0]; задача 3 [15.0, 28.6]; задача 4 [28.6, 39.4]; спецификация задачи 6 [39.4, 40.6]; проверка задачи 7 [40.6, 41.4]; проверка задачи 6 [46.6, 47.8]. Интервалы человека попарно не пересекаются, суммарная занятость равна 38.4 ч =  $H$ .

Все зависимости соблюдены. Задача 3 начинается в момент 15.0 после полной приёмки задачи 2; задача 7 — после приёмки задачи 5; задача 6 — после задачи 4. Период ожидания задачи 7 не используется для другой работы на агенте 2. Несмотря на эту локальную блокировку, время завершения сохраняется равным 47.8 ч, поскольку задача 7 находится вне критического пути. Таким образом, человеческий ресурс не увеличивает срок только при явном планировании фаз и очереди; исходное расписание было невыполнимым.

Скорректированное расписание допустимо и достигает нижней оценки, поэтому оно оптимально:

$$T(\pi_2) = T^* = B_4 = 47.8 \text{ ч.}$$

Достижимое ускорение с учётом графа и человеческого ресурса:

$$S_E^{\text{ach}} = \frac{76}{47.8} \approx 1.59.$$

**Interpretation.** Если считать задачи независимыми, нижняя оценка  $W/P = 29.9$  ч соответствовала бы ускорению примерно  $\times 2.54$ . Однако граф зависимостей существенно сокращает доступный параллелизм: задачи  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6$  образуют критический путь длиной 47.8 ч. С учётом графа достижимое ускорение снижается до  $\times 1.59$ . Наблюдаемость и развёртывание действительно выполняются параллельно, но находятся вне критического пути и почти не влияют на общий срок.

Ключевое наблюдение: параллелизм  $P = 2$  помогает только там, где граф допускает независимую работу. В этом варианте большая часть трудоёмкости оказывается свернута в последовательную цепочку, поэтому балансировка потоков без учёта зависимостей завышает эффект конфигурации в иллюстративном сценарии.

Человеческий ресурс пока не является активным ограничением:  $H = 38.4$  ч  $< L = 47.8$  ч, а достигнутое ускорение ниже потолка  $1/\bar{h}_w \approx 1.98$  ( $1.59 < 1.98$ ). Однако этот резерв реализуется только при явном планировании окон проверки. Три интерактивные задачи с  $h = k$  занимают разработчика в общей сложности на 30.4 ч, поэтому проверки делегированных задач необходимо размещать в оставшихся промежутках.

### B.3.4 Variant 3. Delegated Agents with Detailed Specifications

В этом варианте четыре ИИ-агента работают параллельно ( $P = 4$ ). Конфигурация задаётся как  $\sigma_3 = (P=4, \text{ делегированный режим с подробными спецификациями})$ .

Коэффициенты  $k_i$  ниже, чем в предыдущих вариантах, благодаря смене профиля режима, а не увеличению параллелизма: агенты получают подробные спецификации и автономно выполняют агентные фазы, после чего результат принимает разработчик. Граф зависимостей между крупными задачами остаётся прежним и ограничивает доступный параллелизм.

Таблица 15: Параметры варианта 3

| $i$ | Задача                           | $Z_i$ | $k_i$ | $h_i$ |
|-----|----------------------------------|-------|-------|-------|
| 1   | API-контракт и проверка запросов | 2     | 0.65  | 0.25  |
| 2   | Слой хранения                    | 3     | 0.70  | 0.25  |
| 3   | Бизнес-логика                    | 4     | 0.80  | 0.25  |
| 4   | Фоновый обработчик               | 3     | 0.75  | 0.25  |
| 5   | Наблюдаемость и конфигурация     | 2     | 0.60  | 0.25  |
| 6   | Тесты                            | 3     | 0.65  | 0.25  |
| 7   | Упаковка и развёртывание         | 2     | 0.70  | 0.25  |

**Parameters.** Для всех задач принята одинаковая человеческая составляющая  $h_i = 0.25$ . Участие разработчика состоит в подготовке подробной спецификации и последующей проверке результата; реализация полностью передаётся агенту.

**Calculations.**

$$\begin{aligned} \sum_{i=1}^7 Z_i k_i &= 2 \cdot 0.65 + 3 \cdot 0.70 + 4 \cdot 0.80 + 3 \cdot 0.75 + 2 \cdot 0.60 + 3 \cdot 0.65 + 2 \cdot 0.70 \\ &= 1.30 + 2.10 + 3.20 + 2.25 + 1.20 + 1.95 + 1.40 = 13.40. \end{aligned}$$

Средневзвешенная нормированная длительность:

$$\bar{k}_w = \frac{13.40}{19} \approx 0.705.$$

Веса задач на графе равны  $w_i = Z_i k_i X$ :

$$\{w_i\}_{i=1}^7 = \{5.2, 8.4, 12.8, 9.0, 4.8, 7.8, 5.6\} \text{ ч.}$$

Полная работа:

$$W = \sum_{i=1}^7 w_i = 13.40 \cdot 4 = 53.6 \text{ ч.}$$

Идеализированное время при независимых задачах, то есть нижняя оценка по суммарной работе при  $P = 4$ :

$$T_{ai}^{(0)} = \frac{W}{P} = \frac{53.6}{4} = 13.4 \text{ ч.}$$

Идеализированное ускорение без учёта зависимостей:

$$S_E = \frac{76}{13.4} \approx 5.67.$$



Критический путь графа проходит через задачи  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6$ :

$$L = 5.2 + 8.4 + 12.8 + 9.0 + 7.8 = 43.2 \text{ ч.}$$

Человеческое время (уровень M4):

$$\sum_{i=1}^7 Z_i h_i = 0.25 \cdot 19 = 4.75, \quad H = 4.75 \cdot 4 = 19.0 \text{ ч}, \quad \bar{h}_w = 0.25, \quad \frac{1}{\bar{h}_w} = 4.0.$$

Нижняя оценка времени завершения с учётом графа зависимостей и человеческого ресурса:

$$B_4 = \max \left\{ \frac{W}{P}, L, H \right\} = \max \{13.4, 43.2, 19.0\} = 43.2 \text{ ч.}$$

Одна допустимая раскладка по четырём агентам, соблюдающая зависимости:

- Агент 1: задача 1  $[0, 5.2]$ , задача 2  $[5.2, 13.6]$ , задача 3  $[13.6, 26.4]$ , задача 4  $[26.4, 35.4]$ , задача 6  $[35.4, 43.2]$ .
- Агент 2: задача 5  $[6.7, 11.5]$ , задача 7  $[15.6, 21.2]$ .
- Агенты 3 и 4 простаивают, потому что при таком крупном графе критический путь доминирует над доступным параллелизмом.

Это расписание достигает нижней оценки:

$$T(\pi_3) = T^* = B_4 = 43.2 \text{ ч.}$$

**Human-Resource Check (Level M4).** Чтобы агент не начинал работу до постановки задачи, человеческое время делится поровну между подготовкой входной спецификации и итоговой проверкой; между этими этапами агент выполняет реализацию. Блоки разработчика размещаются так: задача 1 —  $[0, 1.0]$  и  $[4.2, 5.2]$ ; задача 2 —  $[5.2, 6.7]$  и  $[12.1, 13.6]$ ; задача 5 —  $[6.7, 7.7]$  и  $[10.5, 11.5]$ ; задача 3 —  $[13.6, 15.6]$  и  $[24.4, 26.4]$ ; задача 7 —  $[15.6, 16.6]$  и  $[20.2, 21.2]$ ; задача 4 —  $[26.4, 27.9]$  и  $[33.9, 35.4]$ ; задача 6 —  $[35.4, 36.9]$  и  $[41.7, 43.2]$ . Интервалы попарно не пересекаются, а каждая входная спецификация предшествует агентной части соответствующей задачи. Суммарная занятость человека равна  $19.0 \text{ ч} = H$  при длительности проекта  $43.2 \text{ ч}$ , поэтому расписание выполнимо без перестройки.

Достижимое ускорение с учётом графа и человеческого ресурса:

$$S_E^{\text{ach}} = \frac{76}{43.2} \approx 1.76.$$

**Interpretation.** При четырёх агентах суммарная работа снижается до  $W = 53.6 \text{ ч}$ , а оценка для независимых задач составляет  $W/P = 13.4 \text{ ч}$ . Однако при укрупнённом графе это значение недостижимо: цепочка  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6$  имеет длину  $43.2 \text{ ч}$  и становится главным ограничением. Поэтому после учёта зависимостей ускорение составляет не  $\times 5.67$ , а  $\times 1.76$ .

Улучшение относительно варианта 2 следует интерпретировать осторожно. Критический путь сокращается с  $47.8$  до  $43.2 \text{ ч}$  за счёт уменьшения коэффициентов  $k_i$  при переходе к делегированному режиму с подробными спецификациями. Сам рост  $P$  с 2 до 4 не влияет на  $L$ : дополнительные агенты не ускоряют последовательную

цепочку крупных задач и в данном расписании простаивают. Следующее изменение сценария состоит не в дальнейшем увеличении числа агентов, а в разбиении крупных задач критического пути на меньшие, независимо проверяемые части. Именно такое изменение рассматривается в варианте 4.

Уровень М4 задаёт дополнительную верхнюю границу. В данном варианте потолок по человеческому ресурсу  $1/\bar{h}_w = 4.0$  заметно выше достигнутого ускорения  $\times 1.76$ , поэтому активным ограничением остаётся  $L$ . Тем не менее при выбранном режиме ( $\bar{h}_w = 0.25$ ) никакая декомпозиция и никакое число агентов не обеспечат ускорение выше  $\times 4.0$ : 19 часов последовательного человеческого времени устранить нельзя. По мере сокращения критического пути активным ограничением станет  $H$ .

### B.3.5 Variant 4. The Same Configuration with Finer Decomposition

От варианта 3 этот вариант отличается только декомпозицией. Число агентов  $P = 4$ , делегированный режим с подробными спецификациями, коэффициенты  $k$  и человеческие доли  $h$  сохраняются. Меняется лишь граф: крупная задача 6 «Тесты» заменяется тремя подзадачами из Таблицы 11, для каждой из которых задаются точные зависимости. В вариантах 1–3 конфигурация менялась при фиксированном разбиении; здесь, напротив, разбиение меняется при фиксированной конфигурации. Поэтому различие с вариантом 3 обусловлено только декомпозицией.

**Parameters.** Задачи 1–5 и 7 не меняются (Таблица 15). Задача 6 дробится согласно Таблице 11:

Таблица 16: Подзадачи задачи 6 в варианте 4

| $i$ | Подзадача                             | SP | $Z_i$ | $k_i / h_i$ | Зависит от |
|-----|---------------------------------------|----|-------|-------------|------------|
| 6а  | Модульные тесты бизнес-логики         | 2  | 1     | 0.65 / 0.25 | 3          |
| 6б  | Интеграционные тесты API и хранения   | 3  | 1.5   | 0.65 / 0.25 | 1, 2       |
| 6в  | Проверка фоновой обработки и повторов | 1  | 0.5   | 0.65 / 0.25 | 4          |

Предполагается, что разбиение *не меняет объём работы*: подзадачи наследуют  $k = 0.65$  и  $h = 0.25$  исходной задачи. Поэтому величины  $W$ ,  $\bar{k}_w$  и  $H$  сохраняются, а сравнение с вариантом 3 выделяет структурный эффект. На практике декомпозиция требует дополнительных затрат на спецификацию, постановку и приёмку каждой подзадачи. По принятому правилу учёта они входят в  $k^{(r)}$  и при чрезмерном дроблении увеличивают его. С другой стороны, для небольших задач с чёткими границами  $k$  может снижаться. Таким образом, неизменность  $k$  и  $h$  — нейтральное сценарное предположение, а чувствительность результата к издержкам декомпозиции требует отдельной калибровки.

Сложности подзадач дробные ( $Z_{6б} = 1.5$ ,  $Z_{6в} = 0.5$ ): нечётное число SP при курсе 2 SP = 1 Z. Веса подзадач:

$$w_{6а} = 1 \cdot 0.65 \cdot 4 = 2.6 \text{ ч}, \quad w_{6б} = 1.5 \cdot 0.65 \cdot 4 = 3.9 \text{ ч}, \quad w_{6в} = 0.5 \cdot 0.65 \cdot 4 = 1.3 \text{ ч};$$

в сумме 7.8 ч — вес исходной задачи 6.

**Calculations.** Агрегаты совпадают с вариантом 3:

$$W = 53.6 \text{ ч}, \quad \bar{k}_w \approx 0.705, \quad \frac{W}{P} = 13.4 \text{ ч}, \quad H = 19.0 \text{ ч}, \quad \frac{1}{\bar{h}_w} = 4.0.$$

Меняется критический путь. Монолитная вершина 6 и входящие в неё рёбра  $3 \rightarrow 6$ ,  $4 \rightarrow 6$  заменены тремя вершинами с точными зависимостями  $3 \rightarrow 6a$ ,  $\{1, 2\} \rightarrow 6b$ ,  $4 \rightarrow 6в$ , и самым длинным путём становится

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6в : \quad L = 5.2 + 8.4 + 12.8 + 9.0 + 1.3 = 36.7 \text{ ч}$$

(конкурирующие пути короче:  $1 \rightarrow 2 \rightarrow 3 \rightarrow 6a$  даёт 29.0 ч,  $1 \rightarrow 2 \rightarrow 6b$  — 17.5 ч,  $1 \rightarrow 5 \rightarrow 7$  — 15.6 ч). Хвост критического пути сократился с 7.8 ч (вся задача 6) до 1.3 ч (только та её часть, которая действительно ждёт фонового обработчика).

Нижняя оценка времени завершения:

$$B_4 = \max \left\{ \frac{W}{P}, L, H \right\} = \max\{13.4, 36.7, 19.0\} = 36.7 \text{ ч}.$$

Допустимая раскладка, соблюдающая зависимости:

- Агент 1: задача 1 [0, 5.2], задача 2 [5.2, 13.6], задача 3 [13.6, 26.4], задача 4 [26.4, 35.4], задача 6в [35.4, 36.7].
- Агент 2: задача 5 [6.7, 11.5]; для задачи 7 входная спецификация занимает [19.5, 20.5], агентная реализация — [20.5, 24.1], после чего результат ожидает освобождения разработчика до проверки [28.4, 29.4]. На интервале ожидания [24.1, 28.4] агент остаётся закреплён за задачей 7 и не может получить другую работу.
- Агент 3: задача 6б [15.6, 19.5], задача 6a [27.9, 30.5]. Агент 4 простаивает.

Это расписание достигает нижней оценки:

$$T(\pi_4) = T^* = B_4 = 36.7 \text{ ч}.$$

**Human-Resource Check (Level M4).** Как и в варианте 3, человеческое время каждой задачи делится поровну между входной спецификацией и итоговой проверкой. Блоки разработчика: задача 1 — [0, 1.0] и [4.2, 5.2]; задача 2 — [5.2, 6.7] и [12.1, 13.6]; задача 5 — [6.7, 7.7] и [10.5, 11.5]; задача 3 — [13.6, 15.6] и [24.4, 26.4]; задача 6б — [15.6, 16.35] и [18.75, 19.5]; задача 7 — [19.5, 20.5] и [28.4, 29.4]; задача 4 — [26.4, 27.9] и [33.9, 35.4]; задача 6a — [27.9, 28.4] и [30.0, 30.5]; задача 6в — [35.4, 35.65] и [36.45, 36.7]. Интервалы попарно не пересекаются, каждая входная спецификация предшествует агентной реализации, а суммарная занятость человека равна  $19.0 \text{ ч} = H$ . Следовательно, расписание допустимо по человеческому ресурсу и действительно достигает 36.7 ч.

Достижимое ускорение с учётом графа и человеческого ресурса:

$$S_E^{\text{ach}} = \frac{76}{36.7} \approx 2.07.$$

**Interpretation.** Время завершения сократилось с 43.2 до 36.7 ч, а ускорение выросло с  $\times 1.76$  до  $\times 2.07$ , то есть на 18%, без добавления агентов, смены режима или улучшения инструмента. Эффект получен за счёт замены одной крупной вершины тремя подзадачами, уже выделенными в Таблице 11. Зависимость «тестирование после завершения всего ядра» оказалась следствием укрупнения графа; её уточнение сократило критический путь на 6.5 ч. В рассматриваемом сравнении результат определяется декомпозицией, а не числом агентов.

Только переход от варианта 3 к варианту 4 представляет собой сравнение при прочих равных: сокращение времени с 43.2 до 36.7 ч обусловлено декомпозицией при неизменных  $P$ ,  $k$  и  $h$ . При переходе от варианта 2 к варианту 3 одновременно меняются  $P$  и режим, поэтому их влияние не разделено. Тем не менее расчёт показывает, что уменьшение активной границы  $L$  с 47.8 до 43.2 ч связано со снижением  $k$  на критическом пути. Само значение  $P$  в формулу  $L$  не входит, а дополнительные агенты простаивают.

Сравнительный вывод для вариантов 3 и 4 сохраняется и без точной абсолютной калибровки. Согласно Предложению 5, умножение всех агентных и человеческих фаз обоих вариантов на один общий множитель  $\kappa > 0$  изменит оба времени завершения в  $\kappa$  раз, но оставит неизменными отношение 43.2/36.7 и ранжирование вариантов. Эта устойчивость не распространяется на неодинаковые ошибки по задачам или на изменение относительных профилей  $k_i$  и  $h_i$ .

Декомпозицию можно продолжить итеративно. Следующий кандидат — зависимость  $3 \rightarrow 4$ , которая на уровне подзадач сводится к модели состояний объёмом 2 SP из 8. Если выделить её в отдельную раннюю подзадачу, фоновый обработчик можно начинать до завершения остальных сценариев бизнес-логики. Предел такого дробления уже замечен: путь  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$  длиной 35.4 ч почти полностью определяет  $L = 36.7$  ч, а снизу время ограничено человеческим ресурсом  $H = 19.0$  ч и потолком ускорения  $\times 4.0$ . По мере дальнейшего сокращения критического пути активным ограничением станет  $H$ , как предсказывает уровень M4.

### B.3.6 Comparison of Variants

Таблица 17: Сравнение вариантов организации работы с учётом графа зависимостей

| Показатель            | Вариант 1     | Вариант 2     | Вариант 3       | Вариант 4        |
|-----------------------|---------------|---------------|-----------------|------------------|
| Параллелизм $P$       | 1             | 2             | 4               | 4                |
| Режим                 | интерактивный | смешанный     | делег. со спец. | делег. со спец.  |
| Декомпозиция          | крупная       | крупная       | крупная         | дроблённые тесты |
| $W$ , ч               | 65.0          | 59.8          | 53.6            | 53.6             |
| $W/P$ , ч             | 65.0          | 29.9          | 13.4            | 13.4             |
| $L$ , ч               | 51.8          | 47.8          | 43.2            | 36.7             |
| $H$ , ч               | 65.0          | 38.4          | 19.0            | 19.0             |
| $B_4$ , ч             | 65.0          | 47.8          | 43.2            | 36.7             |
| $T(\pi)$ , ч          | 65.0          | 47.8          | 43.2            | 36.7             |
| $S_E^{\text{ach}}$    | $\times 1.17$ | $\times 1.59$ | $\times 1.76$   | $\times 2.07$    |
| Потолок $1/\bar{h}_w$ | $\times 1.17$ | $\times 1.98$ | $\times 4.00$   | $\times 4.00$    |

Из таблицы видно, что после учёта зависимостей итоговое ускорение определяется не только суммарной работой  $W$  и числом агентов  $P$ , но и длиной критического пути

$L$  — а сама длина  $L$  зависит от выбранной декомпозиции.

1. **Нижняя оценка  $W/P$  полезна, но оптимистична.** Для варианта 2 она равна 29.9 ч, а для варианта 3 — 13.4 ч. Эти числа достижимы только при достаточно независимых задачах и согласованных по времени запросах к человеку; ожидание в очереди увеличивает занятость потоков сверх  $W$ .
2. **В вариантах 2–4 результат определяет критический путь.** В вариантах 2 и 3 цепочка  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6$  задаёт время завершения 47.8 и 43.2 ч соответственно. После дробления тестов в варианте 4 критический путь сокращается до 36.7 ч, но остаётся активным ограничением. Поэтому достижимое ускорение составляет  $\times 1.59$ ,  $\times 1.76$  и  $\times 2.07$ , а не значения, рассчитанные в предположении полной независимости задач.
3. **Увеличения  $P$  самого по себе недостаточно.** При переходе от  $P = 2$  к  $P = 4$  оценка  $W/P$  для независимых задач резко снижается, но общее время меняется умеренно: дополнительные агенты не могут начать задачи критического пути до завершения их предшественников.
4. **Эффект декомпозиции выделен сравнением при прочих равных.** Переходы  $1 \rightarrow 2$  и  $2 \rightarrow 3$  одновременно меняют несколько параметров конфигурации. Только при переходе  $3 \rightarrow 4$  сохраняются  $P$ , режим,  $k$  и  $h$ , поэтому рост ускорения с  $\times 1.76$  до  $\times 2.07$  относится к изменению декомпозиции и не требует дополнительных ресурсов.
5. **Человеческий ресурс задаёт потолок и ограничивает расписание.** При уменьшении человеческой составляющей потолок  $1/\bar{h}_w$  возрастает:  $\times 1.17 \rightarrow \times 1.98 \rightarrow \times 4.00$ . В варианте 1 он уже достигнут, и процесс ограничен человеком. В вариантах 2–4 остаётся резерв, но использовать его можно только при явном планировании окон обслуживания. В перестроенном варианте 2 агент задачи 7 заблокирован в очереди 22.2 ч, однако это не увеличивает makespan, поскольку задача находится вне критического пути.
6. **Умеренные затраты на переключение не меняют ранжирование примера.** При  $\gamma(P) = 1 + 0.1(P - 1)$  точный пересчёт с уточнённой временной сеткой даёт (часы):

| вариант | $W_4$  | $L_4$  | $\gamma H$ | $B_4$  | $T^*$ |
|---------|--------|--------|------------|--------|-------|
| 2       | 63.640 | 51.320 | 42.240     | 51.320 | 51.36 |
| 3       | 59.300 | 47.700 | 24.700     | 47.700 | 47.70 |
| 4       | 59.300 | 40.450 | 24.700     | 40.450 | 40.45 |

Поэтому сохраняется порядок  $4 < 3 < 2$ , а human-ветвь остаётся неактивной. При более консервативной оценке  $h_i = 0.5k_i$  в вариантах 3–4 получаем  $H = 26.8$  ч и  $\gamma(4)H \approx 34.8$  ч; для варианта 4 это уже близко к критическому пути. Следовательно, по мере сокращения  $L_4$  человеческий ресурс может стать активным ограничением.

## Implications of the Illustrative Scenario.

1. **Зависимости следует учитывать явно.** Балансировка независимых задач превышает ускорение, если работы связаны графом зависимостей. Для рассмотренного примера на уровне М4 нижняя оценка равна  $\max\{W/P, L, H\}$  при  $C(P) = \gamma(P) = 1$ .
2. **Критический путь стал главным ограничением.** В вариантах 2–4 время завершения совпадает с  $L$ . При этом состав критического пути зависит от гранулярности графа: после дробления задачи тестирования в его хвост входит только проверка повторных попыток, тогда как модульные и интеграционные тесты выполняются раньше и параллельно.
3. **Рост числа агентов не даёт пропорционального ускорения.** Если граф содержит крупные последовательно связанные задачи, увеличение  $P$  быстро перестаёт влиять на результат: часть агентов простаивает в ожидании обязательных предшественников.
4. **Декомпозиция даёт измеримый эффект.** Разбиение одной задачи тестирования сократило время завершения на 6.5 ч и повысило ускорение с  $\times 1.76$  до  $\times 2.07$  без изменения  $P$ ,  $k$  и  $h$ . Следующие кандидаты следует искать среди рёбер критического пути, которые связывают крупные задачи целиком, хотя фактическая зависимость относится лишь к одной подзадаче.
5. **Планировать необходимо не только агентов, но и участие человека.** Проверка делегированных задач выполняется одним разработчиком последовательно, поэтому соответствующие интервалы и очередь к ним должны быть явно включены в расписание уровня М4. Ожидающий агент сохраняет свой поток; передать ему другую задачу до приёма текущей нельзя. Длительная непрерывная интерактивная работа блокирует обслуживание параллельных результатов, а предельное ускорение ограничивается величиной  $1/\bar{h}_w$  независимо от числа агентов.

## C Additional Computational Results

### C.1 Additional Visual Results

Рисунок 6 показывает распределение разрывов между структурной границей, оптимумом и исходной политикой. Нулевой разрыв означает достижимость  $B_4$ , а не отсутствие очереди.

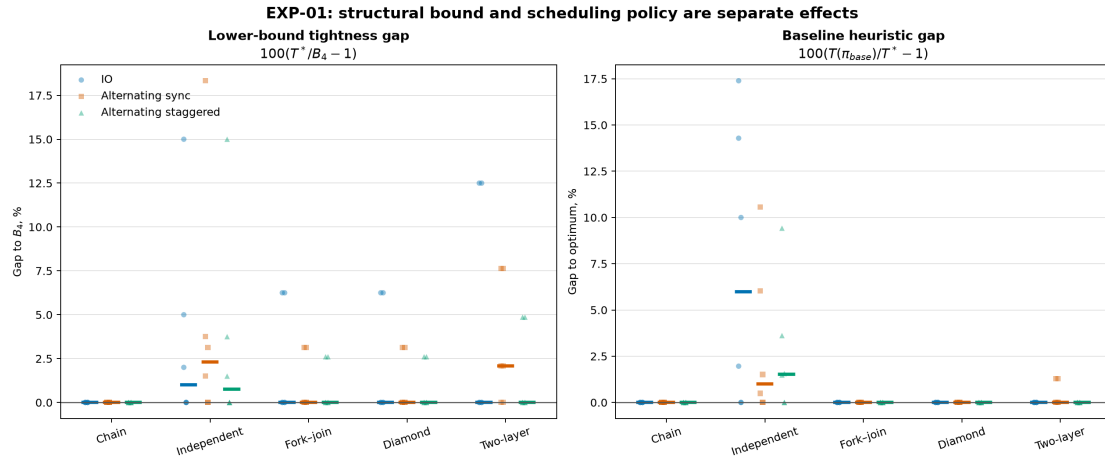


Рис. 6: EXP-01: разрывы  $T^*/B_4 - 1$  и разрывы исходной политики.

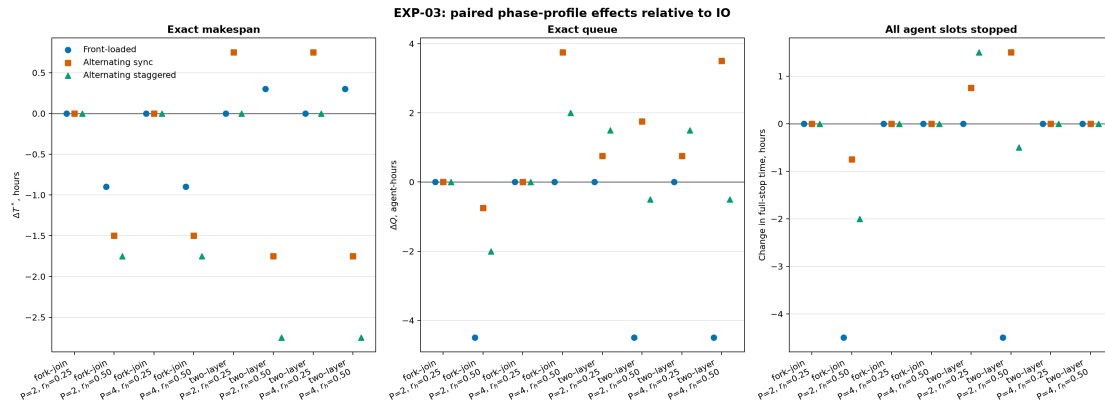


Рис. 7: EXP-03: изменение  $T^*$  при одинаковых агрегатах и разных фазовых профилях. Историческая метка sync здесь не означает синхронизацию wall-clock окон.

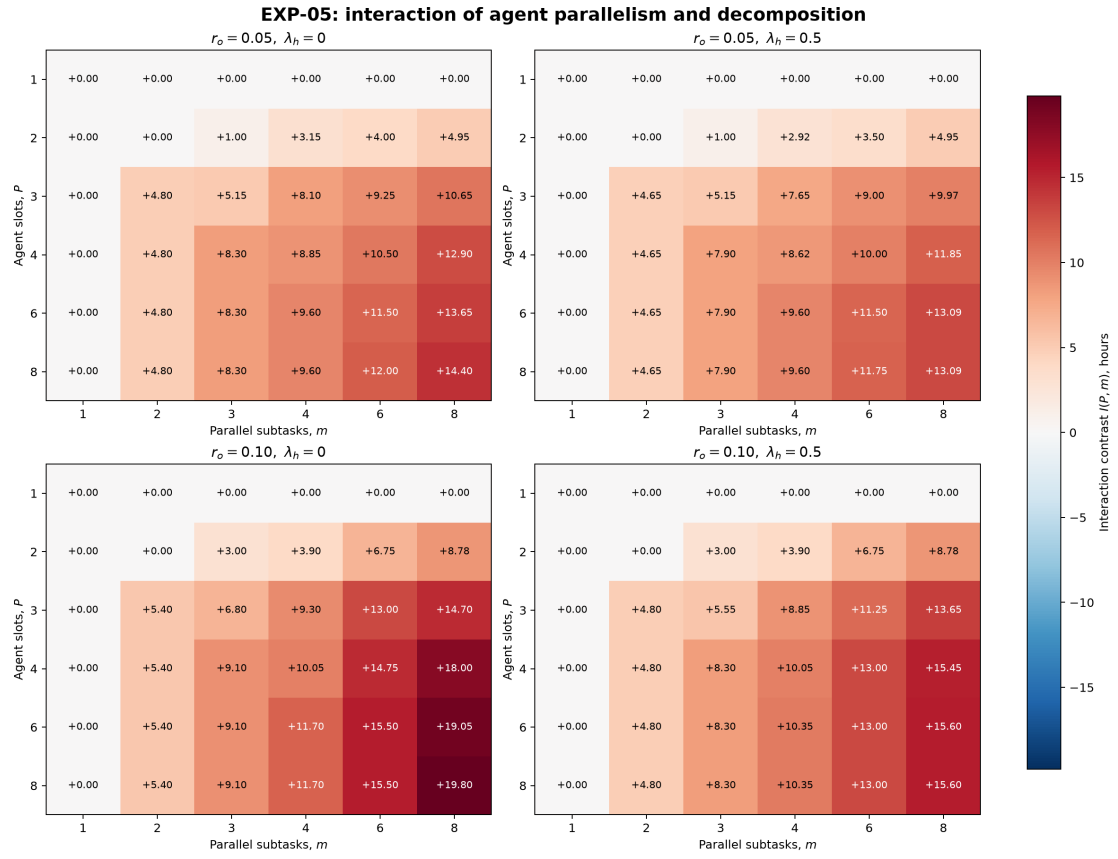


Рис. 8: EXP-05: взаимодействие доступного параллелизма и гранулярности на канонической точной сетке.



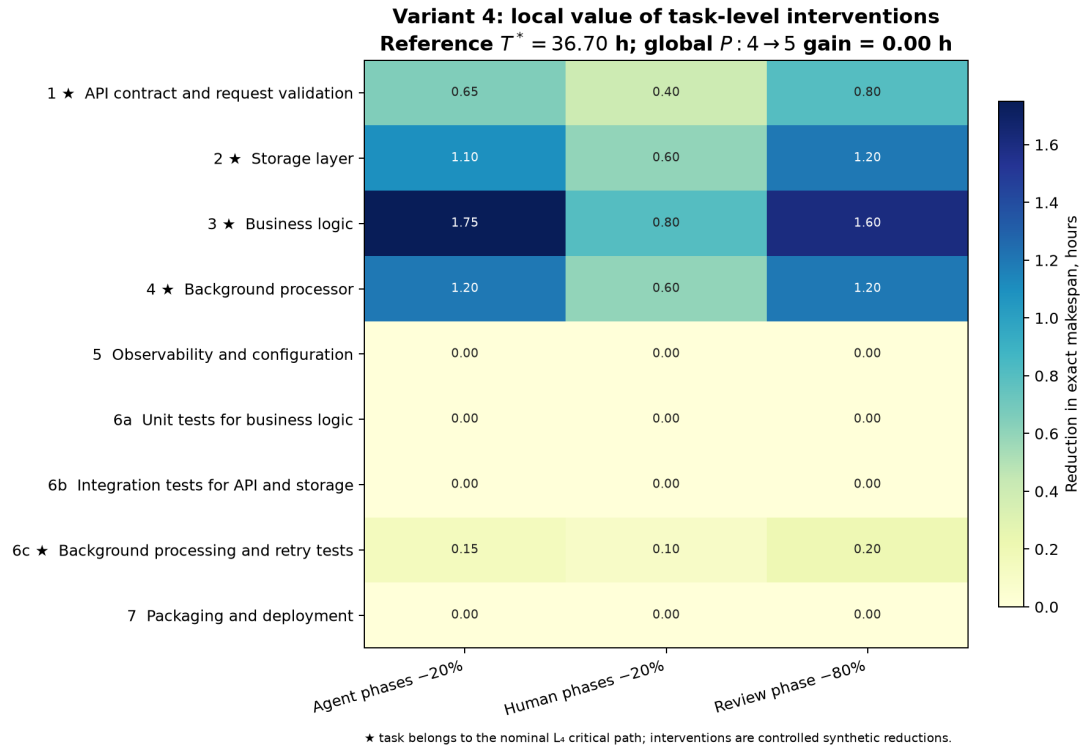


Рис. 9: Иллюстративная локальная чувствительность варианта 4. В ячейках показано сокращение доказанного  $T^*$  после контролируемого уменьшения фаз одной задачи; звёздочкой отмечены задачи номинального критического пути. Масштабы вмешательств синтетические и не являются эмпирическими оценками.

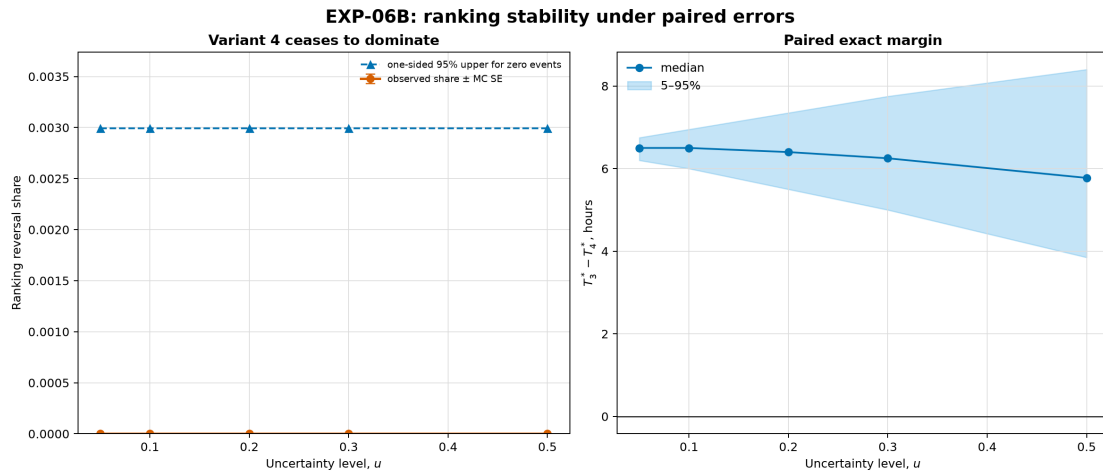


Рис. 10: EXP-06: устойчивость ранжирования иллюстративных вариантов внутри замороженных семейств ошибок.

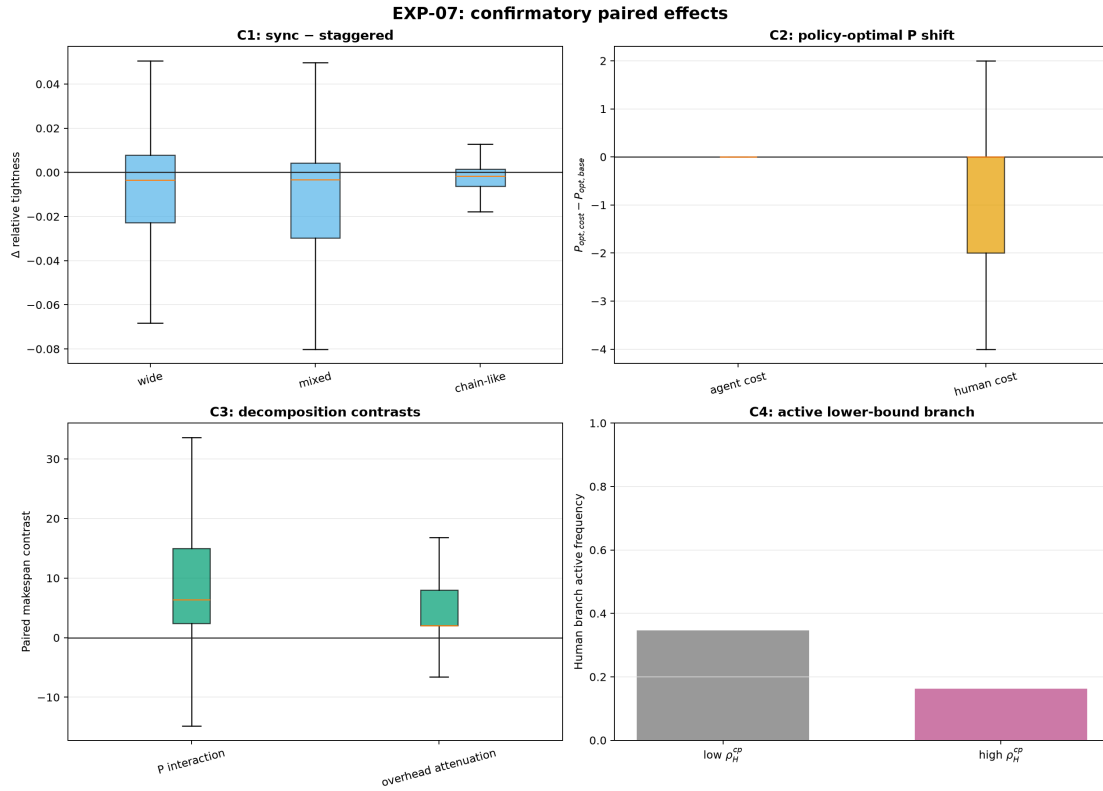


Рис. 11: EXP-07: эффекты на синтетических DAG. C1 и C4 не поддерживаются; C2 и C3 относятся преимущественно к фиксированной политике.

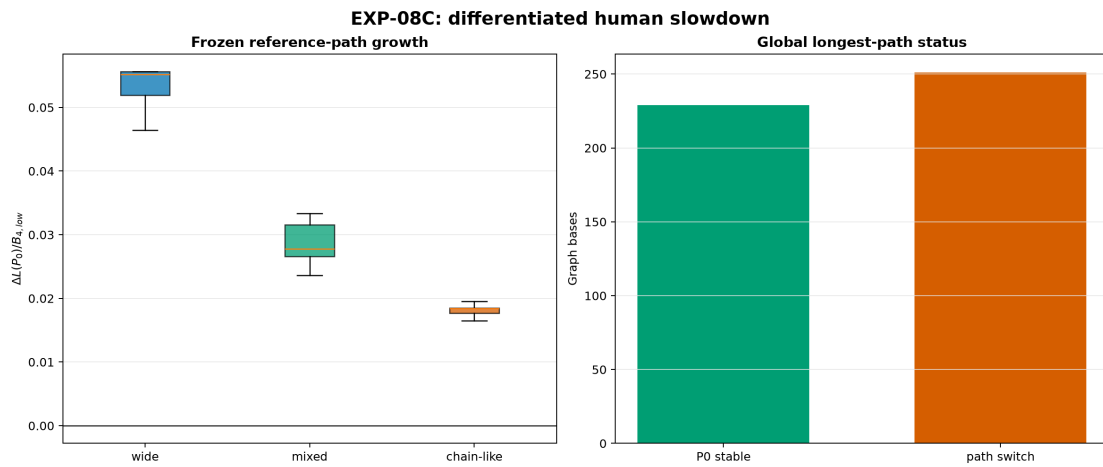


Рис. 12: EXP-08: изменение заранее выбранного пути при  $\gamma > C$  и переключения глобального критического пути.

## C.2 Reproducibility

Исходный код, замороженные входы и сохранённые результаты находятся в каталоге `experiments`. Из корня проекта полный набор команд имеет вид:

```
cd article/experiments
uv sync --python 3.13
```

```

uv run pytest -q
uv run python -m experiments validate scenarios/article
uv run python -m experiments run --experiment EXP-00
uv run python -m experiments run --experiment EXP-01
uv run python -m experiments run --experiment EXP-02
uv run python -m experiments run --experiment EXP-03
uv run python -m experiments run --experiment EXP-04
uv run python -m experiments run --experiment EXP-05
uv run python -m experiments run --experiment EXP-06
uv run python -m experiments run --experiment EXP-07
uv run python -m experiments run --experiment EXP-08 --stage development
uv run python -m experiments run --experiment EXP-08 --stage confirmatory
uv run python -m experiments figures --set candidates

```

EXP-00–EXP-07 независимы и могут повторяться по отдельности. Для EXP-08 порядок существенен: development создаёт `exp08_freeze.json`, а confirmatory-команда отказывается работать без совпадающего freeze и замороженной матрицы. Shortcut `--stage all` выполняет те же две стадии последовательно, но явные команды выше делают границу данных проверяемой.

| Путь                                            | Содержимое                                                                                    |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>scenarios/article/</code>                 | полностью заданные иллюстративные варианты EXP-00                                             |
| <code>scenarios/canonical/</code>               | замороженные матрицы EXP-01–EXP-08, исходный пилот EXP-01 и запись его глобального сокращения |
| <code>src/experiments/</code>                   | schema, validator, fixed policy, exact solver, генераторы серий и код рисунков                |
| <code>tests/</code>                             | проверки схемы, генерации серий, инвариантов и регрессии чисел статьи                         |
| <code>results/exp??_manifest.json</code>        | SHA-256 входов, реализации и выходов, версии зависимостей, settings и статусы                 |
| <code>results/exp??_report.md</code>            | полный дизайн, таблицы результатов, ограничения и ссылки на raw CSV                           |
| <code>results/full_experiments_report.md</code> | единая сводка EXP-00–EXP-08 и карта артефактов                                                |
| <code>figures/</code>                           | пары PNG/SVG и код их построения                                                              |

Основные raw-таблицы хранят параметры сценария,  $B_4$ , срок фиксированной политики, solver status, objective, best bound и gap; отдельные phase/event таблицы позволяют повторно проверить ресурсную допустимость. Manifests содержат commit Git и контрольные суммы перечисленных входов, реализации и выходов, поэтому изменение любого перечисленного файла обнаруживается до сопоставления результатов. Lockfile фиксирует пакеты; сведения об ОС и hardware исходных прогонов в manifests отсутствуют, что ограничивает воспроизводимость wall time, но не критерий OPTIMAL и сохранённые целочисленные objectives.

**Data and code availability.** Код лицензирован по MIT, а текст и исследовательские материалы — по CC BY 4.0. Публичный репозиторий, версионированный релиз и DOI будут указаны после архивирования версии препринта; состав дополнительного артефакта описан в Разделе 11.